CN LAB MANUAL

EXPERIMENT-1

1. Running and using services/commands like tcpdump, netstat, ifconfig, nslookup, FTP, TELNET and traceroute. Capture ping and trace route PDUs using a network protocol analyzer and examine.

To run all the above commands, we need to install the following command first:

```
root@JoshuaUbuntu:/home/joshua/Desktop# sudo apt install net-tools
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  net-tools
0 upgraded, 1 newly installed, 0 to remove and 46 not upgraded.
Need to get 204 kB of archives.
After this operation, 819 kB of additional disk space will be used.
Get:1 http://in.archive.ubuntu.com/ubuntu jammy/main amd64 net-tools amd64 1.60+git20181103.0eebece-1ubuntu5 [204 kB]
Fetched 204 kB in 2s (108 kB/s)
Selecting previously unselected package net-tools.
(Reading database ... 206342 files and directories currently installed.)
Preparing to unpack .../net-tools_1.60+git20181103.0eebece-1ubuntu5_amd64.deb ..
.
Unpacking net-tools (1.60+git20181103.0eebece-1ubuntu5) ...
Setting up net-tools (1.60+git20181103.0eebece-1ubuntu5) ...
Processing triggers for man-db (2.10.2-1) ...
```

1. ifconfig— Interface configurator.

ifconfig (interface configuration) command is used to configure the kernel-resident network interfaces.

- It is used at the boot time to set up the interfaces as necessary.
- After that, it is usually used when needed during debugging or when you need system tuning.
- Also, this command is used to assign the IP address and netmask to an interface or to enable or disable a given interface.

Syntax: ifconfig

- Displays information about all network interfaces currently in operation. The output resembles the following:

```
root@JoshuaUbuntu:/home/joshua/Desktop# ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
        inet6 fe80::2a72:eb24:1961:93  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:b8:37:09  txqueuelen 1000  (Ethernet)
        RX packets 1320  bytes 1592516 (1.5 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 626  bytes 65175 (65.1 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 183  bytes 16558 (16.5 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 183  bytes 16558 (16.5 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Here, **enp0s3**, **lo** are the names of the active network interfaces on the system.

- **enp0s3**is the first Ethernet interface. This type of interface is usually a NIC connected to the network by a category 5 cable.
- **lo** is the loopback interface. This is a special network interface that the system uses to communicate with itself.

Syntax: ifconfig [interface] [options]

Viewing the configuration of all interfaces

If you'd like to view the configuration of all network interfaces on the system (not just the ones that are currently active), you can specify the **-a** option, like this:

```
root@JoshuaUbuntu:/home/joshua/Desktop# ifconfig -a
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
        inet6 fe80::2a72:eb24:1961:93  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:b8:37:09  txqueuelen 1000  (Ethernet)
        RX packets 1350  bytes 1597027 (1.5 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 656  bytes 67558 (67.5 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 183  bytes 16558 (16.5 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 183  bytes 16558 (16.5 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

This produces output similar to running **ifconfig**, but if there are any inactive interfaces on the system, their configuration is also shown.

Viewing the configuration of a specific interface

To view the configuration of a specific interface, specify its name as an option. For instance,

```
root@JoshuaUbuntu:/home/joshua/Desktop# ifconfig enp0s3
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
        inet6 fe80::2a72:eb24:1961:93  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:b8:37:09  txqueuelen 1000  (Ethernet)
        RX packets 1334  bytes 1594529 (1.5 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 640  bytes 66292 (66.2 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

- Displays the configuration of device **enp0s3** only.

```
root@JoshuaUbuntu:/home/joshua/Desktop# ifconfig lo
lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 183  bytes 16558 (16.5 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 183  bytes 16558 (16.5 KB)
```

- Displays the configuration of device **lo** only.

Enabling and disabling an interface

- When a network interface is active, it can send and receive data; when it is inactive, it is not able to transmit or receive. You can use **ifconfig** to change the status of a network interface from inactive to active, or vice versa.
- To enable an inactive interface, provide **ifconfig** with the interface name followed by the keyword **up**.
- Enabling or disabling a device requires superuser permissions, so you either have to be logged in as root, or prefix your command with **sudo** to run it with superuser privileges.
- you can disable an active network interface using the **down** keyword.

```
root@JoshuaUbuntu:/home/joshua/Desktop# sudo ifconfig enp0s3 down
root@JoshuaUbuntu:/home/joshua/Desktop# ifconfig
lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 187  bytes 16935 (16.9 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 187  bytes 16935 (16.9 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

- if network interface **enp0s3** is inactive, you can activate it with the command:

```
root@JoshuaUbuntu:/home/joshua/Desktop# sudo ifconfig enp0s3 up
root@JoshuaUbuntu:/home/joshua/Desktop# ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
        inet6 fe80::2a72:eb24:1961:93  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:b8:37:09  txqueuelen 1000  (Ethernet)
        RX packets 1371  bytes 1601054 (1.6 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 708  bytes 73176 (73.1 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 187  bytes 16935 (16.9 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 187  bytes 16935 (16.9 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Ifconfig –s : Display a short list

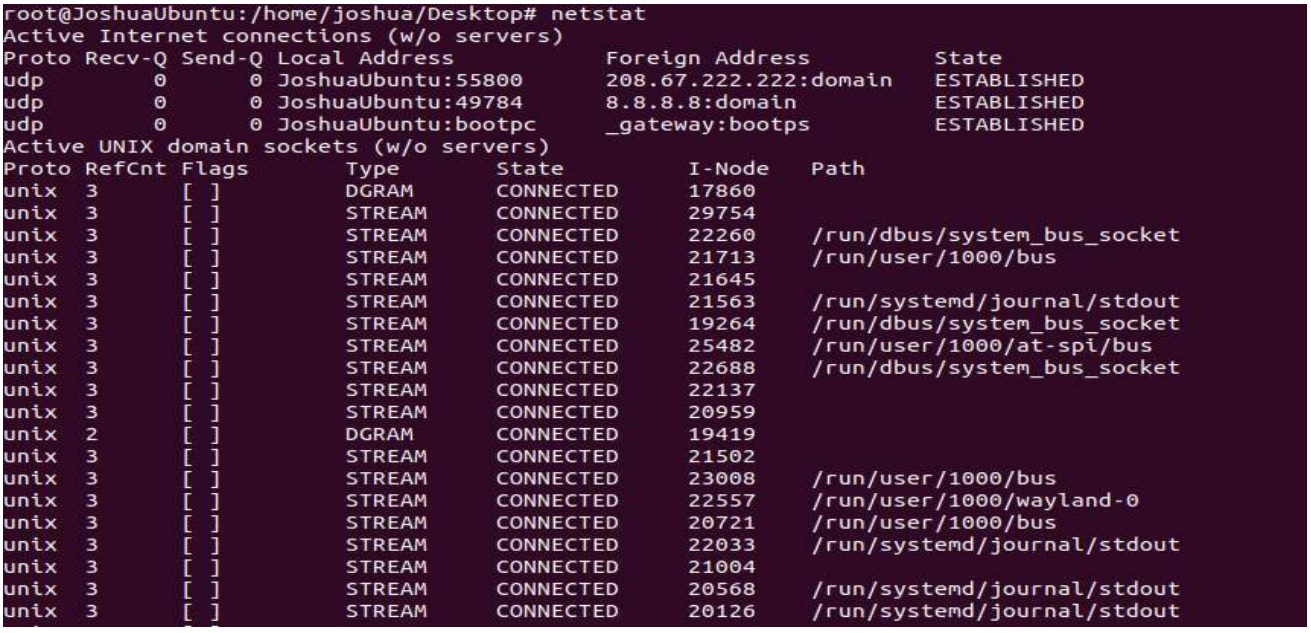
```
root@JoshuaUbuntu:/home/joshua/Desktop# ifconfig -s
Iface      MTU    RX-OK RX-ERR RX-DRP RX-OVR    TX-OK TX-ERR TX-DRP TX-OVR Flg
enp0s3    1500     1382      0      0 0           749      0      0      0 BMRU
lo       65536      260      0      0 0           260      0      0      0 LRU
```

Different Options:

-a	Display information for all network interfaces, even if they are down.
-s	Display a short list in a format identical to the command "**netstat -i**".
-v	Verbose mode; display additional information for certain error conditions.
interface	The name of the interface. This is usually a driver name followed by a unit number, for example "**eth0**" for the first Ethernet interface. If your kernel supports alias interfaces, you can specify them with **eth0:0** for the first alias of **eth0**. You can use them to assign a second address. To delete an alias interface, use **ifconfig eth0:0 down**. Note: for every scope (i.e., same net with address/netmask combination) all aliases are deleted, if you delete the first (primary).
up	This flag causes the interface to be activated. It is implicitly specified if an address is assigned to the interface.
down	This flag causes the driver for this interface to be shut down.

2. netstat

- **netstat** ("network statistics") is a command-line tool that displays network connections (both incoming and outgoing), routing tables, and many network interface (network interface controller or software-defined network interface) and network protocol statistics. It is available on Unix-like operating systems, including OS X, Linux, Solaris, and BSD, and on Windows NT-based operating systems, including Windows XP, Windows Vista, Windows 7 and Windows 8.
- It is used for finding problems in the network and to determine the amount of traffic on the network as a performance measurement.



Displays generic statistics about the network activity of the local system.

netstat –rn

- Displays the routing table for all IP addresses bound to the server.

```
root@JoshuaUbuntu:/home/joshua/Desktop# netstat -rn
Kernel IP routing table
Destination     Gateway         Genmask         Flags   MSS Window  irtt Iface
0.0.0.0         10.0.2.2        0.0.0.0         UG        0 0          0 enp0s3
10.0.2.0        0.0.0.0         255.255.255.0   U         0 0          0 enp0s3
169.254.0.0     0.0.0.0         255.255.0.0     U         0 0          0 enp0s3
```

The netstat -rn command in Unix-like operating systems displays the kernel routing table. Here's what each part of the output typically represents:

Destination:

- The destination network or IP address. This column shows the network or IP address to which the route corresponds.
- Each entry in the "Destination" column represents a network or IP address to which the system can route packets. These destinations may be local networks or remote networks accessible via gateways. The system uses this routing table to determine where to forward packets based on their destination IP addresses.

Gateway:

- The IP address of the next hop or gateway through which packets should be sent to reach the destination.

Genmask:

- The netmask associated with the destination network. It defines the network portion of the IP address.

Flags:

- Additional flags associated with the route, such as U (route is up), G (route is to a gateway), H (target is a host), and more.

Metric:

- The routing metric or cost associated with the route. It's used by the routing algorithm to determine the best path to a destination.

Ref:

- The reference count for the route. It indicates how many routes are using the same route entry.

Use:

- The number of times the route has been used.

Iface:

- The network interface associated with the route. It specifies the outgoing interface used for sending packets to the destination.

3. tcpdump:

- Tcpdump is a command line utility that allows you to capture and analyze network traffic going through your system. It is often used to help troubleshoot network issues, as well as a security tool.
- Check whether tcpdump is installed on your system with the following command:

```
joshua@JoshuaUbuntu:~/Desktop$ which tcpdump
/usr/bin/tcpdump
```

- If tcpdump is not installed, you can install it but using your distribution's package manager.

  $ sudo dnf install -y tcpdump

- To begin, use the command **tcpdump -D** to see which interfaces are available for capture:

```
joshua@JoshuaUbuntu:~/Desktop$ sudo tcpdump -D
1.enp0s3 [Up, Running, Connected]
2.any (Pseudo-device that captures on all interfaces) [Up, Running]
3.lo [Up, Running, Loopback]
4.bluetooth-monitor (Bluetooth Linux Monitor) [Wireless]
5.nflog (Linux netfilter log (NFLOG) interface) [none]
6.nfqueue (Linux netfilter queue (NFQUEUE) interface) [none]
7.dbus-system (D-Bus system bus) [none]
8.dbus-session (D-Bus session bus) [none]
```

- In the example above, you can see all the interfaces available in my machine. The special interface **any** allows capturing in any active interface.

Let's use it to start capturing some packets. Capture all packets in any interface by running this command:

```
joshua@JoshuaUbuntu:~/Desktop$ sudo tcpdump --interface any
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144
 bytes
12:26:11.329411 enp0s3 Out IP JoshuaUbuntu.43957 > dns.google.domain: 17357+ A?
connectivity-check.ubuntu.com. (47)
12:26:11.436137 lo    In  IP localhost.34152 > localhost.domain: 40960+ [1au] PT
R? 8.8.8.8.in-addr.arpa. (49)
12:26:11.436890 enp0s3 Out IP JoshuaUbuntu.33797 > dns.google.domain: 40281+ PTR
? 8.8.8.8.in-addr.arpa. (38)
12:26:16.440829 lo    In  IP localhost.34152 > localhost.domain: 40960+ [1au] PT
R? 8.8.8.8.in-addr.arpa. (49)
12:26:16.441253 enp0s3 Out IP JoshuaUbuntu.52915 > dns.sse.cisco.com.domain: 402
81+ [1au] PTR? 8.8.8.8.in-addr.arpa. (49)
12:26:16.441672 enp0s3 Out IP JoshuaUbuntu.60705 > dns.sse.cisco.com.domain: 173
57+ [1au] A? connectivity-check.ubuntu.com. (58)
12:26:16.465781 enp0s3 In  IP dns.sse.cisco.com.domain > JoshuaUbuntu.52915: 402
81 1/0/0 PTR dns.google. (62)
12:26:16.466287 enp0s3 Out IP JoshuaUbuntu.52915 > dns.sse.cisco.com.domain: 154
37+ PTR? 8.8.8.8.in-addr.arpa. (38)
```

Tcpdump continues to capture packets until it receives an interrupt signal. You can interrupt capturing by pressing Ctrl+C.

To limit the number of packets captured and stop tcpdump, use the **-c (for *count*)** option:

```
joshua@JoshuaUbuntu:~/Desktop$ sudo tcpdump -i any -c 5
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144
 bytes
13:21:11.330082 enp0s3 Out IP JoshuaUbuntu.51261 > dns.sse.cisco.com.domain: 544
43+ A? connectivity-check.ubuntu.com. (47)
13:21:11.358526 enp0s3 In  IP dns.sse.cisco.com.domain > JoshuaUbuntu.51261: 544
43 12/0/0 A 185.125.190.96, A 185.125.190.97, A 185.125.190.98, A 91.189.91.48,
A 91.189.91.49, A 91.189.91.96, A 91.189.91.97, A 91.189.91.98, A 185.125.190.17
, A 185.125.190.18, A 185.125.190.48, A 185.125.190.49 (239)
13:21:11.360533 enp0s3 Out IP JoshuaUbuntu.57970 > ubuntu-content-cache-1.ps5.ca
nonical.com.http: Flags [S], seq 34714273, win 64240, options [mss 1460,sackOK,T
S val 2178481724 ecr 0,nop,wscale 7], length 0
13:21:11.374330 lo    In  IP localhost.53901 > localhost.domain: 61068+ [1au] PT
R? 222.222.67.208.in-addr.arpa. (56)
13:21:11.374816 enp0s3 Out IP JoshuaUbuntu.53654 > dns.sse.cisco.com.domain: 547
08+ PTR? 222.222.67.208.in-addr.arpa. (45)
5 packets captured
29 packets received by filter
0 packets dropped by kernel
```

- Tcpdump is capable of capturing and decoding many different protocols, such as TCP, UDP, ICMP, and many more. let's explore the TCP packet.

```
13:29:41.429579 enp0s3 Out IP JoshuaUbuntu.60847 > dns.sse.cisco.com.domain: 423
48+ [1au] AAAA? connectivity-check.ubuntu.com. (58)
1 packet captured
```

- The first field, **13:29:41.429579**, represents the timestamp of the received packet as per the local clock.
- Next, **IP** represents the network layer protocol—in this case, **IPv4**. For **IPv6** packets, the value is **IP6**.
- The next field, is the **source IP address and port**. This is followed by the **destination IP address and port**, represented by.

To filter packets based on protocol, specifying the protocol in the command line. For example, capture ICMP packets only by using this command:

$ sudo tcpdump -i any -c5 icmp

$ sudo tcpdump -i any -c5 tcp

```
joshua@JoshuaUbuntu:~/Desktop$ sudo tcpdump -i any -c5 tcp
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144
 bytes
13:41:11.443600 enp0s3 Out IP JoshuaUbuntu.57466 > ubuntu-content-cache-1.ps6.ca
nonical.com.http: Flags [S], seq 3133246810, win 64240, options [mss 1460,sackOK
,TS val 2663612875 ecr 0,nop,wscale 7], length 0
13:41:11.669860 enp0s3 In  IP ubuntu-content-cache-1.ps6.canonical.com.http > Jo
shuaUbuntu.57466: Flags [S.], seq 2880001, ack 3133246811, win 65535, options [m
ss 1460], length 0
13:41:11.669956 enp0s3 Out IP JoshuaUbuntu.57466 > ubuntu-content-cache-1.ps6.ca
nonical.com.http: Flags [.], ack 1, win 64240, length 0
13:41:11.670633 enp0s3 Out IP JoshuaUbuntu.57466 > ubuntu-content-cache-1.ps6.ca
nonical.com.http: Flags [P.], seq 1:88, ack 1, win 64240, length 87: HTTP: GET /
 HTTP/1.1
13:41:11.671386 enp0s3 In  IP ubuntu-content-cache-1.ps6.canonical.com.http > Jo
shuaUbuntu.57466: Flags [.], ack 88, win 65535, length 0
5 packets captured
10 packets received by filter
0 packets dropped by kernel
```

4. nslookup

- Nslookup enables users to look up the IP address of a domain or host on a network.
- The **nslookup** command can also perform a reverse lookup using an IP address to find the domain or host associated with that IP address.

```
joshua@JoshuaUbuntu:~/Desktop$ nslookup mvsrec.edu.in
Server:         127.0.0.53
Address:        127.0.0.53#53

Non-authoritative answer:
Name:   mvsrec.edu.in
Address: 43.255.154.67
```

- **nslookup mvsrec.edu.in:** This is the command that you entered. It's asking the DNS (Domain Name System) server to look up the IP address associated with the domain name "mvsrec.edu.in".
- **Server: 127.0.0.53:** This line indicates the DNS server that was used for the lookup. In this case, the DNS server used is running locally on the machine, and its IP address is 127.0.0.53. The address 127.0.0.53 is a loopback address, commonly used for local testing and communication within the same device.
- **Address: 127.0.0.53#53:** This line specifies the IP address and port number of the DNS server. In this case, the IP address is again 127.0.0.53, and the port number is 53. Port 53 is the standard port for DNS communication.
- **Non-authoritative answer:** This line indicates that the response being provided is not from an authoritative DNS server for the domain "mvsrec.edu.in". Non-authoritative answers are typically cached responses provided by DNS servers that have previously looked up the domain.
- **Name: mvsrec.edu.in:** This line confirms the domain name being looked up.
- **Address: 43.255.154.67:** This line provides the IP address associated with the domain name "mvsrec.edu.in". In this case, the IP address is 43.255.154.67. This is the result of the DNS lookup, showing the IP address to which the domain resolves.

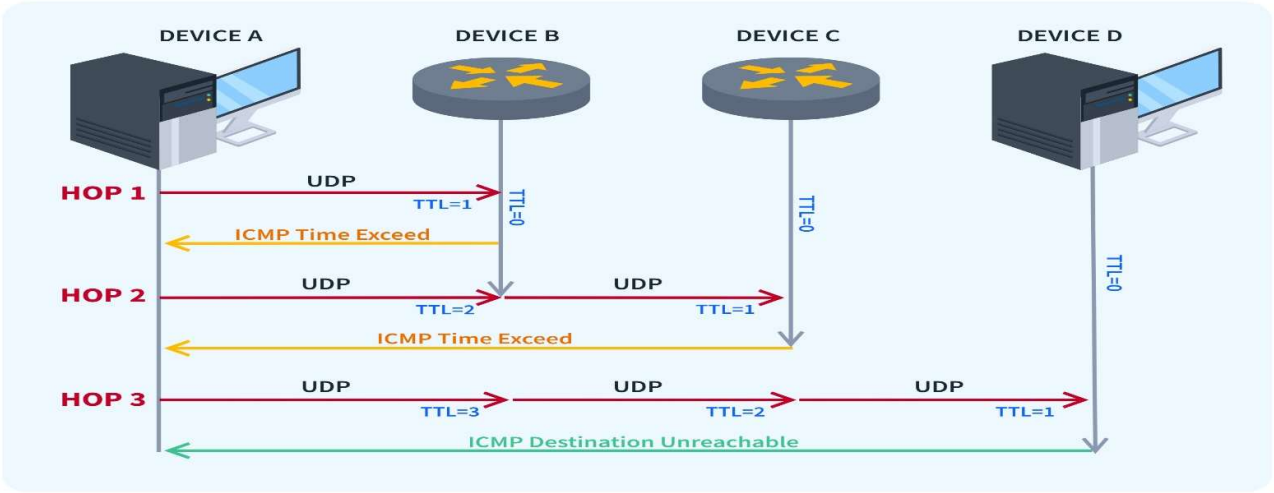
MX records, or Mail Exchanger records, are an essential part of the email routing process. They specify which server is responsible for handling mail for a particular domain.

Here's how you can use nslookup to query MX records:

```
joshua@JoshuaUbuntu:~/Desktop$ nslookup -type=mx mvsrec.edu.in
Server:         127.0.0.53
Address:        127.0.0.53#53

Non-authoritative answer:
mvsrec.edu.in   mail exchanger = 1 aspmx.l.google.com.
mvsrec.edu.in   mail exchanger = 5 alt1.aspmx.l.google.com.
mvsrec.edu.in   mail exchanger = 5 alt2.aspmx.l.google.com.
mvsrec.edu.in   mail exchanger = 10 alt3.aspmx.l.google.com.
mvsrec.edu.in   mail exchanger = 10 alt4.aspmx.l.google.com.
```

5. traceroute

- Traceroute is a Linux command-line utility used to trace the path network packets take from the source to a destination host or IP address.
- Traceroute command performs its diagnostic magic by sending ICMP echo packets, also known as ping packets, with increasing TTL (Time To Live) values. It initially starts with a TTL of 1, so the first router receives the packet, decrements the TTL to 0, and drops the packet while sending an "ICMP time exceeded" message back.
- Going forward, traceroute increments the TTL for each packet that it sends. This means the next router that receives these packets then decrements the TTL down to 0 and then also returns the time-exceeded message. This continues right up until the packet reaches the destination host being tested. Along the way, traceroute records each hop and provides the hostname, IP address, and round-trip time for each one.
- This maps out the path the packets take to reach the destination, giving us a better idea of the route taken to get there. It identifies all routers in between, which can help pinpoint network latency and bottlenecks, and sometimes even failed devices where the packet stops dead in its tracks.



Installing traceroute:

sudo apt install traceroute

```
joshua@JoshuaUbuntu:~/Desktop$ traceroute mvsrec.edu.in
traceroute to mvsrec.edu.in (43.255.154.67), 30 hops max, 60 byte packets
 1  _gateway (10.0.2.2)  2.405 ms  2.382 ms  2.363 ms
 2  * * *
 3  * * *
 4  * * *
 5  * * *
 6  * * *
 7  * * *
 8  * * *
 9  * * *
10  * * *
11  * * *
12  * * *
13  * * *
14  * * *
15  * * *
16  * * *
17  * * *
18  * * *
19  * * *
20  * * *
21  * * *
22  * * *
23  * * *
24  * * *
25  * * *
26  * * *
27  * * *
28  * * *
29  * * *
30  * * *
```

1 _gateway (10.0.2.2) 2.405ms 2.382ms 2.363ms: This line indicates the first hop in the traceroute, which is likely the gateway of your local network. Here's what each part of this line represents:

- **1**: Hop number.
- **_gateway**: The hostname of the first hop, which often represents your local router or gateway.
- **(10.0.2.2)**: The IP address of the first hop.
- **2.405ms 2.382ms 2.363ms**: Three round-trip times (RTT) measured in milliseconds for packets to travel from your computer to the first hop and back. These times indicate the latency experienced during communication with the first hop.
- **2 * * ***: This line indicates the second hop. However, the asterisks (*) suggest that there was no response from the second hop within the allotted time. This lack of response could be due to various reasons, such as a firewall blocking ICMP (Internet Control Message Protocol) packets, a misconfigured router, or network congestion.
- If the traceroute output continues with stars (*) until the maximum number of hops allowed (in this case, 30), it indicates that the traceroute was unable to successfully reach the destination within the specified number of hops.

Typical output:

```
traceroute to mvsrec.edu.in (43.255.154.67), 30 hops max, 60 byte packets
 1  192.168.1.1 (192.168.1.1)  1.234 ms  0.987 ms  0.765 ms
 2  10.0.0.1 (10.0.0.1)  3.456 ms  2.123 ms  2.567 ms
 3  203.0.113.1 (203.0.113.1)  5.678 ms  4.567 ms  4.321 ms
 4  203.0.113.2 (203.0.113.2)  7.654 ms  6.543 ms  6.789 ms
 5  43.255.154.67 (43.255.154.67)  8.987 ms  7.876 ms  8.765 ms
```

6. FTP

- **FTP (File Transfer Protocol)** is a network protocol used for transferring files from one computer system to another.

How to Use ftp Command in Linux

- The **ftp** command connects a computer system to a remote server using the FTP protocol. Once connected, it also lets users transfer files between the local machine and the remote system, and manage files and directories on the remote system.

Establish an FTP Connection

- To establish an FTP connection to a remote system, use the **ftp** command with the remote system's IP address:

  ftp [IP]

- For instance, connecting to a remote server with the IP address *192.168.100.9*:

  ftp 192.168.100.9

Log into the FTP Server

- Once you initiate a connection to a remote system using the **ftp** command, the FTP interface requires you to enter a username and password to log in:

```
phoenixnap@test-system:~$ ftp 192.168.100.9
Connected to 192.168.100.9.
220 (vsFTPd 3.0.3)
Name (192.168.100.9:phoenixnap): phoenixnap
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

- Entering the required credentials logs you in and starts the FTP interface. In this example, we are logging in as the *phoenixnap* user:

```
phoenixnap@test-system:~$ ftp 192.168.100.9
Connected to 192.168.100.9.
220 (vsFTPd 3.0.3)
Name (192.168.100.9:phoenixnap): phoenixnap
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

- The FTP interface is now active and ready to execute commands:

```
phoenixnap@test-system:~$ ftp 192.168.100.9
Connected to 192.168.100.9.
220 (vsFTPd 3.0.3)
Name (192.168.100.9:phoenixnap): phoenixnap
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

7. telnet

- A terminal emulation that lets users connect to a remote host or device using a telnet client, usually over port 23.
- For example, typing **telnet hostname** connects a user to a hostname named **hostname**.
- **Telnet** lets users manage an account or device remotely. For example, a user may telnet into a computer that hosts their website to manage their files remotely.

Note: Telnet is considered insecure because it transfers all data in clear text. If a user was sniffing a network, they could grab your username and password as they were transmitted. Users concerned about transmitting data securely should use SSH (secure shell) instead of telnet.

The primary use of the telnet command is to connect to a remote server. Here's an example:

```
joshua@JoshuaUbuntu:~/Desktop$ telnet google.com 80
Trying 142.250.183.110...
Connected to google.com.
Escape character is '^]'.
```

- In this example, we're using telnet to connect to **google.com on port 80**, which is often used for **HTTP**. The output shows that the connection was successful.
- This basic use of the telnet command is straightforward, but it's also very powerful. By simply changing the host and port, you can connect to a wide variety of servers and services.

- One of the most common uses of the telnet command is for network troubleshooting. For example, you can use telnet to check if a specific port is open on a remote server.

```
joshua@JoshuaUbuntu:~/Desktop$ telnet google.com 80
Trying 142.250.183.110...
Connected to google.com.
Escape character is '^]'.
```

In this example, we're using telnet to connect to google.com on port 80. The output shows that the connection was successful, which means the port is open.

- You can also use the telnet command to connect to a mail server and interact with it. This can be useful for testing the server or diagnosing issues.

```
joshua@JoshuaUbuntu:~/Desktop$ telnet smtp.gmail.com 25
Trying 172.217.194.109...
Connected to smtp.gmail.com.
Escape character is '^]'.
220 smtp.gmail.com ESMTP m10-20020a17090b068a00b0029f349cc253sm4663255pjz.54 - g
smtp
```

In this example, we're using telnet to connect to smtp.gmail.com on port 25, which is the standard port for SMTP. The output shows that the connection was successful and the server is ready to accept commands.

8. ping

Allows you to test the reachability of a host on an Internet Protocol (IP) network and to measure the round-trip time for messages sent from the originating host to a destination computer.

To use the ping command in Linux, you simply type 'ping' followed by the IP address or domain name of the server you want to test.

The complete syntax would be,

ping [option] domain_or_ip.

```
joshua@JoshuaUbuntu:~/Desktop$ ping google.com
PING google.com (142.250.183.110) 56(84) bytes of data.
64 bytes from bom12s13-in-f14.1e100.net (142.250.183.110): icmp_seq=1 ttl=115 ti
me=38.6 ms
64 bytes from bom12s13-in-f14.1e100.net (142.250.183.110): icmp_seq=2 ttl=115 ti
me=30.9 ms
64 bytes from bom12s13-in-f14.1e100.net (142.250.183.110): icmp_seq=3 ttl=115 ti
me=39.7 ms
64 bytes from bom12s13-in-f14.1e100.net (142.250.183.110): icmp_seq=4 ttl=115 ti
me=34.7 ms
64 bytes from bom12s13-in-f14.1e100.net (142.250.183.110): icmp_seq=5 ttl=115 ti
me=30.5 ms
64 bytes from bom12s13-in-f14.1e100.net (142.250.183.110): icmp_seq=6 ttl=115 ti
me=30.7 ms
64 bytes from bom12s13-in-f14.1e100.net (142.250.183.110): icmp_seq=7 ttl=115 ti
me=36.4 ms
^C
--- google.com ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6008ms
rtt min/avg/max/mdev = 30.487/34.503/39.733/3.605 ms
```

```
joshua@JoshuaUbuntu:~/Desktop$ ping mvsrec.edu.in
PING mvsrec.edu.in (43.255.154.67) 56(84) bytes of data.
64 bytes from 67.154.255.43.host.secureserver.net (43.255.154.67): icmp_seq=1 tt
l=45 time=73.4 ms
64 bytes from 67.154.255.43.host.secureserver.net (43.255.154.67): icmp_seq=2 tt
l=45 time=76.5 ms
64 bytes from 67.154.255.43.host.secureserver.net (43.255.154.67): icmp_seq=3 tt
l=45 time=70.4 ms
64 bytes from 67.154.255.43.host.secureserver.net (43.255.154.67): icmp_seq=4 tt
l=45 time=68.9 ms
64 bytes from 67.154.255.43.host.secureserver.net (43.255.154.67): icmp_seq=5 tt
l=45 time=76.0 ms
^C
--- mvsrec.edu.in ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4005ms
rtt min/avg/max/mdev = 68.906/73.049/76.538/2.998 ms
```

```
joshua@JoshuaUbuntu:~/Desktop$ ping 43.255.154.67
PING 43.255.154.67 (43.255.154.67) 56(84) bytes of data.
64 bytes from 43.255.154.67: icmp_seq=1 ttl=45 time=66.8 ms
64 bytes from 43.255.154.67: icmp_seq=2 ttl=45 time=70.9 ms
64 bytes from 43.255.154.67: icmp_seq=3 ttl=45 time=70.3 ms
64 bytes from 43.255.154.67: icmp_seq=4 ttl=45 time=80.5 ms
64 bytes from 43.255.154.67: icmp_seq=5 ttl=45 time=96.9 ms
64 bytes from 43.255.154.67: icmp_seq=6 ttl=45 time=69.6 ms
64 bytes from 43.255.154.67: icmp_seq=7 ttl=45 time=94.4 ms
64 bytes from 43.255.154.67: icmp_seq=8 ttl=45 time=73.8 ms
64 bytes from 43.255.154.67: icmp_seq=9 ttl=45 time=69.1 ms
64 bytes from 43.255.154.67: icmp_seq=10 ttl=45 time=74.9 ms
64 bytes from 43.255.154.67: icmp_seq=11 ttl=45 time=69.8 ms
^C
--- 43.255.154.67 ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 10014ms
rtt min/avg/max/mdev = 66.752/76.078/96.880/9.869 ms
```

EXPERIMENT-2

Implement programs to illustrate socket programming using UDP and TCP

Introduction to Socket Programming:

- A socket is a **software interface/communication endpoint** that enables communication between two computers over a network.
- It allows **data to be exchanged between processes** running on different hosts.
- Sockets are used extensively in **network programming** to establish connections, transmit data, and receive data.
- Sockets can be implemented in C, C++, Java, Python etc.

Key features of sockets include:

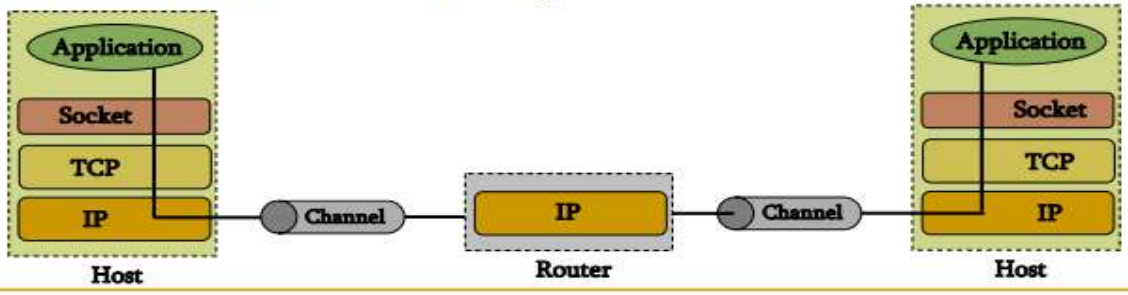
- **Endpoint Identification**: A socket is identified by a combination of an IP address and a port number. This combination uniquely identifies a communication endpoint on a network.
- **Communication Protocols**: Sockets can use various communication protocols such as Transmission Control Protocol (TCP) or User Datagram Protocol (UDP). TCP provides reliable, connection-oriented communication, while UDP offers faster, connectionless communication.
- **Socket Types**: There are different types of sockets, including stream sockets and datagram sockets. Stream sockets provide a reliable, bidirectional, byte-stream communication channel, while datagram sockets offer connectionless, unreliable communication using discrete messages.
- **APIs**: Sockets are typically used through programming interfaces provided by the operating system or network libraries. These APIs allow developers to create, bind, connect, send, and receive data through sockets.
- **Applications**: Sockets are widely used in various networked applications such as web browsers, email clients, file transfer protocols, online gaming, and more. They enable processes running on different computers to communicate and exchange data seamlessly over a network.

Berkley Sockets

- Berkeley sockets, also known as BSD sockets, are a programming interface for network socket communication. They originated from the **University of California, Berkeley**, in the 1980s as part of the Berkeley Software Distribution (BSD) UNIX operating system.
- Berkeley sockets provide a set of system calls, functions, and data structures for creating, configuring, and managing network sockets in Unix-like operating systems.
- They offer a standardized interface for network programming, allowing developers to write networked applications that can communicate over the Internet.

Berkley Sockets

- Universally known as Sockets
- It is an abstraction through which an application may send and receive data
- Provide generic access to interprocess communication services
  - e.g. IPX/SPX, Appletalk, TCP/IP
- Standard API for networking



- When we **combine IP Address and Port Number**, we get a unique identifier for a Socket. This combination is referred to as **SOCKET ADDRESS** or **END POINT**.
  - Uniquely identified by
    - an internet address
    - an end-to-end protocol (e.g. TCP or UDP)
    - a port number

Socket Types:

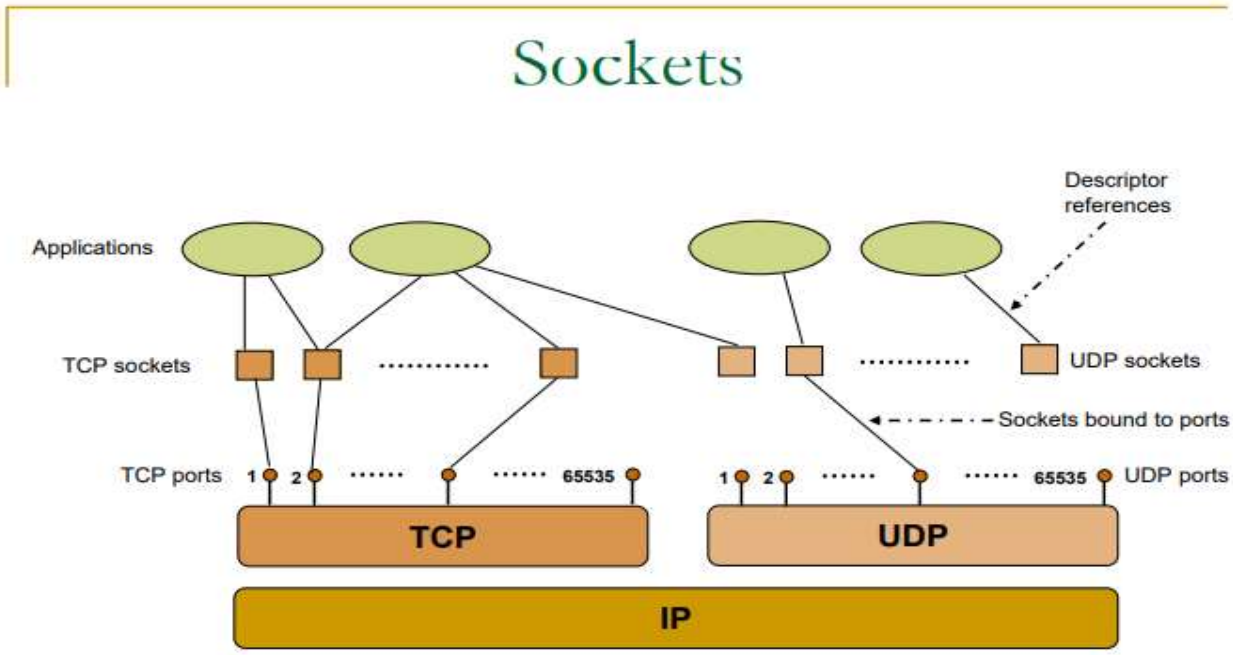
- **Stream sockets** and **datagram sockets** are **two types of communication** mechanisms provided by the sockets API for network communication. They have different characteristics and are suitable for different types of communication.

Stream Sockets (SOCK_STREAM):

- Stream sockets provide a **reliable**, **connection-oriented**, **byte-stream** communication between two endpoints.
  - In a byte-stream communication, data is treated as a continuous stream of bytes with no specific message boundaries.
  - This means that when using stream sockets, data is transmitted as a continuous flow of bytes, and the receiver must interpret where one message ends and the next begins based on the protocol or application layer semantics.
- They use the **Transmission Control Protocol (TCP)** as the underlying protocol.
- Stream sockets **guarantee the delivery of data** in the order it was sent and ensure that no data is lost or duplicated.
- They are **suitable for applications** where the **integrity and reliability of the data are crucial**, such as file transfer, email, web browsing, and real-time streaming.
- Examples of APIs for stream sockets include the **socket(), connect(), bind(), listen(), and accept()** functions in the sockets API.

Datagram Sockets (SOCK_DGRAM):

- Datagram sockets provide an **unreliable, connectionless, message-oriented communication** between two endpoints.
    - Datagram sockets have distinct message boundaries.
    - Datagram communication, also known as message-oriented communication, is a type of communication where data is transmitted in discrete chunks called datagrams.
    - In datagram communication, each datagram is treated as an independent message with its own boundaries.
- They use the **User Datagram Protocol (UDP)** as the underlying protocol.
- Datagram sockets **do not guarantee the delivery of data**, nor do they guarantee the order of delivery.
- They are suitable for applications where **real-time communication** or **low-latency transmission** is **more critical than reliability**, such as **online gaming**, **multimedia streaming, DNS queries**, and network monitoring.
- Examples of APIs for datagram sockets include the **socket() and sendto()** functions for sending data, and the **recvfrom()** function for receiving data in the sockets API.



Client-Server communication:

- The client-server model is one of the most used communication paradigms in networked systems.
- Clients normally communicates with one server at a time.
- From a server's perspective, at any point in time, it is not unusual for a server to be communicating with multiple clients.
- Client need to know of the existence of and the address of the server, but the server does not need to know the address of (or even the existence of) the client prior to the connection being established
- Client and servers communicate by means of multiple layers of network protocols.

Two scenarios exist:

- The scenario of the client and the server on the same local network (usually called LAN, Local Area Network)
- The client and the server may be in different LANs, with both LANs connected to a Wide Area Network (WAN) by means of *routers*. The largest WAN is the Internet, but companies may have their own WANs.

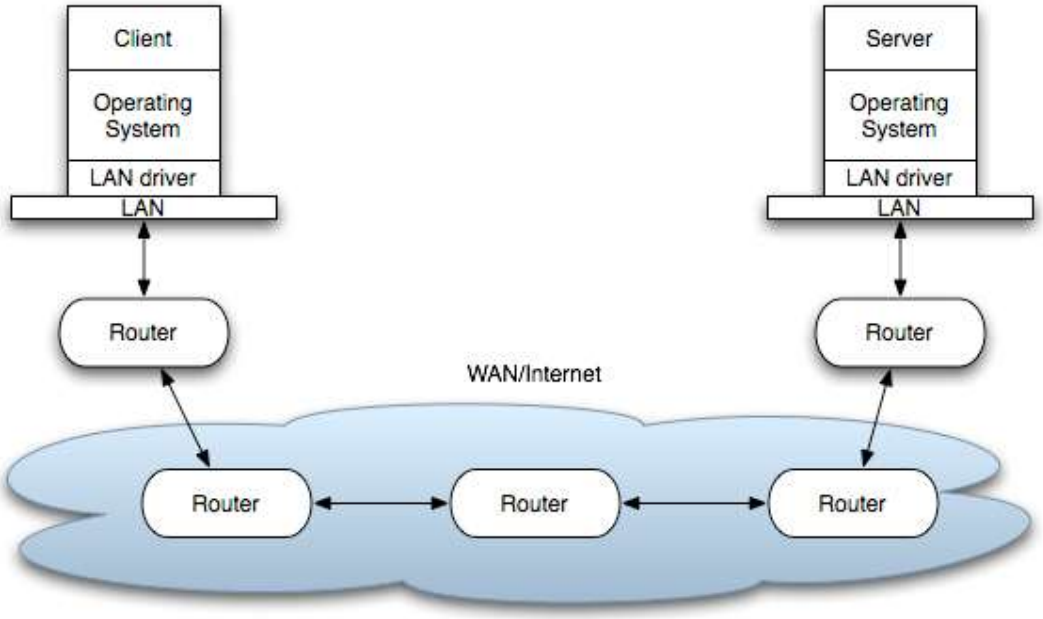


Figure: Client and server on different LANs connected through WAN/Internet.

The flow of information between the client and the server goes down the protocol stack on one side, then across the network and then up the protocol stack on the other side.

PASSIVE SOCKET(Server)	ACTIVE SOCKET(Client)
Also Known as Listening Socket	Also Known as Connected Socket
Created by Server to listen for incoming connection requests from clients	Created by client to establish connection with server

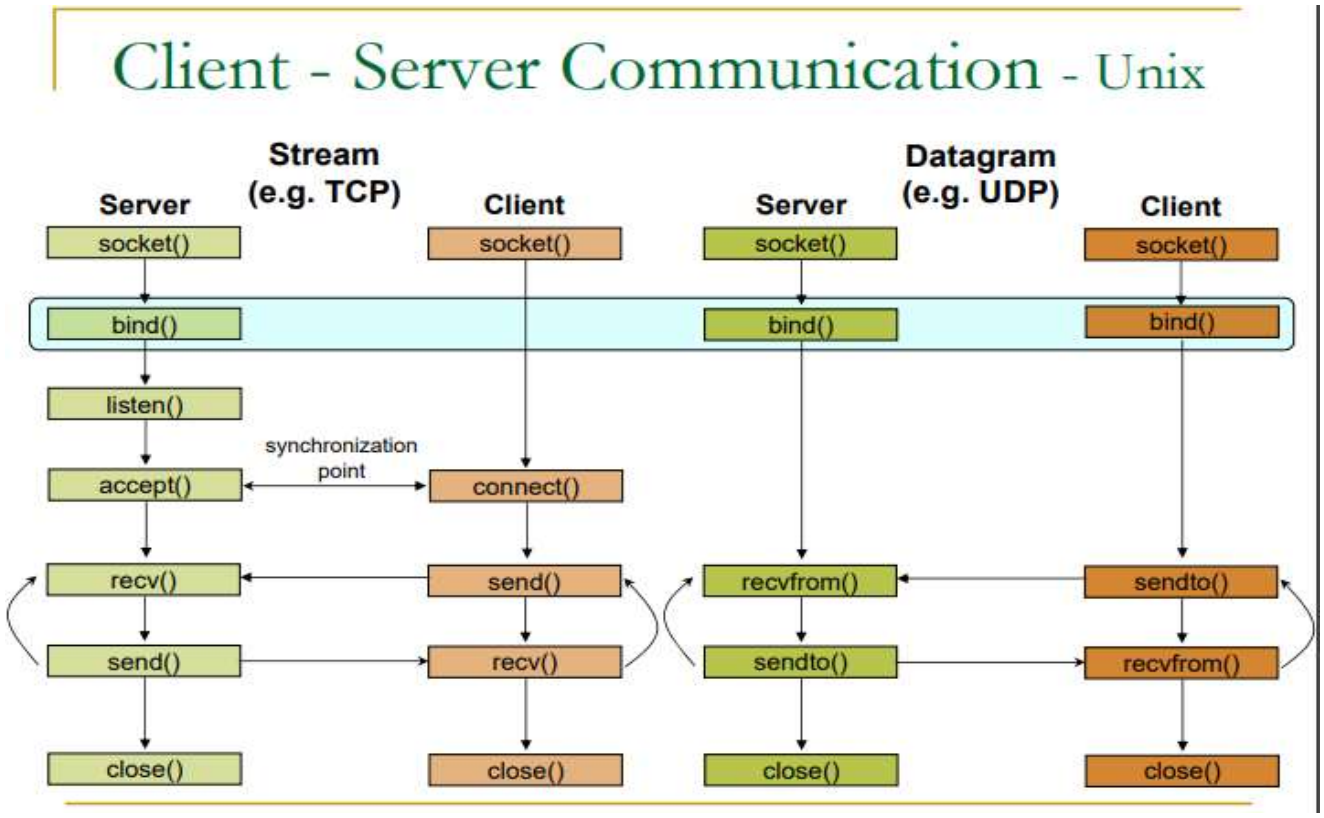
The Socket Primitives

- Socket primitives, also known as socket system calls, are low-level functions provided by the operating system's socket API for creating, configuring, and managing network sockets. These primitives are used by applications to establish network connections, send and receive data, and perform other networking tasks.
- Also known as elementary socket system calls

Primitive	Meaning
Socket	Create a new communication endpoint
Bind	Attach a local address to a socket
Listen	Announce willingness to accept connections
Accept	Block caller until a connection request arrives
Connect	Actively attempt to establish a connection
Send	Send some data over the connection
Receive	Receive some data over the connection
Close	Release the connection

Server side socket primitives:

socket(), bind(), listen(), accept(), recv()/recvfrom(), send()/sendto(), close()



socket():

- The SOCKET primitive creates a new endpoint and allocates table space for it within the transport entity.
- The parameter includes
    1. The Type of addressing format to be used (Unix/Internet),
    2. The type of service desired (Byte stream or Message stream) and
    3. The protocol (TCP/UDP).
- The function is defined as follows:

```
#include <sys/socket.h>

int socket (int family, int type, int protocol);
```

The function returns a non-negative integer number, similar to a file descriptor, that we define *socket descriptor* or -1 on error.

- `int sockid = socket(family, type, protocol);`
    - **sockid**: socket descriptor, an integer (like a file-handle)
    - **family**: integer, communication domain, e.g.,
        - PF_INET, IPv4 protocols, Internet addresses (typically used)
        - PF_UNIX, Local communication, File addresses
    - **type**: communication type
        - SOCK_STREAM - reliable, 2-way, connection-based service
        - SOCK_DGRAM - unreliable, connectionless, messages of maximum length
    - **protocol**: specifies protocol
        - IPPROTO_TCP IPPROTO_UDP
        - usually set to 0 (i.e., use default protocol)
    - upon failure returns -1

☞ NOTE: socket call does not specify where data will be coming from, nor where it will be going to – it just creates the interface!

bind():

- Newly created sockets do not have network addresses.
- This primitive associates a socket with a specific local address and port number.
- It is typically used by servers to bind a socket to a well-known address and port, enabling clients to connect to it.
- Once a server has bound an address to a socket, remote clients can connect to it.

```
#include <sys/socket.h>

int bind(int sockfd, const struct sockaddr *servaddr, socklen_t addrlen);
```

1. where sockfd is the socket descriptor,
2. servaddr is a pointer to a protocol-specific address and
3. addrlen is the size of the address structure.

Specifying Addresses

- Socket API defines a **generic** data type for addresses:

```
struct sockaddr {
   unsigned short sa_family;  /* Address family (e.g. AF_INET) */
   char sa_data[14];          /* Family-specific address information */
}
```

- Particular form of the sockaddr used for **TCP/IP** addresses:

```
struct in_addr {
   unsigned long s_addr;          /* Internet address (32 bits) */
}
struct sockaddr_in {
   unsigned short sin_family;     /* Internet protocol (AF_INET) */
   unsigned short sin_port;       /* Address port (16 bits) */
   struct in_addr sin_addr;     /* Internet address (32 bits) */
   char sin_zero[8];              /* Not used */
}
```

☞ **Important**: sockaddr_in can be casted to a sockaddr

- bind() returns 0 if it succeeds, -1 on error.

- `int status = bind(sockid, &addrport, size);`
  - sockid: integer, socket descriptor
  - addrport: struct sockaddr, the (IP) address and port of the machine
    - for TCP/IP server, internet address is usually set to INADDR_ANY, i.e., chooses any incoming interface
  - size: the size (in bytes) of the addrport structure
  - **status**: upon failure -1 is returned

listen():

- The listen() function converts an unconnected socket into a passive socket, indicating that the kernel should accept incoming connection requests directed to this socket. It is defined as follows:

```
#include <sys/socket.h>

int listen(int sockfd, int backlog);
```

where

1. sockfd is the socket descriptor and
2. backlog is the maximum number of connections the kernel should queue for this socket.

- The backlog argument provides a hint to the system of the number of outstanding connect requests that it should enqueue on behalf of the process.
- Once the queue is full, the system will reject additional connection requests. The backlog value must be chosen based on the expected load of the server.
- The function listen() return 0 if it succeeds, -1 on error.

- Instructs TCP protocol implementation to listen for connections

- `int status = listen(sockid, queueLimit);`
  - **sockid**: integer, socket descriptor
  - **queuelen**: integer, # of active participants that can "wait" for a connection
  - **status**: 0 if listening, -1 if error
- `listen()` is **non-blocking**: returns immediately

- The listening socket (sockid)
  - is never used for sending and receiving
  - is used by the server only as a way to get new sockets

accept():

- The accept() is used to retrieve a connect request and convert that into a request. It is defined as follows:

```
#include <sys/socket.h>

int accept(int sockfd, struct sockaddr *cliaddr,
socklen_t *addrlen);
```

where

1. sockfd is a new file descriptor that is connected to the client that called the connect().
2. The cliaddr and addrlen arguments are used to return the protocol address of the client.

- The new socket descriptor has the same socket type and address family of the original socket.
- The original socket passed to accept() is not associated with the connection, but instead remains available to receive additional connect requests.
- The kernel creates one connected socket for each client connection that is accepted.

Incoming Connection: accept()

- The server gets a socket for an incoming client connection by calling accept()
- `int s = accept(sockid, &clientAddr, &addrLen);`
  - s: integer, the new socket (used for data-transfer)
  - sockid: integer, the orig. socket (being listened on)
  - clientAddr: struct sockaddr, address of the active participant
    - filled in upon return
  - addrLen: sizeof(clientAddr): value/result parameter
    - must be set appropriately before call
    - adjusted upon return
- accept()
  - is **blocking**: waits for connection before returning
  - dequeues the next connection on the queue for socket (sockid)

send(), recv():

Exchanging data with stream socket

- `int count = send(sockid, msg, msgLen, flags);`
  - msg: const void[], message to be transmitted
  - msgLen: integer, length of message (in bytes) to transmit
  - flags: integer, special options, usually just 0
  - count: # bytes transmitted (-1 if error)
- `int count = recv(sockid, recvBuf, bufLen, flags);`
  - recvBuf: void[], stores received bytes
  - bufLen: # bytes received
  - flags: integer, special options, usually just 0
  - count: # bytes received (-1 if error)
- Calls are **blocking**
  - returns only after data is sent / received

sendto(), recvfrom():

Exchanging data with datagram socket

- `int count = sendto(sockid, msg, msgLen, flags, &foreignAddr, addrlen);`
  - msg, msgLen, flags, count: same with send()
  - foreignAddr: struct sockaddr, address of the destination
  - addrLen: sizeof(foreignAddr)
- `int count = recvfrom(sockid, recvBuf, bufLen, flags, &clientAddr, addrlen);`
  - recvBuf, bufLen, flags, count: same with recv()
  - clientAddr: struct sockaddr, address of the client
  - addrLen: sizeof(clientAddr)
- Calls are **blocking**
  - returns only after data is sent / received

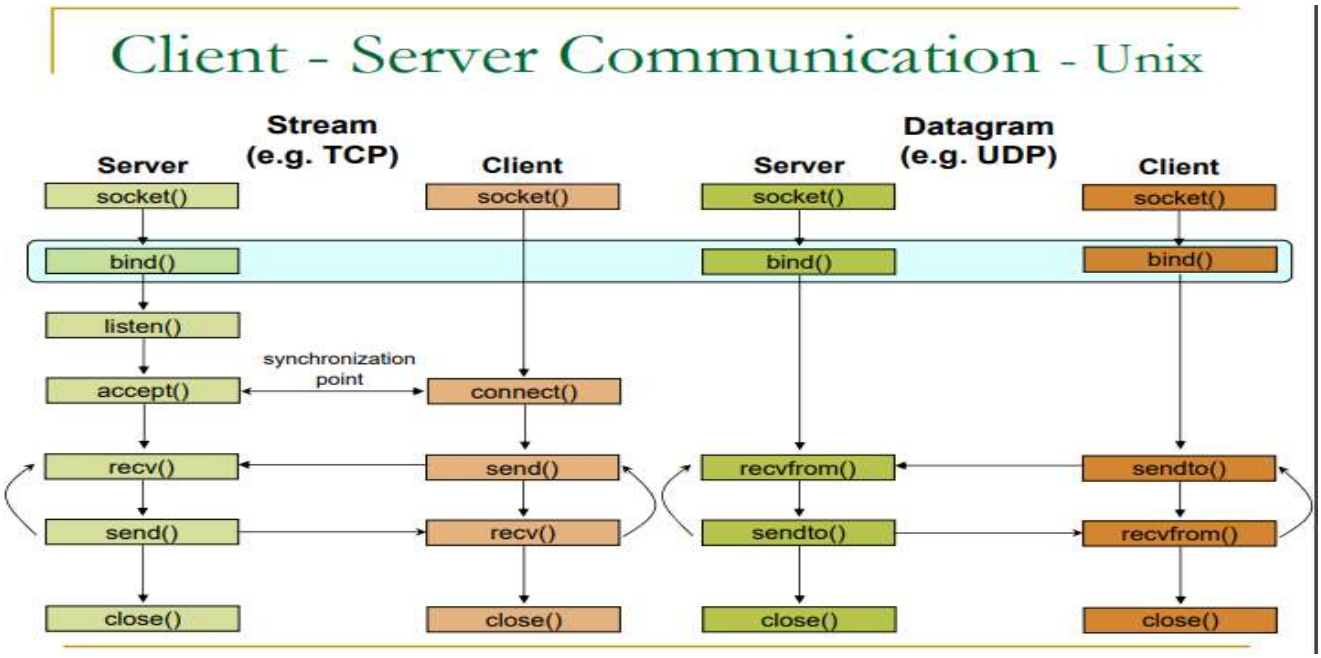
close():

Socket close in C: `close()`

- When finished using a socket, the socket should be closed
- `status = close(sockid);`
  - **sockid**: the file descriptor (socket being closed)
  - **status**: 0 if successful, -1 if error
- Closing a socket
  - closes a connection (for stream socket)
  - frees up the port used by the socket

Client side socket primitives:

socket(), connect(),send()/sendto(), recv()/recvfrom(), close()



connect():

- The connect() function is used by a TCP client to establish a connection with a TCP server/
- The function is defined as follows:

```
#include <sys/socket.h>

int connect (int sockfd, const struct sockaddr *servaddr, socklen_t addrlen);
```

where

1. sockfd is the socket descriptor returned by the socket function.

- The function returns 0 if it succeeds in establishing a connection (i.e., successful TCP three-way handshake, -1 otherwise.
- The client does not have to call bind() before calling this function: the kernel will choose both an ephemeral port and the source IP if necessary.

Establish Connection: connect()

- The client establishes a connection with the server by calling `connect()`
- `int status = connect(sockid, &foreignAddr, addrlen);`
  - **sockid**: integer, socket to be used in connection
  - **foreignAddr**: struct sockaddr: address of the passive participant
  - **addrlen**: integer, sizeof(name)
  - status: 0 if successful connect, -1 otherwise
- `connect()` is **blocking**

TYPES OF SERVERS:

- There are two main classes of servers, iterative and concurrent.
- An iterative server and a concurrent server are two different approaches to handling multiple client connections in network programming:

Iterative Server:

- In an iterative server architecture, the server handles one client connection at a time, sequentially.
- When a client connects to the server, the server accepts the connection and services the client's request until the interaction is complete.
- While the server is processing the request from one client, it does not accept connections from other clients. Only after the current client's request is processed does the server become available to accept connections from other clients.
- Iterative servers are generally simpler to implement compared to concurrent servers, as they do not require mechanisms for managing multiple concurrent client connections.

Concurrent Server:

- In a concurrent server architecture, the server is capable of handling multiple client connections simultaneously.
- When a client connects to the server, the server spawns a new process, thread, or uses non-blocking I/O to handle the client's request while still being available to accept connections from other clients.
- Concurrent servers can service multiple clients concurrently, allowing for better responsiveness and utilization of server resources.
- However, concurrent servers typically require more complex programming techniques to manage concurrency, synchronization, and resource sharing among multiple client-handling processes or threads.

Aspect	Iterative Server	Concurrent Server
Handling of Clients	Handles one client at a time sequentially	Can handle multiple clients simultaneously
Complexity	Generally simpler to implement	More complex, requiring management of concurrency
Resource Usage	Resources are utilized sequentially	Resources are utilized concurrently
Responsiveness	May be less responsive if one client's request is long	More responsive as multiple clients can be serviced concurrently
Scalability	Limited scalability due to sequential processing	Better scalability as multiple clients can be handled concurrently

The choice between an iterative and concurrent server depends on factors such as the expected workload, performance requirements, and the complexity of the application. Iterative servers may be suitable for simple applications with low to moderate client loads, while concurrent servers are often preferred for handling large numbers of simultaneous client connections or for applications requiring high responsiveness.

1.Write a program to illustrate connection oriented iterative Server

Server Program:

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <string.h>
int main()
{
   int sockfd, newsockfd, clilen;
   struct sockaddr_in serv_addr, cli_addr;
   char a[50];
   // Create a socket for communication
   // AF_INET for IPv4 addresses
   // SOCK_STREAM provides reliable, two-way, connection-based byte streams
   // 0 for default protocol for the socket
   sockfd = socket(AF_INET, SOCK_STREAM, 0);
   if (sockfd < 0)
  {
      printf("Socket creation failed\n");
      exit(0);
   }
   serv_addr.sin_family = AF_INET;
   serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
   // INADDR_ANY - Accept connections from any address (client)
   // Change address to the client IPv4 Address to accept only on client
   if (serv_addr.sin_addr.s_addr < 0)
  {
      printf("Invalid IP address: Unable to decode\n");
```

```
        exit(0);
    }
    serv_addr.sin_port = htons(4568);
    if (bind(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0)
   {
        printf("Bind failed\n");
        exit(1);
    }
    if (listen(sockfd, 5) < 0)
    {
        printf("Listen failed\n");
        exit(0);
    }
    clilen = sizeof(cli_addr);
    printf("Waiting for clients' messages (\'exit\' to close)\n");
    while (1)
   {
        newsockfd = accept(sockfd, (struct sockaddr *)&cli_addr, (socklen_t *) &clilen);
        memset(a, 0, sizeof(a));
        read(newsockfd, a, 50);
        printf("Server Received: %s\n", a);
        write(newsockfd, a, 50);
        close(newsockfd);
        if (!strcmp(a, "exit"))
        {
            printf("Exiting server\n");
            break;
        }
    }
    return 0;
}
```

Explanation:

```c
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <string.h>
```

<stdio.h>:

- printf(): Print formatted output to stdout.
- scanf(): Read formatted input from stdin.
- fprintf(): Print formatted output to a file.
- fscanf(): Read formatted input from a file.
- sprintf(): Write formatted data to a string.

<sys/types.h>:

- socket(): Create a new socket.
- bind(): Bind a socket to a specific address and port.
- listen(): Listen for incoming connections on a socket.
- accept(): Accept a new incoming connection on a listening socket.
- connect(): Initiate a connection to a remote socket.
- getaddrinfo(): Resolve hostnames and service names into socket addresses.
- getnameinfo(): Convert socket addresses to corresponding hostnames and service names.

<sys/socket.h>:

- Various socket-related constants, such as address families (AF_INET, AF_INET6), socket types (SOCK_STREAM, SOCK_DGRAM), and protocol families (PF_INET, PF_INET6).

<netinet/in.h>:

- struct sockaddr_in: Structure representing an IPv4 socket address.
- struct in_addr: Structure representing an IPv4 address.
- Constants and macros related to network byte order conversion (htonl(), htons(), ntohl(), ntohs()).

<stdlib.h>:

- exit(): Terminate the program with an exit status.
- atoi(), atol(), atof(): Convert strings to integers, long integers, and floating-point numbers.
- malloc(), calloc(), realloc(), free(): Memory allocation and deallocation functions.

<arpa/inet.h>:

- inet_addr(), inet_aton(): Convert IPv4 addresses from string to binary form.
- inet_ntoa(): Convert IPv4 addresses from binary to string form.
- inet_pton(), inet_ntop(): Convert IP addresses between binary and string forms (IPv4 and IPv6).

<unistd.h>:

- close(): Close a file descriptor.
- read(), write(): Read from or write to a file descriptor.
- fork(), exec(): Create a new process or execute a new program.
- sleep(): Suspend execution for a specified number of seconds.

<string.h>:

- memset(), memcpy(), memcmp(): Memory manipulation functions for setting, copying, and comparing memory regions.
- strcpy(), strncpy(), strcat(), strncat(), strcmp(), strncmp(): String manipulation functions for copying, concatenating, and comparing strings.

```
int sockfd, newsockfd, clilen;
struct sockaddr_in serv_addr, cli_addr;
char a[50];
```

- `sockfd`, `newsockfd`: File descriptors for the socket and the new socket created when accepting connections.
- `clilen`: Length of the client's address structure.
- `serv_addr`, `cli_addr`: Structures for storing server and client address information.
- `a`: Character array to store messages sent by clients.

```
// Create a socket for communication
// AF_INET for IPv4 addresses
// SOCK_STREAM provides reliable, two-way, connection-based byte streams
// 0 for default protocol for the socket
sockfd = socket(AF_INET, SOCK_STREAM, 0);
if (sockfd < 0) {
    printf("Socket creation failed\n");
    exit(0);
}
```

- This block of code creates a socket for communication using the `socket()` system call.
- It specifies the address family (`AF_INET` for IPv4), socket type (`SOCK_STREAM` for TCP), and protocol (0 for default protocol).
- If the socket creation fails, an error message is printed, and the program exits.

```
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
// INADDR_ANY - Accept connections from any address (client)
// Change address to the client IPv4 Address to accept only on client
if (serv_addr.sin_addr.s_addr < 0) {
    printf("Invalid IP address: Unable to decode\n");
    exit(0);
}
serv_addr.sin_port = htons(4568);
```

- Here, the server address structure `serv_addr` is initialized:
  - `sin_family` is set to `AF_INET` for IPv4.
  - `sin_addr.s_addr` is set to `INADDR_ANY`, indicating that the server will accept connections from any IP address. This can be changed to restrict connections to a specific IP address.
  - `sin_port` is set to port 4568 using `htons()` to convert it to network byte order.

1. `serv_addr.sin_addr.s_addr`: This field in the `struct sockaddr_in` structure (`serv_addr`) represents the IP address of the server. It is stored in binary form as an unsigned 32-bit integer (`in_addr_t` type).
2. `serv_addr.sin_addr.s_addr < 0`: This condition checks if the value stored in `sin_addr.s_addr` is less than zero. If the IP address is valid, it should always be greater than or equal to zero.
3. If the condition evaluates to true (i.e., the IP address is less than zero), it means that the IP address could not be properly decoded or parsed. This could happen if there was an error in converting the IP address from string format to binary form (e.g., using functions like `inet_addr()` or `inet_pton()`), resulting in an invalid IP address.
4. If an invalid IP address is detected, the program prints an error message `"Invalid IP address: Unable to decode"` using `printf()`. Then, the program exits with a status code of 0 using `exit(0)`. Exiting the program at this point indicates that it cannot continue further without a valid IP address, so it terminates execution.

```
if (bind(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
    printf("Bind failed\n");
    exit(1);
}
```

- This block of code binds the socket to the server address (`serv_addr`) using the `bind()` system call.
- If the binding fails, an error message is printed, and the program exits.

```
if (listen(sockfd, 5) < 0) {
    printf("Listen failed\n");
    exit(0);
}
```

- Here, the socket is set to listen for incoming connections using the `listen()` system call.
- The second argument specifies the maximum number of connections that can be queued for processing (5 in this case).
- If the listening fails, an error message is printed, and the program exits.

```
  clilen = sizeof(cli_addr);
  printf("Waiting for clients' messages (\'exit\' to close)\n");
```

- The variable `clilen` is initialized with the size of the client's address structure.
- A message is printed to indicate that the server is waiting for clients' messages.

```
while (1) {
    newsockfd = accept(sockfd, (struct sockaddr *)&cli_addr, (socklen_t *) &clilen);
    memset(a, 0, sizeof(a));
    read(newsockfd, a, 50);
    printf("Server Received: %s\n", a);
    write(newsockfd, a, 50);
    close(newsockfd);
    if (!strcmp(a, "exit")) {
        printf("Exiting server\n");
        break;
    }
}
```

- This block of code runs in a loop to continuously accept and process client connections.
- `accept()` system call accepts incoming connections, creating a new socket `newsockfd` for communication with the client.
- The received message from the client is read into the buffer `a` using `read()` system call.
- The server prints the received message.
- The same message is sent back to the client using the `write()` system call.
- The new socket is closed after the interaction with the client.
- If the received message is "exit", the server prints a message and exits the loop, terminating the program.

Client Program:

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <arpa/inet.h>
```

```
#include <unistd.h>
#include <string.h>
int main()
{
int sockfd;
struct sockaddr_in serv_addr;
char a[50],a1[50];
sockfd=socket(AF_INET,SOCK_STREAM,0);
if(sockfd<0)
{
printf("socket failed\n");
exit(0);
}
serv_addr.sin_family=AF_INET;
serv_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
//Change address to server's IPv4 address, don’t change if on same machine
serv_addr.sin_port=htons(4568);
if(connect(sockfd,(struct sockaddr *)&serv_addr,sizeof(serv_addr))<0)
{
printf("Connection failed\n");
exit(0);
}
memset(a, 0, sizeof(a));
printf("Enter the msg :\n");
scanf("%s",a); //use this if you want to read a single word
fgets(a, sizeof(a), stdin); //use this if you want to read multiple words in a line.
write(sockfd,a,50);
read(sockfd,a1,50);
printf("Client Received the msg: %s\n",a1);
close(sockfd);
if(!strcmp(a1,"exit"))
printf("Closing client program\n");
return 0;
}
```

Explanation:

```
int sockfd;
struct sockaddr_in serv_addr;
char a[50], a1[50];
```

- Here, variables are declared:
  - `sockfd`: File descriptor for the socket.
  - `serv_addr`: Structure for storing server address information.
  - `a` and `a1`: Character arrays to store messages to be sent to the server and received from the server, respectively.

```
sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

- This line creates a socket for communication using the `socket()` system call.
- It specifies the address family (`AF_INET` for IPv4), socket type (`SOCK_STREAM` for TCP), and protocol (0 for default protocol).
- The file descriptor of the socket is stored in the variable `sockfd`.

```
if (sockfd < 0) {
    printf("Socket creation failed\n");
    exit(0);
}
```

- If the socket creation fails (i.e., `sockfd` is less than 0), an error message is printed, and the program exits.

```
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
// Change address to server's IPv4 address, don't change if on the same machine
serv_addr.sin_port = htons(4568);
```

- Here, the server address structure `serv_addr` is initialized:
  - `sin_family` is set to `AF_INET` for IPv4.
  - `sin_addr.s_addr` is set to the server's IP address, which is `127.0.0.1` (localhost) in this case. This should be changed to the actual IP address of the server if it's on a different machine.
  - `sin_port` is set to port 4568 using `htons()` to convert it to network byte order.

```
if (connect(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
    printf("Connection failed\n");
    exit(0);
}
```

- This block of code establishes a connection to the server using the `connect()` system call.
- If the connection fails (i.e., `connect()` returns a value less than 0), an error message is printed, and the program exits.

```
memset(a, 0, sizeof(a));
printf("Enter the message:\n");
scanf("%s", a);
write(sockfd, a, 50);
read(sockfd, a1, 50);
printf("Client Received the message: %s\n", a1);
```

- Here, the client sends a message to the server and receives a response.
- The message to be sent is read from the user using `scanf()` and stored in the array `a`.
- The message is then sent to the server using the `write()` system call.
- The response from the server is read into the array `a1` using the `read()` system call.
- Finally, the received message is printed to the console.

```
    close(sockfd);
    if (!strcmp(a1, "exit"))
        printf("Closing client program\n");
    return 0;
}
```

- The socket descriptor `sockfd` is closed using the `close()` system call to release system resources.
- If the received message from the server is "exit", the program prints a message indicating that the client program is closing.
- Finally, the `main()` function ends, and the program returns 0 to indicate successful completion.

Execution:

1. Open two terminals. One for server and one for client.
2. execute the following commands in server terminal

```
joshua@JoshuaUbuntu:~/Desktop$ cd ..
joshua@JoshuaUbuntu:~$ cd joshua/
joshua@JoshuaUbuntu:~/joshua$ cd cn_programs/
joshua@JoshuaUbuntu:~/joshua/cn_programs$ cc iterative_server.c
joshua@JoshuaUbuntu:~/joshua/cn_programs$ ./a.out
Waiting for clients' messages ('exit' to close)
Server Received: hi
```

3. Execute the following commands in client terminal

```
joshua@JoshuaUbuntu:~/joshua/cn_programs$ cc iterative_client.c
joshua@JoshuaUbuntu:~/joshua/cn_programs$ ./a.out
Enter the msg :
hi
Client Received the msg: hi
```

2. Write a program to illustrate connection less iterative Server

Server Program:

```
#include<stdlib.h>
#include<stdio.h>
#include<sys/socket.h>
#include<sys/types.h>
#include<netinet/in.h>
int main()
{
int sockfd,newsockfd,clilen;
struct sockaddr_in serv_addr,cli_addr;
char msg[50];
sockfd=socket(AF_INET,SOCK_DGRAM,0);
if(sockfd<0)
{
printf("\n Socket Failed");
exit(0);
}
serv_addr.sin_family=AF_INET;
        serv_addr.sin_addr.s_addr=htonl(INADDR_ANY);
serv_addr.sin_port=htons(3456);
if(bind(sockfd,(struct sockaddr *)&serv_addr,sizeof(serv_addr))<0)
{
printf("\n Bind Failed");
exit(0);
}
clilen=sizeof(cli_addr);
recvfrom(sockfd,msg,80,0,(struct sockaddr *)&cli_addr,&clilen);
printf("Server Received: %s",msg);
sendto(sockfd,msg,80,0,(struct sockaddr *) &cli_addr,clilen);
write(sockfd);
close(sockfd);
}
```

Client:

```
#include<stdlib.h>
#include<stdio.h>
#include<sys/types.h>
#include<netinet/in.h>
#include<string.h>
#include<sys/socket.h>
int main()
{
int sockfd,n,clilen,servlen;
struct sockaddr_in cli_addr,serv_addr;
char msg[50],msg1[50];
sockfd=socket(AF_INET,SOCK_DGRAM,0);
if(sockfd<0)
{
printf("\n Sokcet Failed");
exit(0);
```

```
}
 serv_addr.sin_family=AF_INET;
serv_addr.sin_addr.s_addr=inet_addr("192.168.2.58");
serv_addr.sin_port=htons(3456);
cli_addr.sin_family=AF_INET;
cli_addr.sin_addr.s_addr=htonl(INADDR_ANY);
cli_addr.sin_port=htons(0);
if(bind(sockfd,(struct sockaddr*)&cli_addr,sizeof(cli_addr))<0)
{
printf("Client cantt bind");
 exit(1);
        }
printf("Enter Strin");
fgets(msg,50,stdin);
if(sendto(sockfd,msg,50,0,(struct sockaddr *)&serv_addr,sizeof(serv_addr))<0)
{
printf("Client send to error");
exit(0);
}
servlen=sizeof(serv_addr);
n=recvfrom(sockfd,msg1,50,0,(struct sockaddr *)&serv_addr,&servlen);
if(n<0)
{
printf("Recv error");
exit(1);
}
else
{
        printf("\n Client received msg : %s",msg1);
}
close(sockfd);
}
```

Execution:

```
joshua@JoshuaUbuntu:~/joshua/cn_programs$ cc connlessiterserv.c
joshua@JoshuaUbuntu:~/joshua/cn_programs$ ./a.out
Server Received: hello
```

```
joshua@JoshuaUbuntu:~/joshua/cn_programs$ gedit connlessitercli.c
joshua@JoshuaUbuntu:~/joshua/cn_programs$ cc connlessitercli.c
joshua@JoshuaUbuntu:~/joshua/cn_programs$ ./a.out
Enter String:hello

 Client received msg : hello
```

3 Write a program to illustrate connection oriented concurrent Server

Server:

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <string.h>
int main()
{
int sockfd,newsockfd,clilen, pid;
struct sockaddr_in serv_addr,cli_addr;
char a[50];
sockfd=socket(AF_INET,SOCK_STREAM,0); //create a socket for communication
//AF_INET for IPv4 addresses
//SOCK_STREAM provides reliable, two-way, connection-based byte streams

//0 for default protocol for the socket
if(sockfd<0)
{
printf("socket failed\n");
exit(0);
}
serv_addr.sin_family = AF_INET;
//Set address to accept connection from any client with any IP address
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
//INADDR_ANY - Accept connections from any address (client)
//change address to the client IPv4 Address to accept only on client
if(serv_addr.sin_addr.s_addr < 0)

{
printf("Invalid IP address: Unable to decode\n");

exit(0);
}
serv_addr.sin_port = htons(3100);
if(bind(sockfd,(struct sockaddr *)&serv_addr,sizeof(serv_addr))<0)
{
printf("Bind failed\n");
exit(1);
}
if(listen(sockfd,5)<0)
{
printf("Listen failed\n");
exit(0);
}
clilen=sizeof(cli_addr);
        printf("Waiting for clients\n");

while(1)
{
//Accept connection from clients
newsockfd=accept(sockfd,(struct sockaddr *)&cli_addr, (socklen_t *) &clilen);
```

```
pid = fork(); //create a new process to serve each request
if(pid==0)
{
//Child process serving requests will execute this block
while(1)
{
memset(a, 0, sizeof(a));
read(newsockfd,a,50);//Read message from client
//Also print the process id of the instance to check if concurrency works
printf("Instance : %d \n\tServer Recieved: %s\n",(int)getpid(),a);
write(newsockfd,a,50); //Return the same message to the client
                if(!strcmp(a, "exit"))

{
printf("Closing connection : Instance %d\n", (int)getpid());
break;
}
}
close(newsockfd); //Close the connection
break; //Break the loop to end the process (serving process)
}
}
return 0;
}
```

Client Program:

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <string.h>
int main()
{
int sockfd;
struct sockaddr_in serv_addr;
char a[50],a1[50], *pos;
sockfd=socket(AF_INET,SOCK_STREAM,0);

if(sockfd<0)
{
printf("socket failed\n");

exit(0);
}
serv_addr.sin_family=AF_INET;
serv_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
//Change address to server's IPv4 address, dont change if on same machine
serv_addr.sin_port=htons(3100);
if(connect(sockfd,(struct sockaddr *)&serv_addr,sizeof(serv_addr))<0)
{
printf("Connection failed\n");
exit(0);
}
memset(a, 0, sizeof(a));
```

```
while(1)
{
printf("Enter the msg :\n");
fgets(a,sizeof(a), stdin); // read entire line into a[]
//This blocks removes trailing newline character (if present) left form fgets
if( (pos = strchr(a, '\n'))!= NULL)
*pos = '\0';
write(sockfd,a,50);
read(sockfd,a1,50);
printf("Client Received the msg: %s\n",a1);
if(!strcmp(a, "exit"))
{
printf("Closing connection\n");
break;
}
}

close(sockfd);
if(!strcmp(a1,"exit"))
printf("Closing client program\n");

return 0;
}
```

Execution:

```
joshua@JoshuaUbuntu:~/joshua/cn_programs$ cc concservconn.c
joshua@JoshuaUbuntu:~/joshua/cn_programs$ ./a.out
Waiting for clients
Instance : 2656
        Server Recieved: hi hello
Instance : 2656
        Server Recieved: this is second time
Instance : 2656
        Server Recieved: this is third time
```

```
joshua@JoshuaUbuntu:~/joshua/cn_programs$ cc conccliconn.c
joshua@JoshuaUbuntu:~/joshua/cn_programs$ ./a.out
Enter the msg :
hi hello
Client Received the msg: hi hello
Enter the msg :
this is second time
Client Received the msg: this is second time
Enter the msg :
this is third time
Client Received the msg: this is third time
Enter the msg :
^C
```

4 Write a program to illustrate connection less concurrent Server

Server Program:

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <string.h>
int main()
{
int sockfd, n,clilen, pid;
struct sockaddr_in serv_addr,cli_addr;
char a[50];
sockfd=socket(AF_INET,SOCK_DGRAM,0); //create a socket for communication
//AF_INET for IPv4 addresses
//Communication with Datagrams (UDP - Connectionless, non-reliable)

//0 for default protocol for the socket
if(sockfd<0)
{
printf("socket failed\n");
exit(0);
}
serv_addr.sin_family = AF_INET;
//Set address to accept connection from any client with any IP address
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
//INADDR_ANY - Accept connections from any address (client)
//change address to the client IPv4 Address to accept only on client
if(serv_addr.sin_addr.s_addr < 0)
{

printf("Invalid IP address: Unable to decode\n");
exit(0);
}
serv_addr.sin_port = htons(3100);
if(bind(sockfd,(struct sockaddr *)&serv_addr,sizeof(serv_addr))<0)
{
printf("Bind failed\n");
exit(1);
}
clilen=sizeof(cli_addr);
printf("Waiting for clients\n");
while(1)
{
memset(a, 0, sizeof(a));
//Read messages from clients (without connection) into a[]; type "man 2 recvfrom" in terminal
for details
n = recvfrom(sockfd, a, 50, 0, (struct sockaddr *)&cli_addr, (socklen_t *) &clilen);

if(n>0)
{
pid = fork(); //create a new process to serve each request
if(pid==0)
```

```
{
//Child process serving requests will execute this block
//read(newsockfd,a,50); //Read message from client
//Also print the process id of the instance to check if concurrency works
printf("Instance : %d \n\tServer Recieved: %s\n",(int)getpid(),a);
if( sendto(sockfd, a, 50, 0, (struct sockaddr *)&cli_addr, (socklen_t) clilen) < 0)
//Return the same message to the client
{
printf("UDP sending failed\nExiting... \n");
close(sockfd);

exit(1);
}
close(sockfd); //Close the connection
break; //Break the loop to end the process (serving process)
}
}
else
{
printf("UDP receiving failed\nExiting... \n");
close(sockfd);
exit(1);
}
}
return 0;
}
```

Client Program:

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <string.h>

int main()
{
int sockfd, servlen;
struct sockaddr_in serv_addr;
char a[50],a1[50], *pos;

servlen = sizeof(serv_addr);
sockfd=socket(AF_INET,SOCK_DGRAM,0);
if(sockfd<0)
{
printf("socket failed\n");
exit(0);
}
serv_addr.sin_family=AF_INET;
serv_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
//Change address to server's IPv4 address, dont change if on same machine
serv_addr.sin_port=htons(3100);
memset(a, 0, sizeof(a));
printf("Enter the msg :\n");
```

```
fgets(a,sizeof(a), stdin); // read entire line into a[]
//This blocks removes trailing newline character (if present) left form fgets
if( (pos = strchr(a, '\n'))!= NULL)
*pos = '\0';

if(sendto(sockfd, a, 50, 0, (struct sockaddr *)&serv_addr, (socklen_t) servlen) < 0)
{
printf("UDP client : Message sending failed\nExiting...");
close(sockfd);
exit(1);
}
if(recvfrom(sockfd, a1, 50, 0, (struct sockaddr *)&serv_addr,(socklen_t *) &servlen) < 0)
{
printf("UDP client : Message receiveing failed\nExiting...");
close(sockfd);
exit(1);
}
printf("Client Received the msg: %s\n",a1);
close(sockfd);

return 0;
}
```

Execution:

```
joshua@JoshuaUbuntu:~/joshua/cn_programs$ cc concservconnless.c
joshua@JoshuaUbuntu:~/joshua/cn_programs$ ./a.out
Waiting for clients
Instance : 2958
        Server Recieved: hi this is concurrent client connless
Instance : 2958
        Server Recieved: hi, for the second time
Instance : 2958
        Server Recieved: hi, for the third time
```

```
joshua@JoshuaUbuntu:~/joshua/cn_programs$ cc conccliconnless.c
joshua@JoshuaUbuntu:~/joshua/cn_programs$ ./a.out
Enter the msg :
hi this is concurrent client connless
Client Received the msg: hi this is concurrent client connless
Enter the msg :
hi, for the second time
Client Received the msg: hi, for the second time
Enter the msg :
hi, for the third time
Client Received the msg: hi, for the third time
Enter the msg :
```

5. Write a program to illustrate date time client/Server

Server Program:

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <string.h>
#include <time.h>
int main()
{
int sockfd,newsockfd,clilen;
struct sockaddr_in serv_addr,cli_addr;
char a[50];
time_t now;
struct tm present;
sockfd=socket(AF_INET,SOCK_STREAM,0); //create a socket for communication

//AF_INET for IPv4 addresses
//SOCK_STREAM provides reliable, two-way, connection-based byte streams
//0 for default protocol for the socket
if(sockfd<0)
{
printf("socket failed\n");
exit(1);
}
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
//INADDR_ANY - Accept connections from any address (client)
//change address to the client IPv4 Address to accept only on client

if(serv_addr.sin_addr.s_addr < 0)
{
printf("Invalid IP address: Unable to decode\n");
exit(1);
}
serv_addr.sin_port = htons(3100);
if(bind(sockfd,(struct sockaddr *)&serv_addr,sizeof(serv_addr))<0)
{
printf("Bind failed\n");
exit(1);
}
if(listen(sockfd,5)<0)
{
printf("Listen failed\n");
exit(1);
}

clilen=sizeof(cli_addr);
while(1)
{
printf("Waiting for clients: \n");
newsockfd=accept(sockfd,(struct sockaddr *)&cli_addr, (socklen_t *) &clilen);
```

```
memset(a, 0, sizeof(a));
read(newsockfd,a,50);
time(&now); //get the present time in seconds - see 'man 2 time' on terminal
present = *localtime(&now);
//localtime breaks time_t variable 'now' into 'stuct tm'
//and returns the pointer to the newly created sturcture
//The structure is copied into 'present'
sprintf(a,"Time: %d-%d-%d %d:%d:%d\n", present.tm_year + 1900, present.tm_mon +
1, present.tm_mday, present.tm_hour, present.tm_min, present.tm_sec);

//The value from the structure 'present' is encoded as date and time
//The formatted date and time (as string) is copied into 'char *a' using sprintf
write(newsockfd,a,50); //Write (transmit) a into socket
close(newsockfd);
}
return 0;
}
```

Client:

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <string.h>
int main()
{
int sockfd;
struct sockaddr_in serv_addr;
char a[50],a1[50];
sockfd=socket(AF_INET,SOCK_STREAM,0);
if(sockfd<0)
{
printf("socket failed\n");

exit(0);
}
serv_addr.sin_family=AF_INET;
serv_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
//Change address to server's IPv4 address, dont change if on same machine

serv_addr.sin_port=htons(3100);
if(connect(sockfd,(struct sockaddr *)&serv_addr,sizeof(serv_addr))<0)
{
printf("Connection failed\n");
exit(0);
}
memset(a, 0, sizeof(a));
//printf("Press enter to get time\n");
//scanf("%s",a);
write(sockfd,a,50);
read(sockfd,a1,50);
```

```
printf("Client Received %s\n",a1);
close(sockfd);
if(!strcmp(a1,"exit"))
printf("Closing client program\n");
return 0;
}
```

Execution:

```
joshua@JoshuaUbuntu:~/joshua/cn_programs$ cc datetimeserv.c
joshua@JoshuaUbuntu:~/joshua/cn_programs$ ./a.out
Waiting for clients:
Waiting for clients:
Waiting for clients:
```

```
joshua@JoshuaUbuntu:~/joshua/cn_programs$ cc datetimecli.c
joshua@JoshuaUbuntu:~/joshua/cn_programs$ ./a.out
Client Received Time: 2024-3-27 16:9:28

joshua@JoshuaUbuntu:~/joshua/cn_programs$ ./a.out
Client Received Time: 2024-3-27 16:9:41
```

EXPERIMENT-3

Configuration of Router and Switch using Cisco Packet Tracer

The Cisco Packet Tracer:

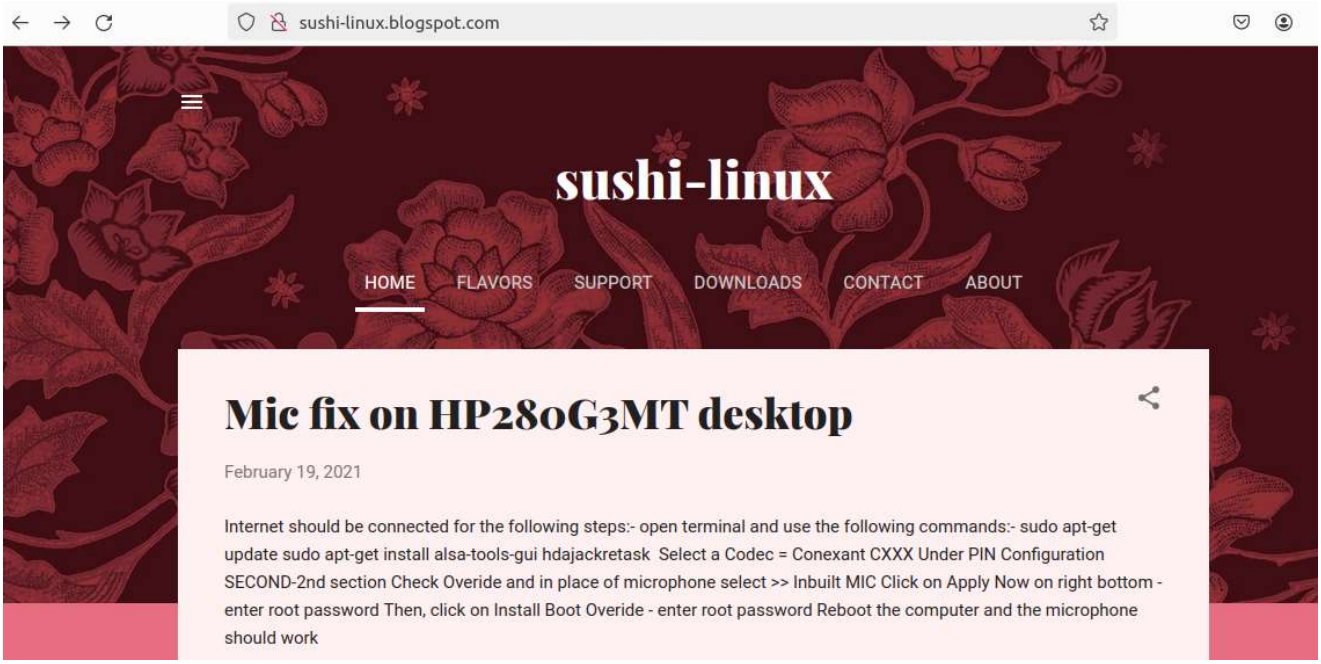
- Cisco Packet Tracer is a network simulation and visualization tool developed by Cisco Systems. It's designed for educational purposes, particularly for students and professionals studying and practicing networking concepts. Packet Tracer allows users to create, configure, and simulate network topologies using virtual Cisco devices.

Key features of Cisco Packet Tracer include:

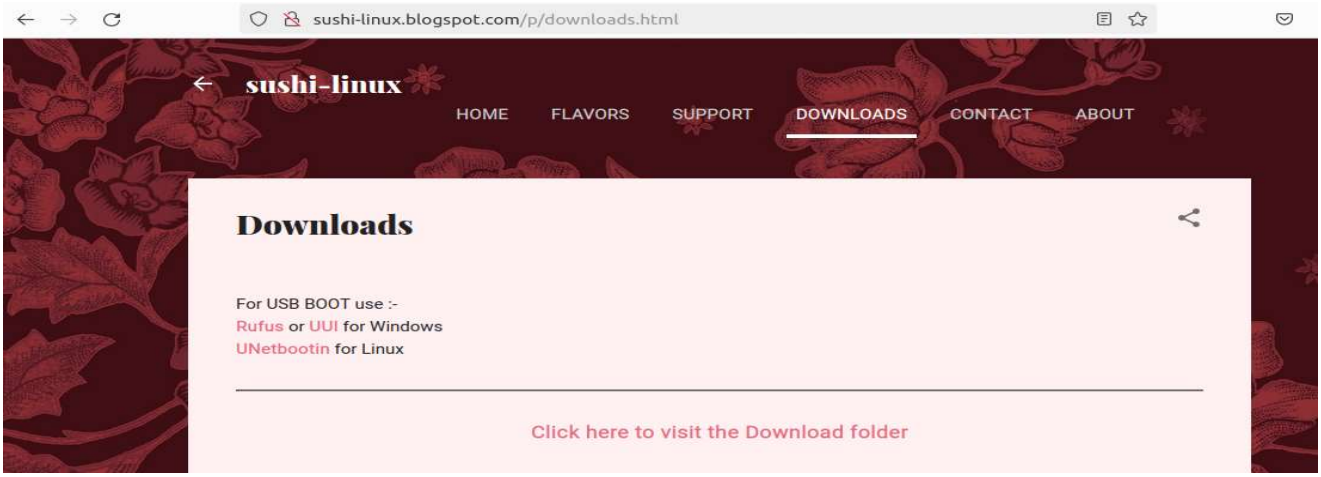
- **Device Simulation:** Packet Tracer provides a wide range of virtual Cisco networking devices such as routers, switches, wireless access points, and end devices (such as PCs and servers) that can be configured and interconnected to simulate real-world network environments.
- **Topology Design:** Users can create network topologies by dragging and dropping devices onto a workspace, then connecting them with cables and configuring their settings.
- **Configuration and Testing:** Users can configure the behavior and settings of network devices, including IP addressing, routing protocols, VLANs, security settings, and more. They can then test the configurations by simulating network traffic and observing how devices respond.
- **Visualization:** Packet Tracer offers visual representations of network topologies, device configurations, and traffic flows, helping users understand networking concepts more intuitively.
- **Learning Materials:** Cisco provides various learning materials, tutorials, and pre-built network scenarios within Packet Tracer to help users learn networking concepts and practice hands-on skills.
- **Collaboration:** Packet Tracer supports collaboration features, allowing multiple users to work together on the same network simulation, making it useful for group projects or instructor-led classroom activities.

INSTALLATION STEPS FOR CISCO PACKET TRACER IN LINUX OPERATING SYSTEM

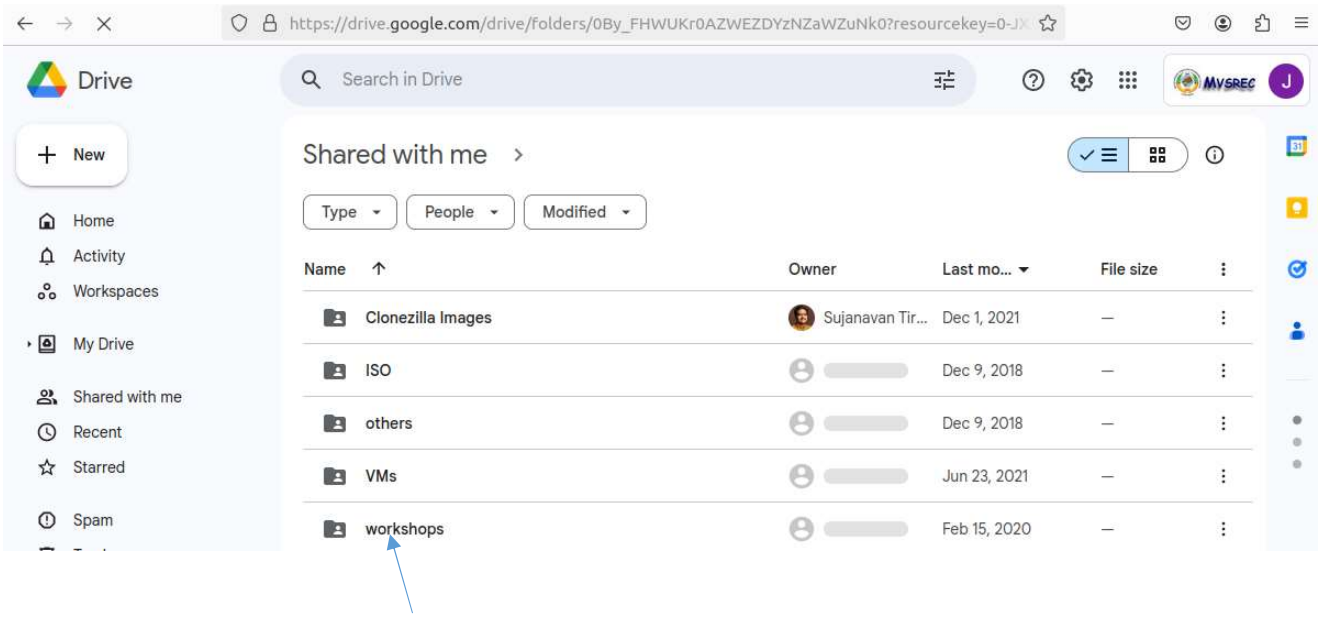
1. Go to sushi-linux.blogspot.com



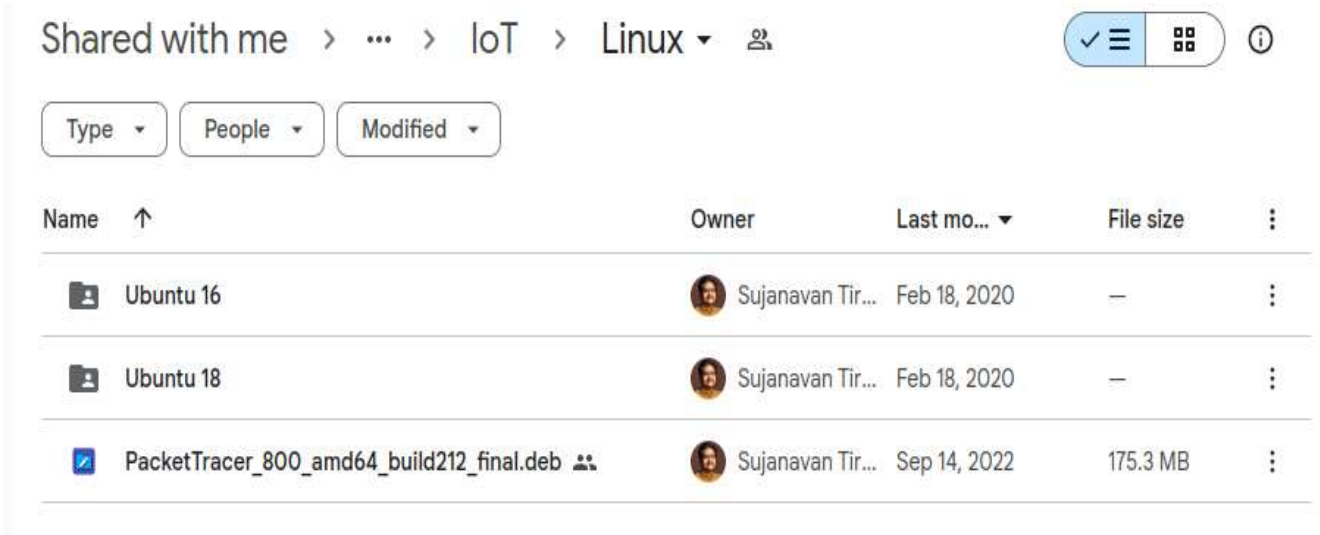
2. Go to DOWNLOADS TAB and click on “click here to visit download folder”



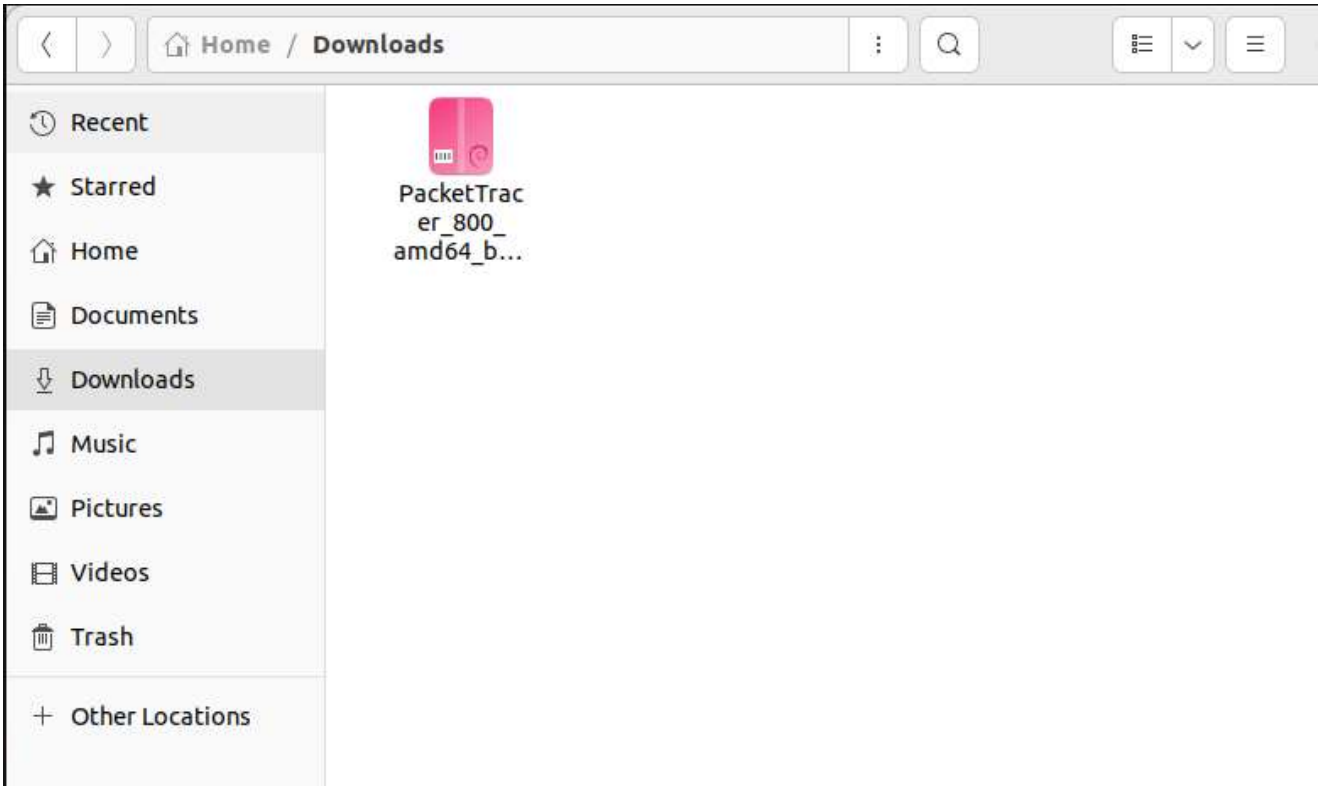
3. Click on workshops



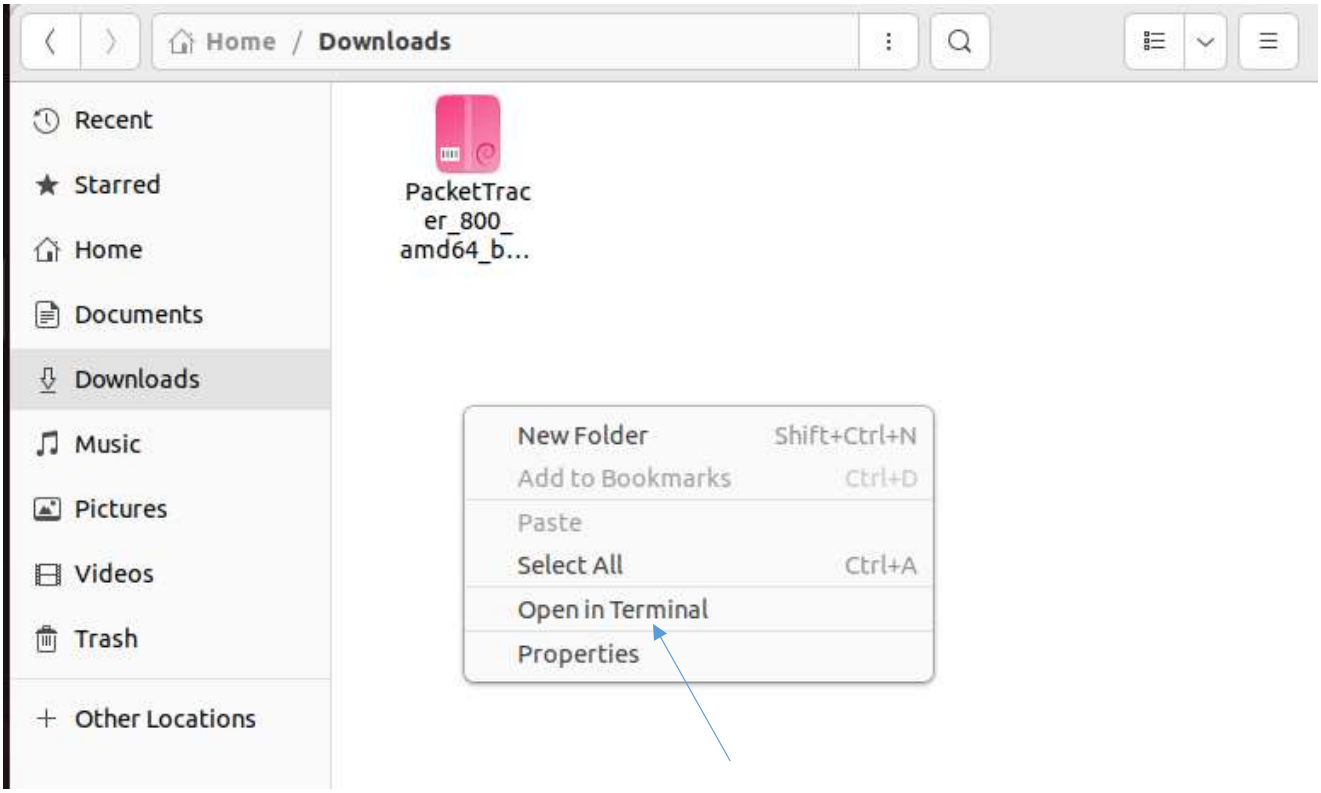
4. Click on IoT-> Linux



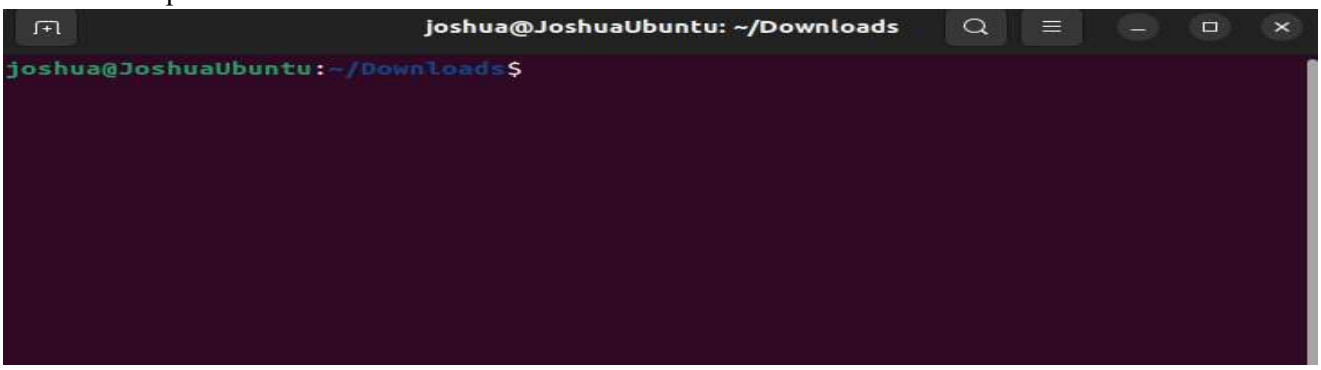
5. Download packet tracer software.



6. Open new terminal in the same folder



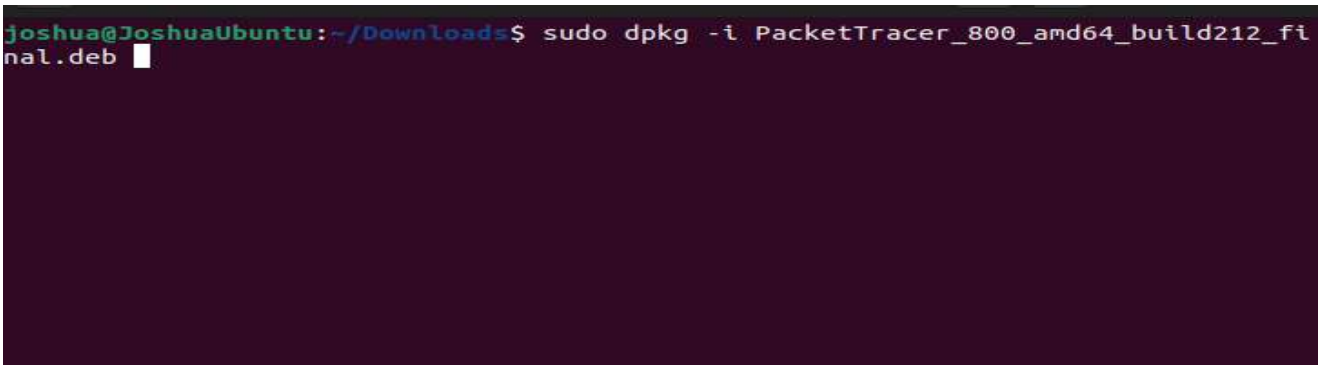
7. Terminal opens like this



8. Type the following command and press enter:

Tip: you can type sudo dpkg –i P and then press tab key, you will get the complete file name.

joshua@JoshuaUbuntu:~/Downloads$ sudo dpkg -i PacketTracer_800_amd64_build212_final.deb

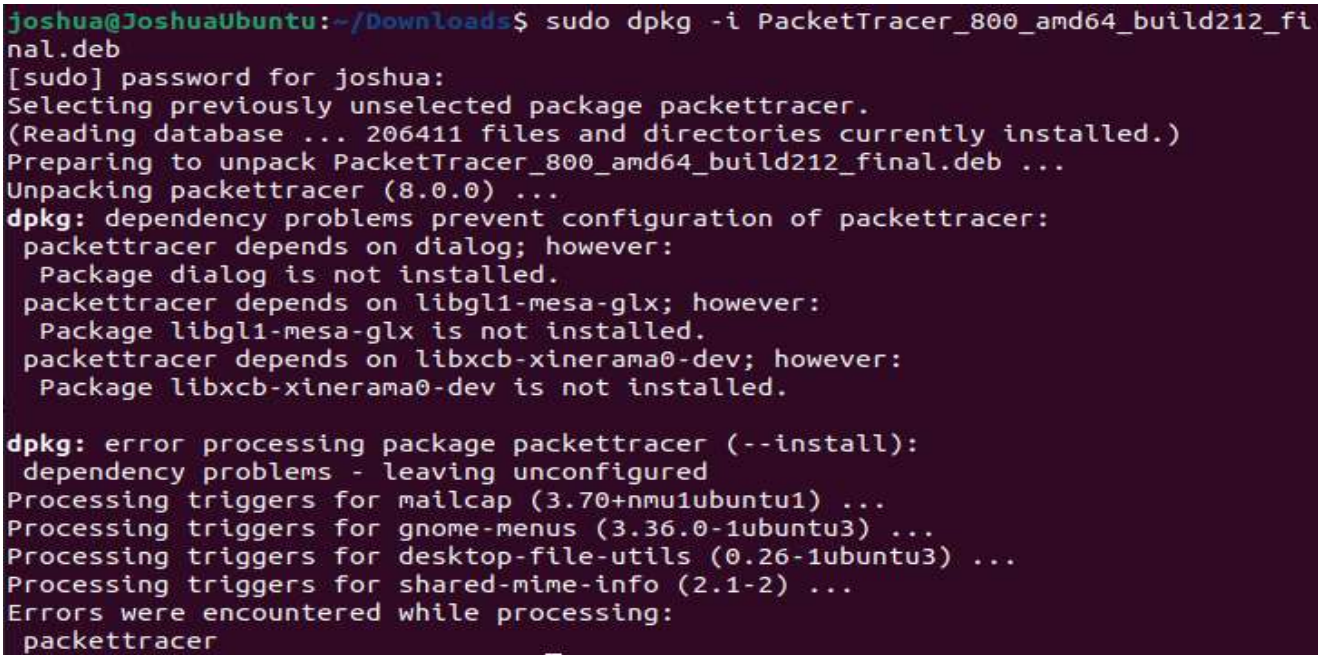


Next, enter sudo password,

Next, press ok(mouse will not work , only keyboard usage)

Accept the terms: yes

9. if you encounter an error like this:



10. Give the command:

sudo apt install –f

11. Your installation will be now successful and returns to prompt.

12. Search for packet tracer in your applications

13. Press **NO** for multi-user environment

14. Hurray, you can start working now!!!

Private IP Addresses:

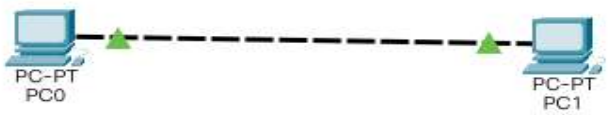
Private IP addresses are a range of IP addresses reserved for use within private networks and are not routable on the public internet. They are defined in RFC 1918 and include the following ranges:

1. 10.0.0.0 - 10.255.255.255 (a single Class A network)
2. 172.16.0.0 - 172.31.255.255 (16 contiguous Class B networks)
3. 192.168.0.0 - 192.168.255.255 (256 contiguous Class C networks)

- These addresses are commonly used in home, office, and enterprise networks for devices like computers, printers, routers, and other networked devices.
- Private IP addresses are typically assigned dynamically by a router using DHCP (Dynamic Host Configuration Protocol) or can be manually configured.
- They allow multiple devices to share a single public IP address for internet access through a process called Network Address Translation (NAT).

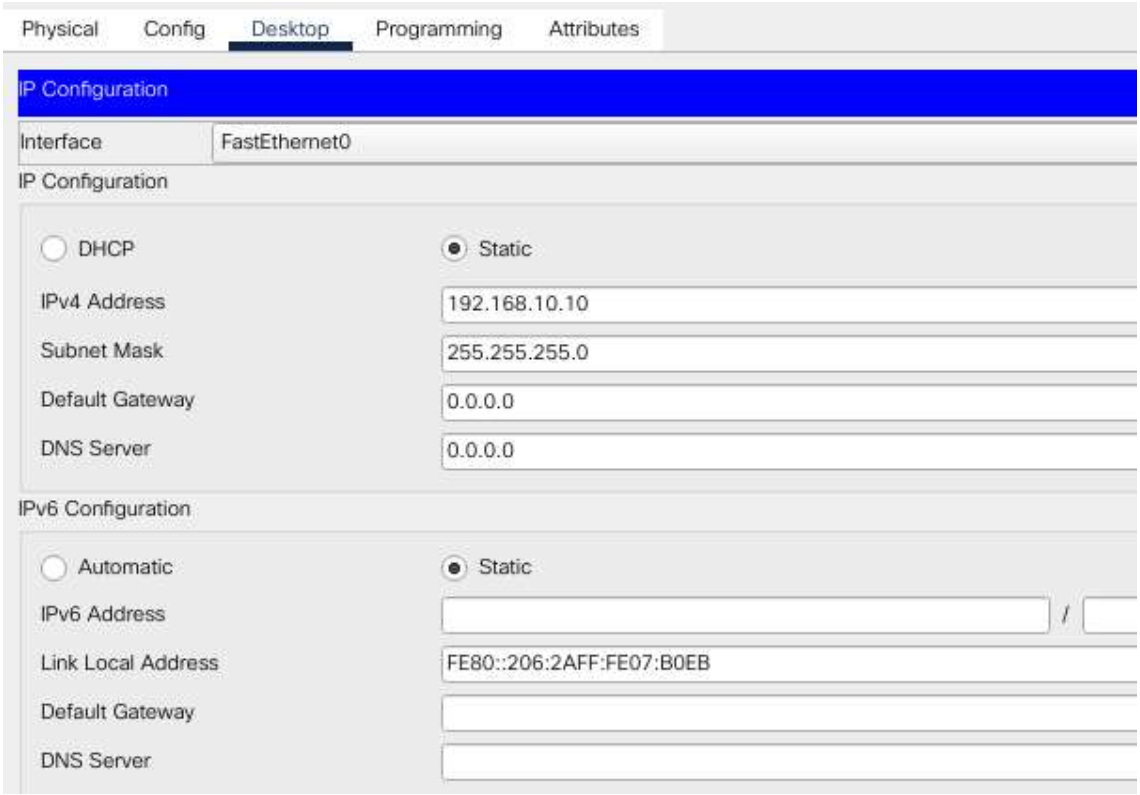
Example 1: Using CPT to connect to end systems and establish communication between them.

Step 1: Add two end devices on to the work space and connect them using automatic cable

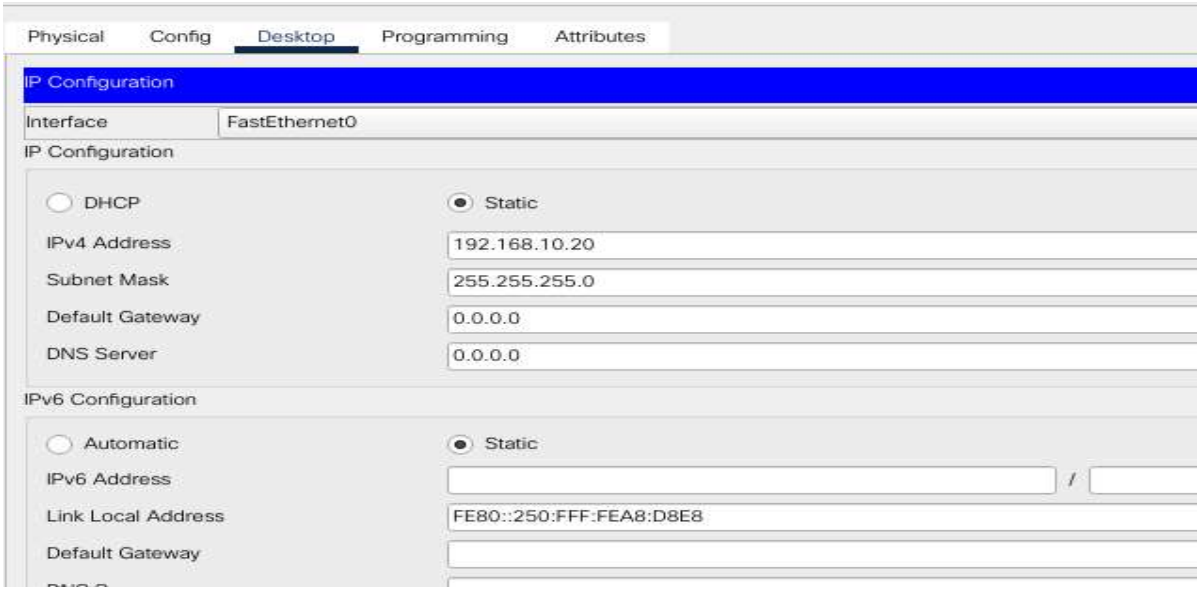


Step 2: Click on computer so that u get:

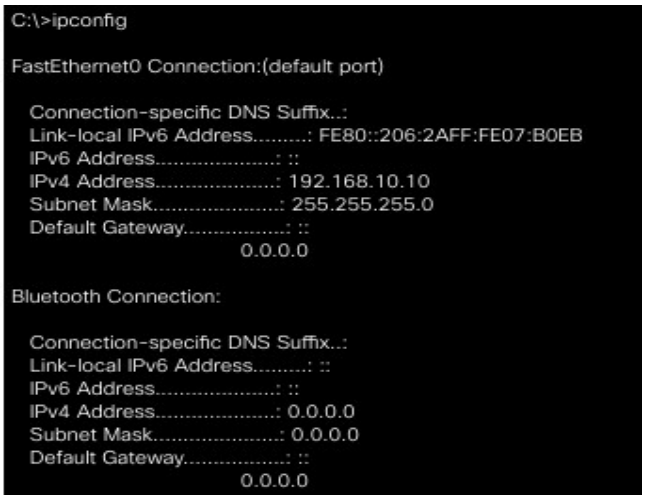
Step 3: In Desktop tab, select IP Configuration and give 192.168.10.10 in IPv4 address text box and press tab



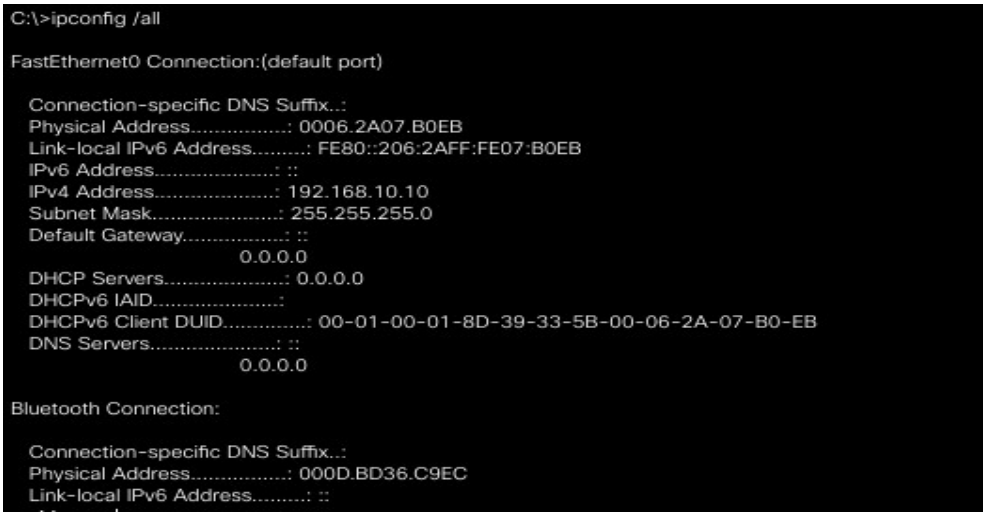
Do the same for the second pc, with IP: 192.168.10.20 and press tab.



Step 4: To see the ip address, type **ipconfig** in the command prompt of the host:



To see MAC/PHYSICAL ADDRESS: TYPE: ipconfig /all



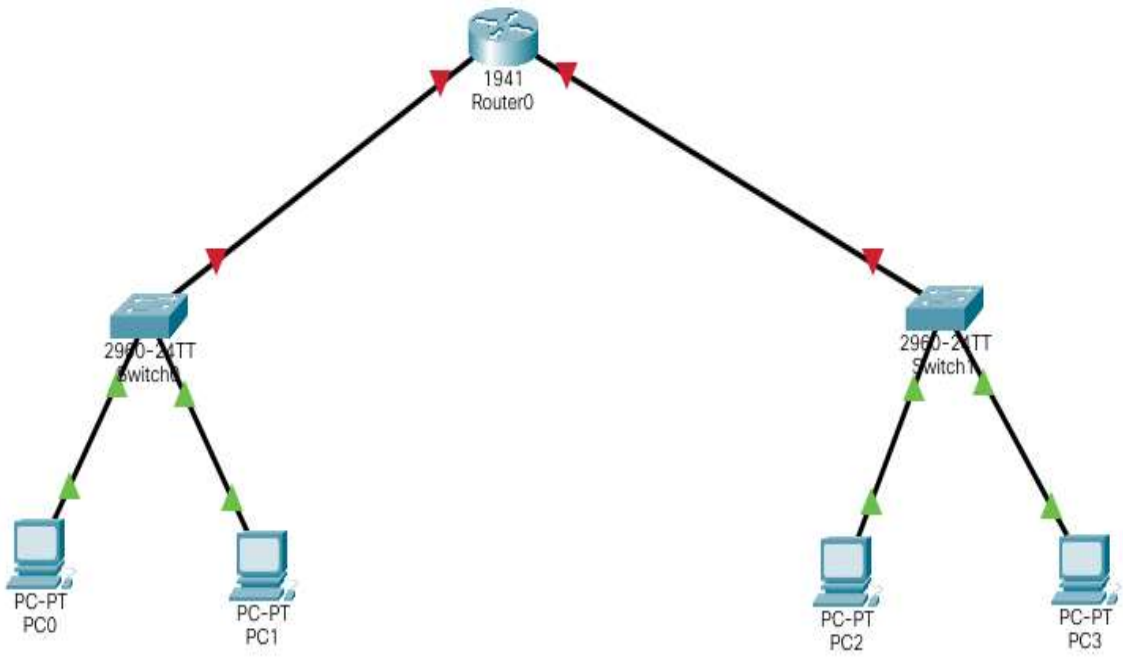
Step 5: to check for the communication established or not type

Ping 192.168.10.20 (you should type this command in host 1: 192.168.10.10)

```
C:\>ping 192.168.10.20

Pinging 192.168.10.20 with 32 bytes of data:

Reply from 192.168.10.20: bytes=32 time<1ms TTL=128
Reply from 192.168.10.20: bytes=32 time<1ms TTL=128
Reply from 192.168.10.20: bytes=32 time<1ms TTL=128
Reply from 192.168.10.20: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.10.20:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

Example 2: Using CPT to configure router to establish connection between two different networks.

Step 1: Add end devices (computers), switches (2960), router (1941) and create a network as follows:



Step 2: Assign IP Addresses to end devices in both Network 1 and Network 2.

Network 1:

192.168.10.10 (PC0)

192.168.10.20 (PC1)

Network 2:

192.168.20.10 (PC2)

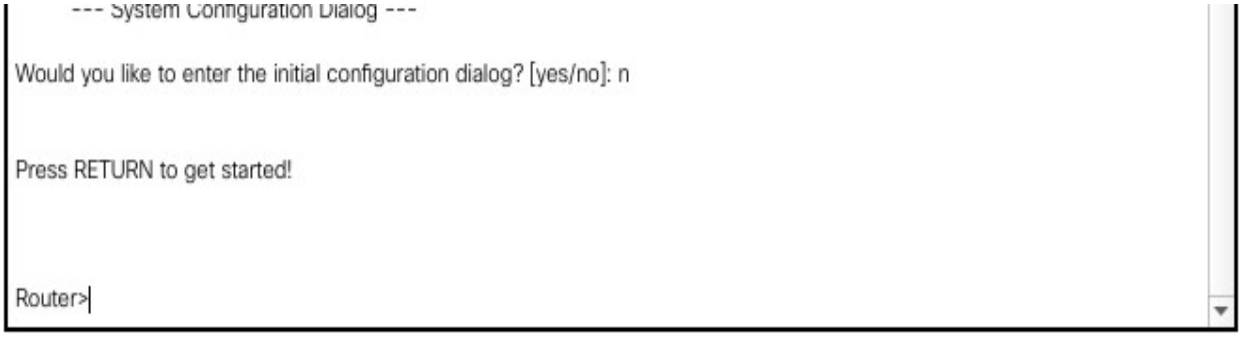
192.168.20.20 (PC3)

Step 3: Click on Router and go to CLI(Command Line Interface) tab.

Step 4: It asks for "would you like to enter intial configuration dialog":

Type **n** and then press **enter.**

You will get the following prompt:

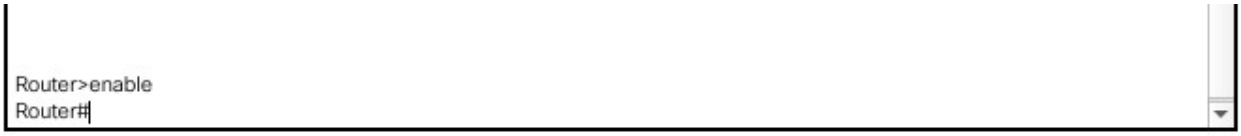


This mode is the **USER MODE**

User mode (User EXEC mode)

- User Mode is the first mode a user has access to after logging into the router.
- **The user mode can be identified by the > prompt following the router name.**
- This mode allows the user to **execute only the basic commands**, such as those that show the system's status.
- The **router cannot be configured or restarted from this mode**.

Step 5: Now type **enable** in this mode, you will now enter into privileged mode.



Privileged mode (Privileged EXEC Mode)

Privileged mode allows users to view the system configuration, restart the system, and enter router configuration mode. Privileged mode also allows all the commands that are available in user mode. Privileged mode can be identified by the # prompt following the router name. From the user mode, a user can change to Privileged mode, by running the "enable" command. Also we can keep a enable password or enable secret to restrict access to Privileged mode. An enable secret password uses stronger encryption when it is stored in the configuration file and it is safer.

Step 6: Type command **show running-config.**

It displays the configuration information of the router.

```
Router#show running-config
Building configuration...

Current configuration : 610 bytes
!
version 15.1
no service timestamps log datetime msec
no service timestamps debug datetime msec
no service password-encryption
!
hostname Router
!
!
!
!
!
!
!
!
ip cef
no ipv6 cef
!
!
```

Step 7: Type command **configure terminal**

```
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#
```

Now, we entered into global configuration mode.

Global Configuration mode

- Global Configuration mode mode allows users to modify the running system configuration.
- From the Privileged mode a user can move to configuration mode by running the "configure terminal" command from privileged mode.
- To exit configuration mode, the user can enter "end" command or press Ctrl-Z.

Now, we are ready to configure the router in the global configuration mode.

Step 8: Type command **interface gigabitEthernet 0/0**. (Enters the configuration mode for a Gigabit Ethernet interface on the router.)

You will get the following.

```
Router(config)#interface gigabitEthernet 0/0
Router(config-if)#
```

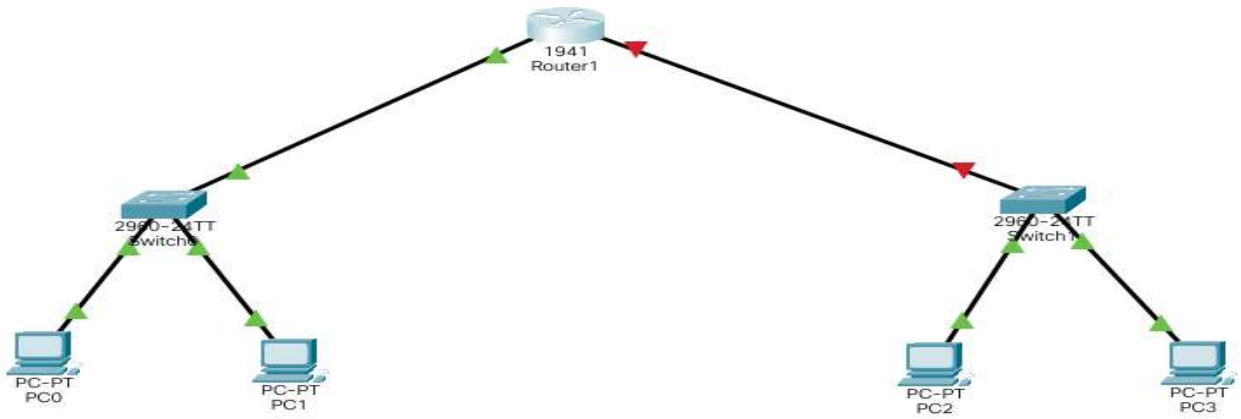
Step 9: Type command **ip address 192.168.10.100 255.255.255.0 (**Sets the IP address and subnet mask for the specified Gigabit Ethernet interface. Use this Step if you are configuring an IPv4 address.**)**

```
Router(config-if)#ip address 192.168.10.100 255.255.255.0
Router(config-if)#
```

Step 10: Type command **no shutdown (**Enables the Gigabit Ethernet interface and changes its state from administratively down to administratively up.**)**

```
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up
```

Now, we can observe, the arrows on the left turns from red to green.



Step 11: Type command **exit (**Exits configuration mode for the Gigabit Ethernet interface and returns to privileged EXEC mode.**)**

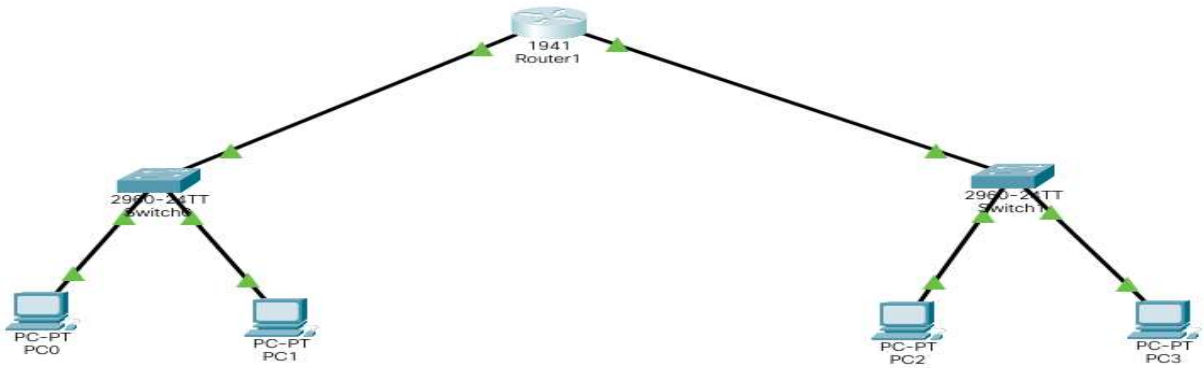
```
Router(config-if)#exit
Router(config)#
```

Step 12: Now, repeat the same for the other interface of the router (gigabitEthernet 0/1)

The sequence of commands are as follows:

```
Router(config)#interface gigabitEthernet 0/1

Router(config-if)#ip address 192.168.20.100 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up

Router(config-if)#
```

Now, we can observe, the arrows on the right turns from red to green.

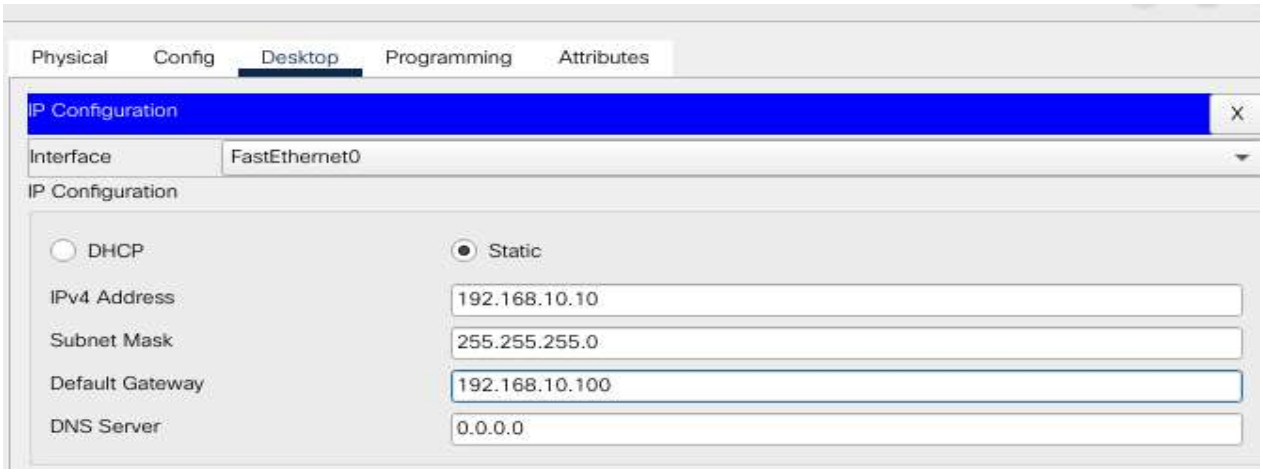


Step 13: To establish communication between two different networks, we require gateway addresses.

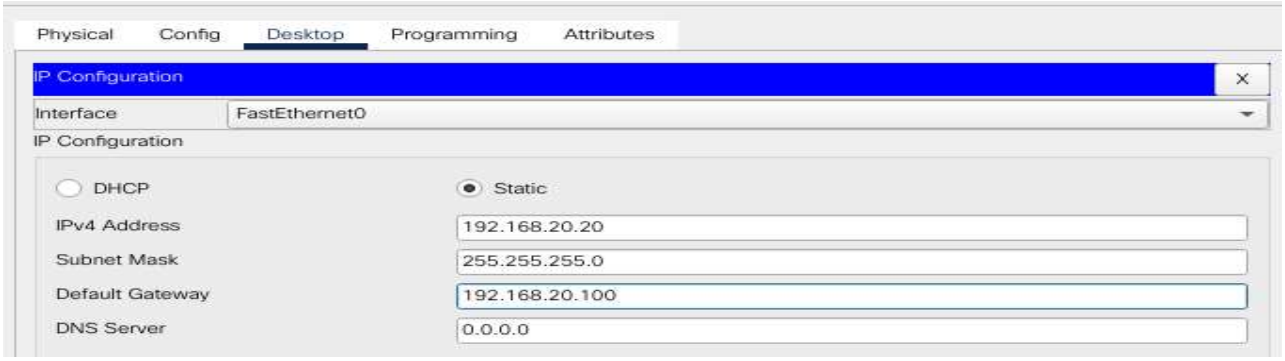
A gateway address, also known as a default gateway or router address, is a network device's IP address on a local network that serves as an entry point or exit point to other networks.

So, for the two hosts in Network 1, the gateway address should be the interface address of **gigabitEthernet 0/0,** which is **192.168.10.100.**

It is set as follows:



- for the two hosts in Network 2, the gateway address should be the interface address of **gigabitEthernet 0/1,** which is **192.168.20.100.**



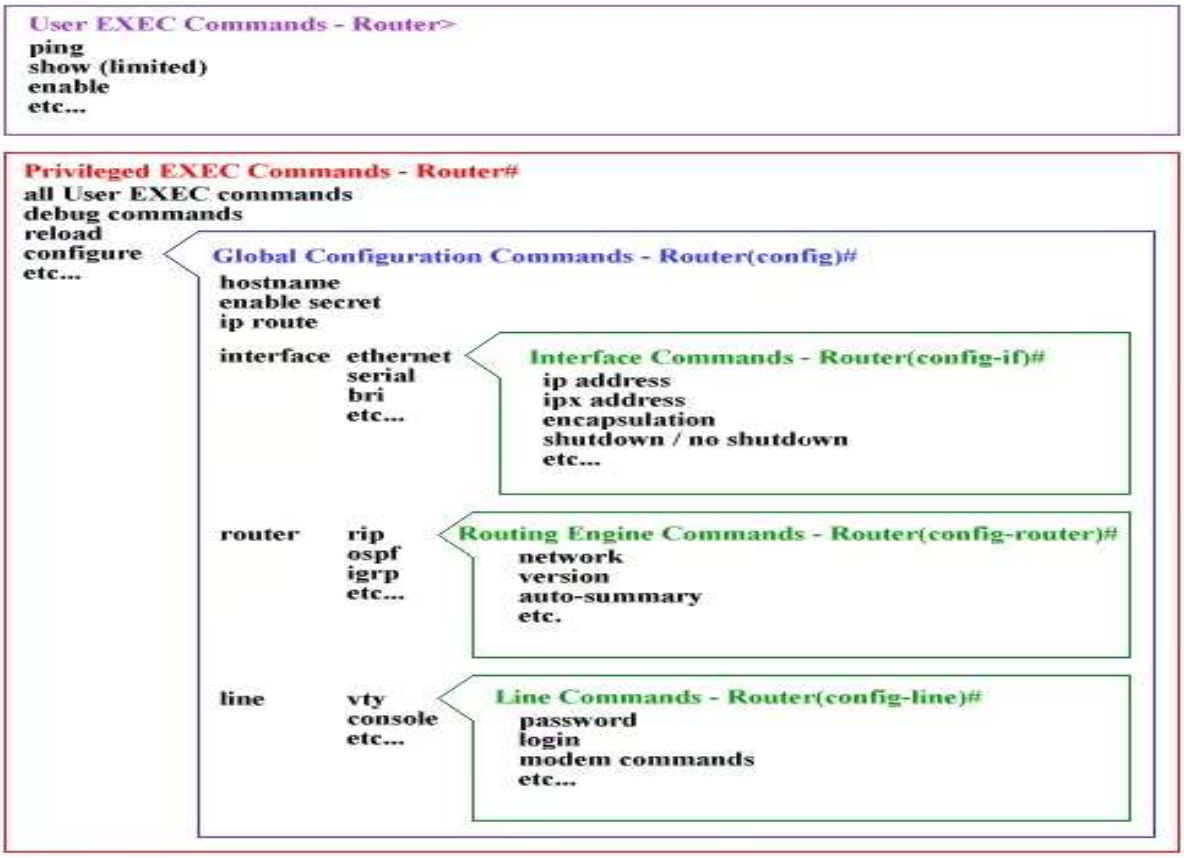
Step 14: The complete router configuration is done successfully. Now we can check whether the configuration is correct or not, using the ping command.

We need to give ping command from the host of network 1 to the host of network 2.

```
C:\>ping 192.168.20.10

Pinging 192.168.20.10 with 32 bytes of data:

Request timed out.
Reply from 192.168.20.10: bytes=32 time=11ms TTL=127
Reply from 192.168.20.10: bytes=32 time=11ms TTL=127
Reply from 192.168.20.10: bytes=32 time=12ms TTL=127

Ping statistics for 192.168.20.10:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
    Minimum = 11ms, Maximum = 12ms, Average = 11ms

C:\>ping 192.168.20.10

Pinging 192.168.20.10 with 32 bytes of data:

Reply from 192.168.20.10: bytes=32 time<1ms TTL=127
Reply from 192.168.20.10: bytes=32 time=15ms TTL=127
Reply from 192.168.20.10: bytes=32 time=15ms TTL=127
Reply from 192.168.20.10: bytes=32 time=37ms TTL=127

Ping statistics for 192.168.20.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 37ms, Average = 16ms
```

The above ping command from the host of network 1(192.168.10.10 or 192.168.10.20) is successful in communicating to the host in network2 (192.168.20.10 or 192.168.20.20).
Thus, we have successfully configured the router.

THE CISCO COMMAND HIERARCHY



EXPERIMENT-4

Perform Network Packet Analysis Using Wireshark.

What is Wireshark?

- Wireshark is a network packet analyzer. A network packet analyzer presents captured packet data in as much detail as possible.
- You could think of a network packet analyzer as a measuring device for examining what's happening inside a network cable, just like an electrician uses a voltmeter for examining what's happening inside an electric cable (but at a higher level, of course).
- In the past, such tools were either very expensive, proprietary, or both. However, with the advent of Wireshark, that has changed. Wireshark is available for free, is open source, and is one of the best packet analyzers available today.

Some intended purposes

- Network administrators use it to troubleshoot network problems
- Network security engineers use it to examine security problems
- QA engineers use it to verify network applications
- Developers use it to debug protocol implementations
- People use it to learn network protocol internals

Features

- Available for UNIX and Windows.
- Capture live packet data from a network interface.
- Open files containing packet data captured with tcpdump/WinDump, Wireshark, and many other packet capture programs.
- Import packets from text files containing hex dumps of packet data.
- Display packets with very detailed protocol information.
- Save packet data captured.
- Export some or all packets in a number of capture file formats.
- Filter packets on many criteria.
- Search for packets on many criteria.
- Colorize packet display based on filters.
- Create various statistics.

Installing Wireshark

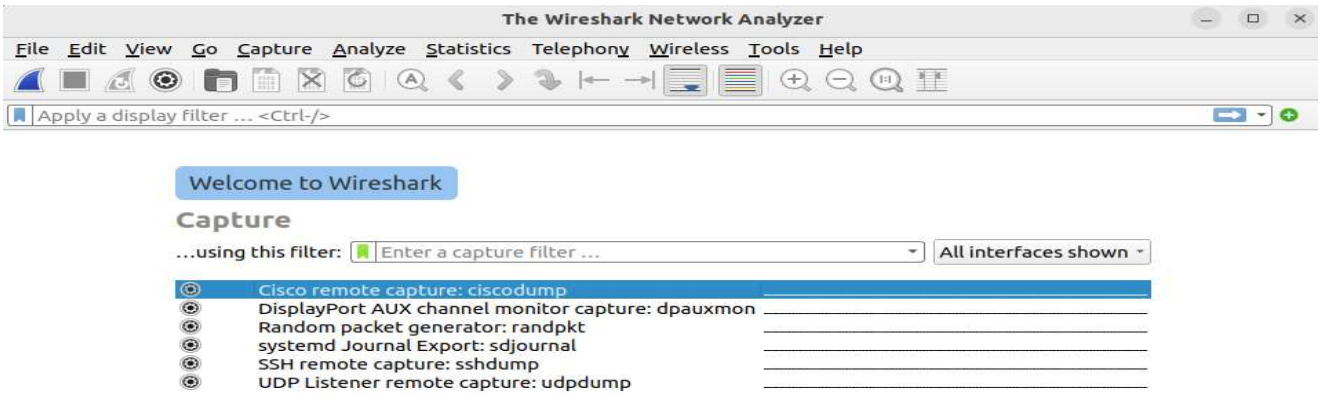
Step 1: Install Wireshark

```
joshua@JoshuaUbuntu:~/joshua/cn_programs/wireshark$ sudo apt install wireshark
[sudo] password for joshua:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

Step 2: Open Wireshark

```
joshua@JoshuaUbuntu:~/joshua/cn_programs/wireshark$ wireshark
 ** (wireshark:2713) 10:15:20.101206 [GUI WARNING] -- Warning: QT_DEVICE_PIXEL_R
ATIO is deprecated. Instead use:
   QT_AUTO_SCREEN_SCALE_FACTOR to enable platform plugin controlled per-screen f
actors.
   QT_SCREEN_SCALE_FACTORS to set per-screen DPI.
   QT_SCALE_FACTOR to set the application global scale factor.
```

Now, if the Wireshark interface does not show your Ethernet/Wireless LAN Networks it looks as follows:



Now, exit Wireshark (close the Wireshark application)

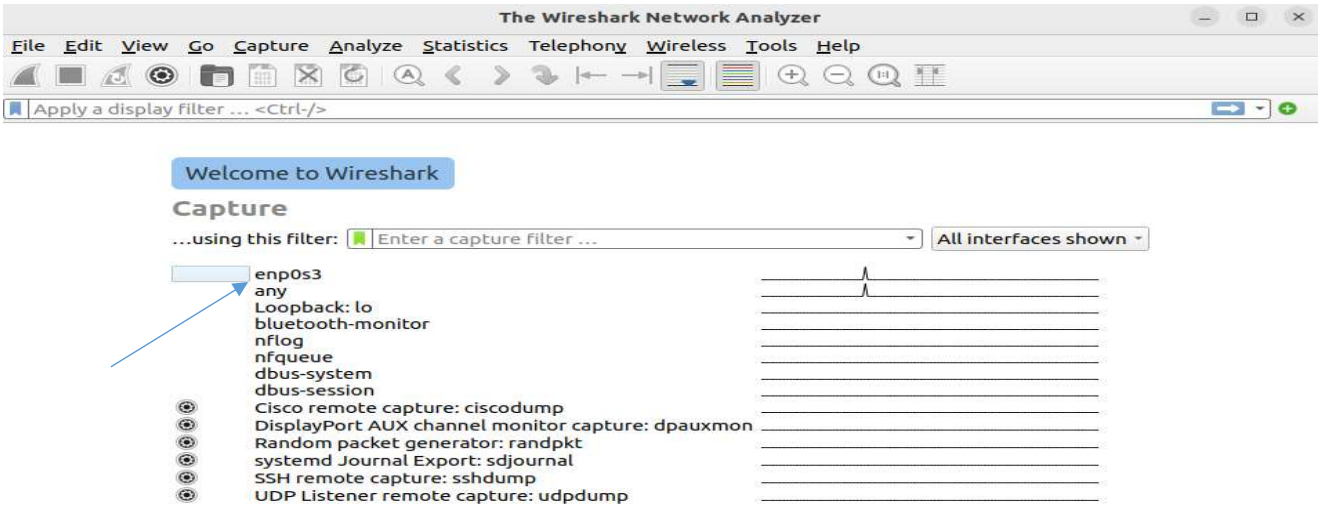
So, to view the network interface, execute the following command:

```
joshua@JoshuaUbuntu:~/joshua/cn_programs/wireshark$ sudo chmod +x /usr/bin/dumpcap
[sudo] password for joshua:
joshua@JoshuaUbuntu:~/joshua/cn_programs/wireshark$
```

Now, again open Wireshark:

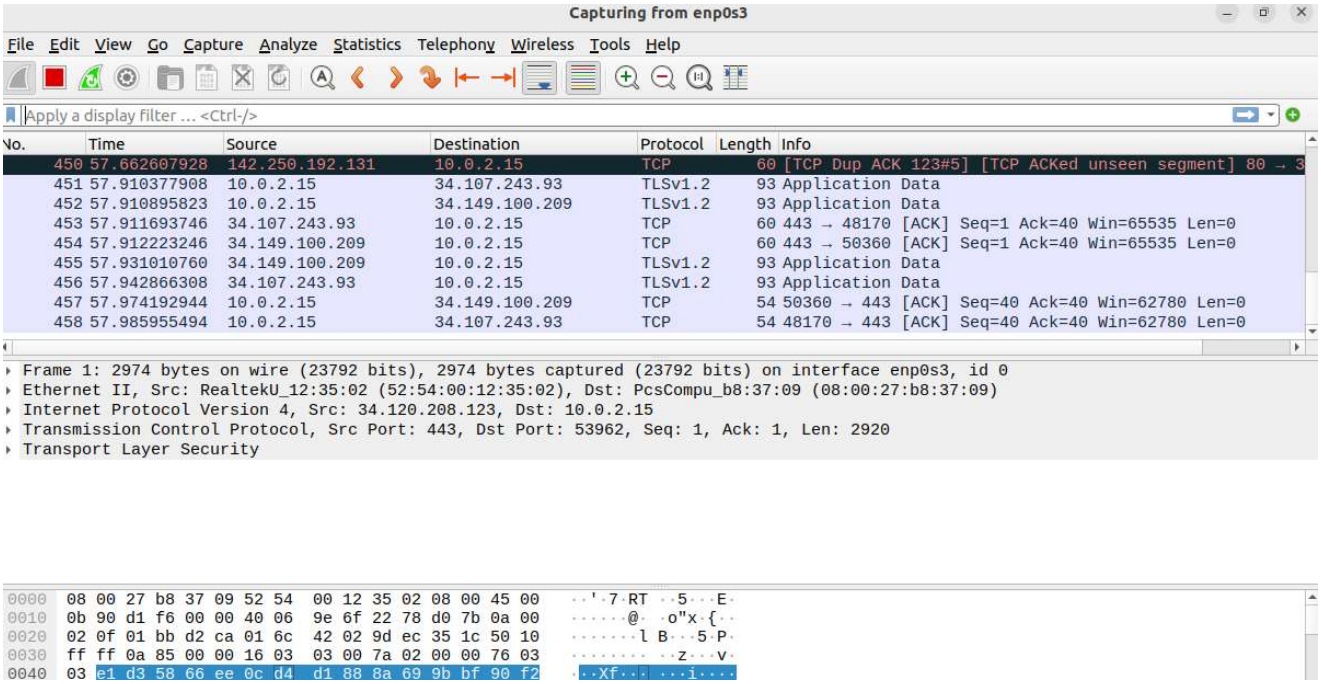
```
joshua@JoshuaUbuntu:~/joshua/cn_programs/wireshark$ wireshark
 ** (wireshark:2822) 10:32:53.808697 [GUI WARNING] -- Warning: QT_DEVICE_PIXEL_RATIO is d
eprecated. Instead use:
   QT_AUTO_SCREEN_SCALE_FACTOR to enable platform plugin controlled per-screen factors.
   QT_SCREEN_SCALE_FACTORS to set per-screen DPI.
   QT_SCALE_FACTOR to set the application global scale factor.
```

You will now see the available interfaces as follows:



We shall now start capturing the packets using the enp0s3 interface. (click on enp0s3 interface to see how the packets are captured.)

It looks as follows:



The Menu

File

This menu contains items to open and merge capture files, save, print, or export capture files in whole or in part, and to quit the Wireshark application.

Edit

This menu contains items to find a packet, time reference or mark one or more packets, handle configuration profiles, and set your preferences.

View

This menu controls the display of the captured data, including colorization of packets, zooming the font, showing a packet in a separate window, expanding and collapsing trees in packet details.

Go

This menu contains items to go to a specific packet.

Capture

This menu allows you to start and stop captures and to edit capture filters.

Analyze

This menu contains items to manipulate display filters, enable or disable the dissection of protocols, configure user specified decodes and follow a TCP stream.

Statistics

This menu contains items to display various statistic windows, including a summary of the packets that have been captured, display protocol hierarchy statistics and much more.

Telephony

This menu contains items to display various telephony related statistic windows, including a media analysis, flow diagrams, display protocol hierarchy statistics and much more.

Wireless

This menu contains items to display Bluetooth and IEEE 802.11 wireless statistics.

Tools

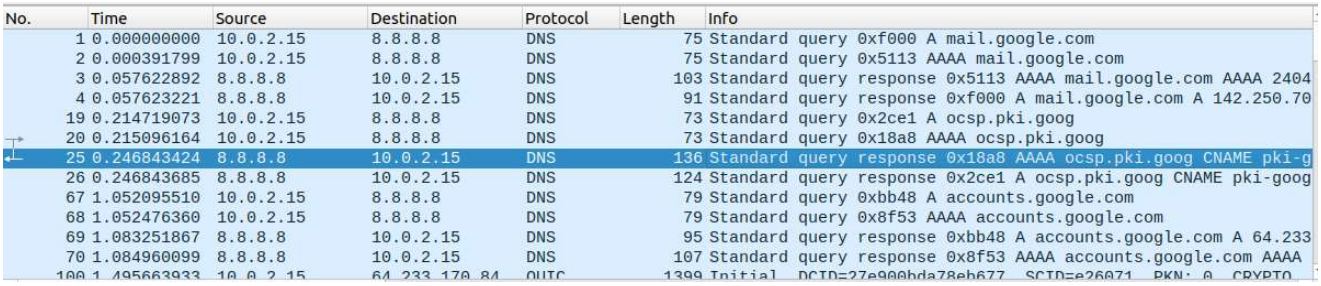
This menu contains various tools available in Wireshark, such as creating Firewall ACL Rules.

For more information visit:

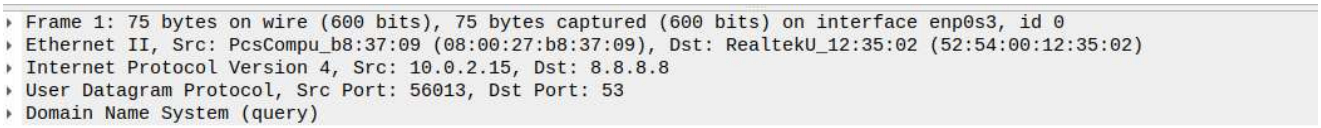
https://www.wireshark.org/docs/wsug_html_chunked/ChapterUsing.html

The "Packet List" Pane

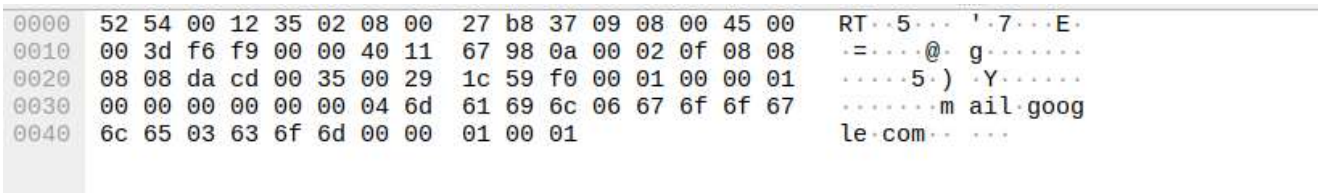
The packet list pane displays all the packets in the current capture file.



The "Packet Details" Pane



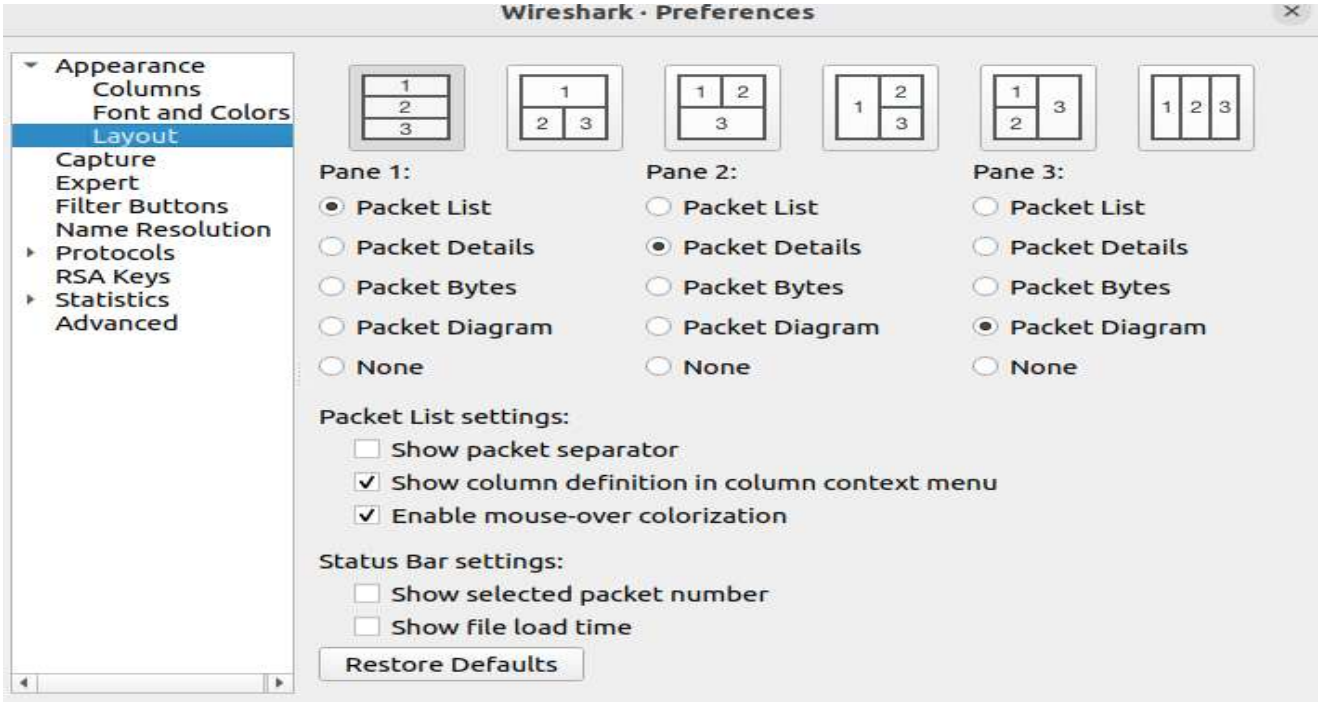
The "Packet Bytes" Pane



The default columns will show:

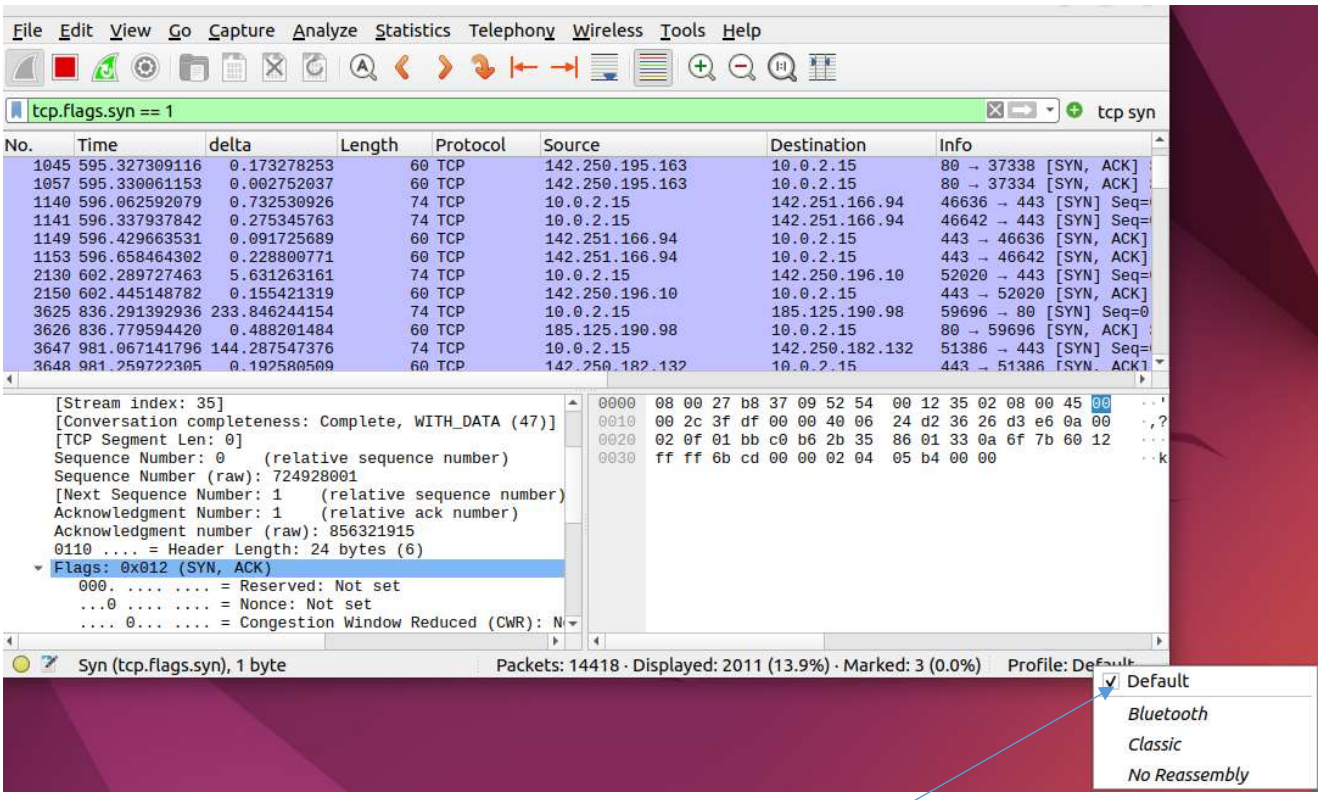
- **No.** The number of the packet in the capture file. This number won't change, even if a display filter is used.
- **Time** The timestamp of the packet. The presentation format of this timestamp can be changed.
- **Source** The address where this packet is coming from.
- **Destination** The address where this packet is going to.
- **Protocol** The protocol name in a short (perhaps abbreviated) version.
- **Length** The length of each packet.
- **Info** Additional information about the packet content.

TO VIEW A PARTICULAR PANE, GO TO: EDIT-> PREFERENCES->APPEARANCE->LAYOUT

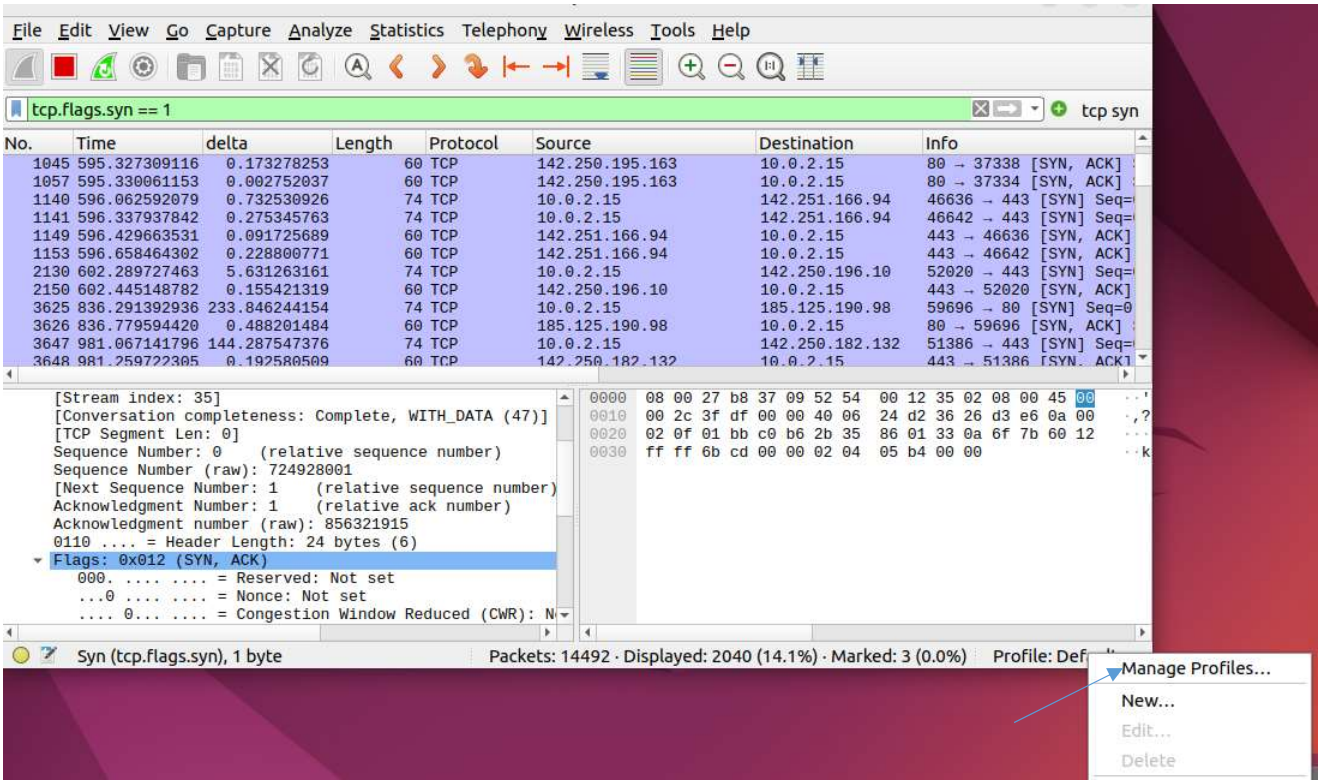


YOU CAN SELECT THE DIFFERENT VIEWS USING THE RADIO BUTTONS IN DIFFERENT PANES

PROFILE:

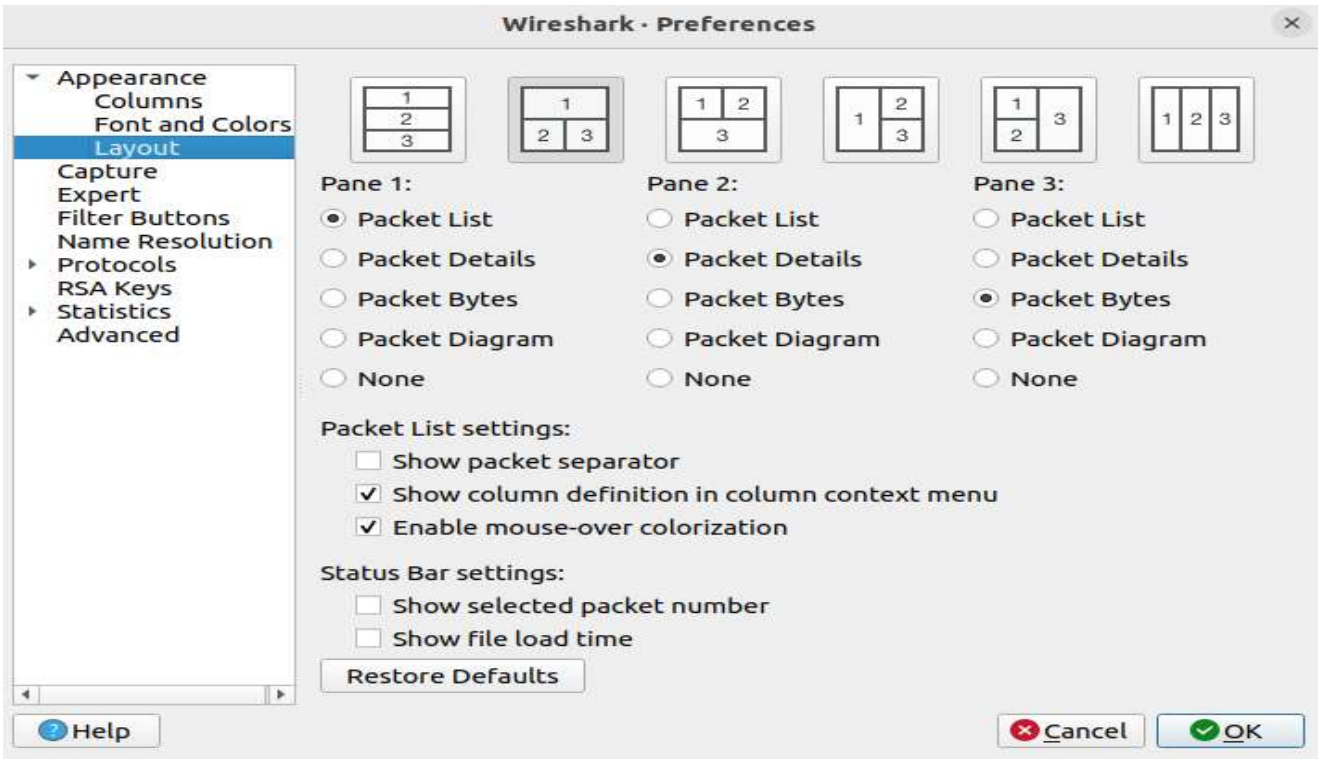


ADD NEW PROFILE:



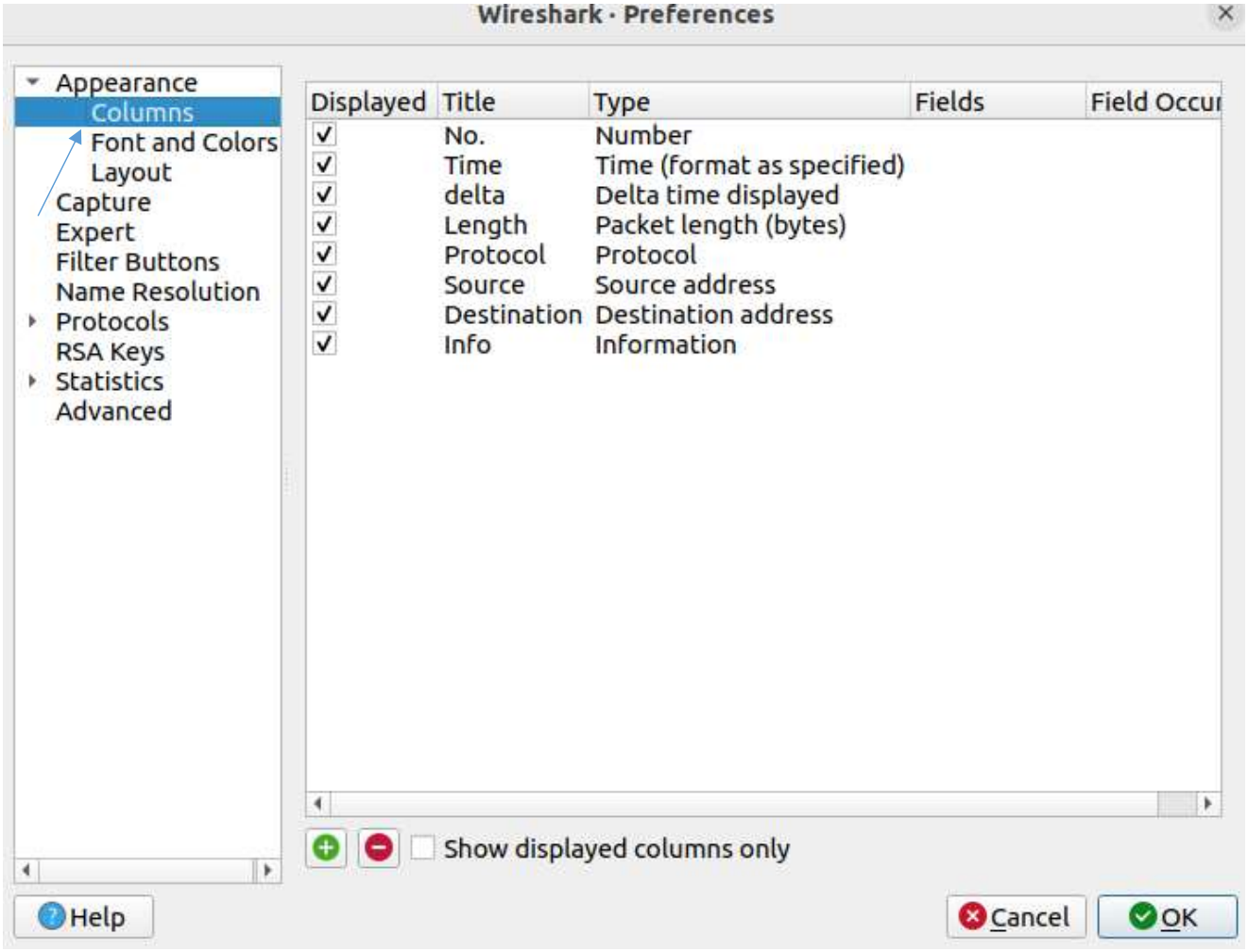
PREFERENCES:

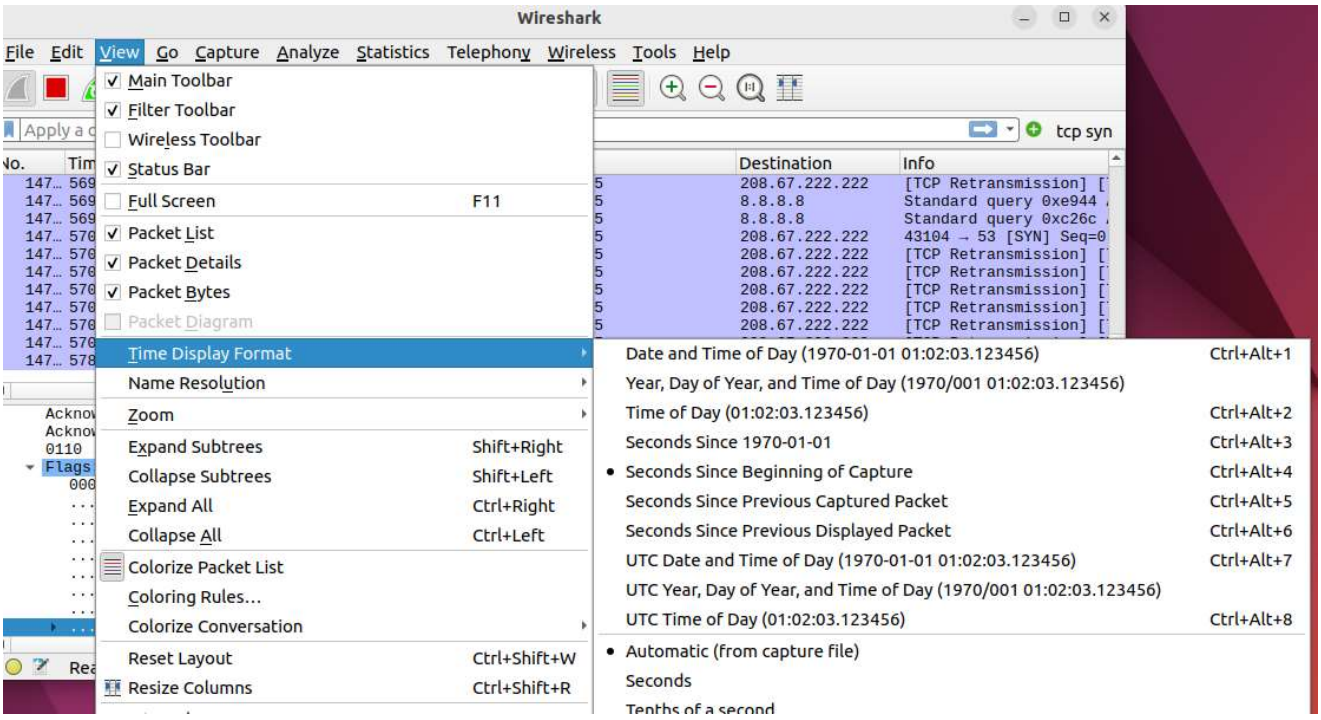
EDIT->PREFERENCES



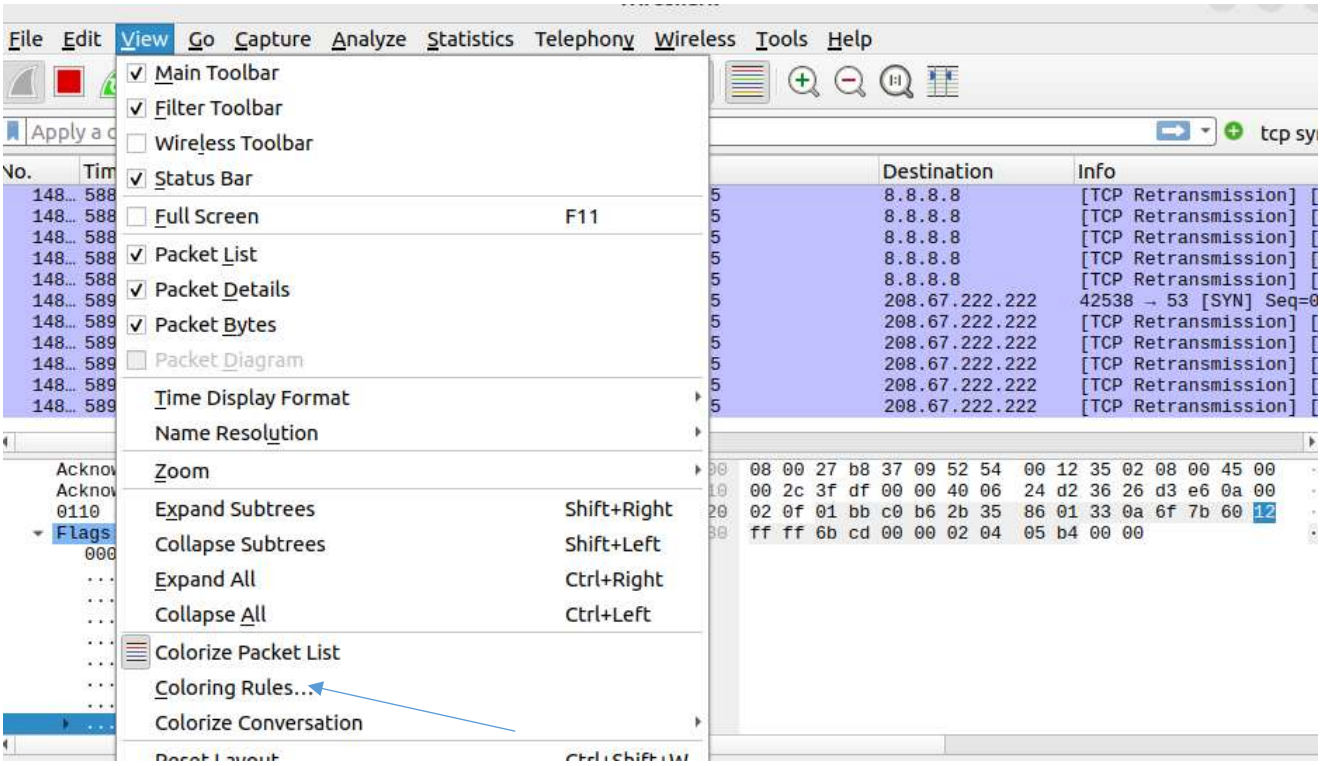
YOU CAN CHOOSE PACKET DIAGRAM AS PANE 3 IF YOU WANT.

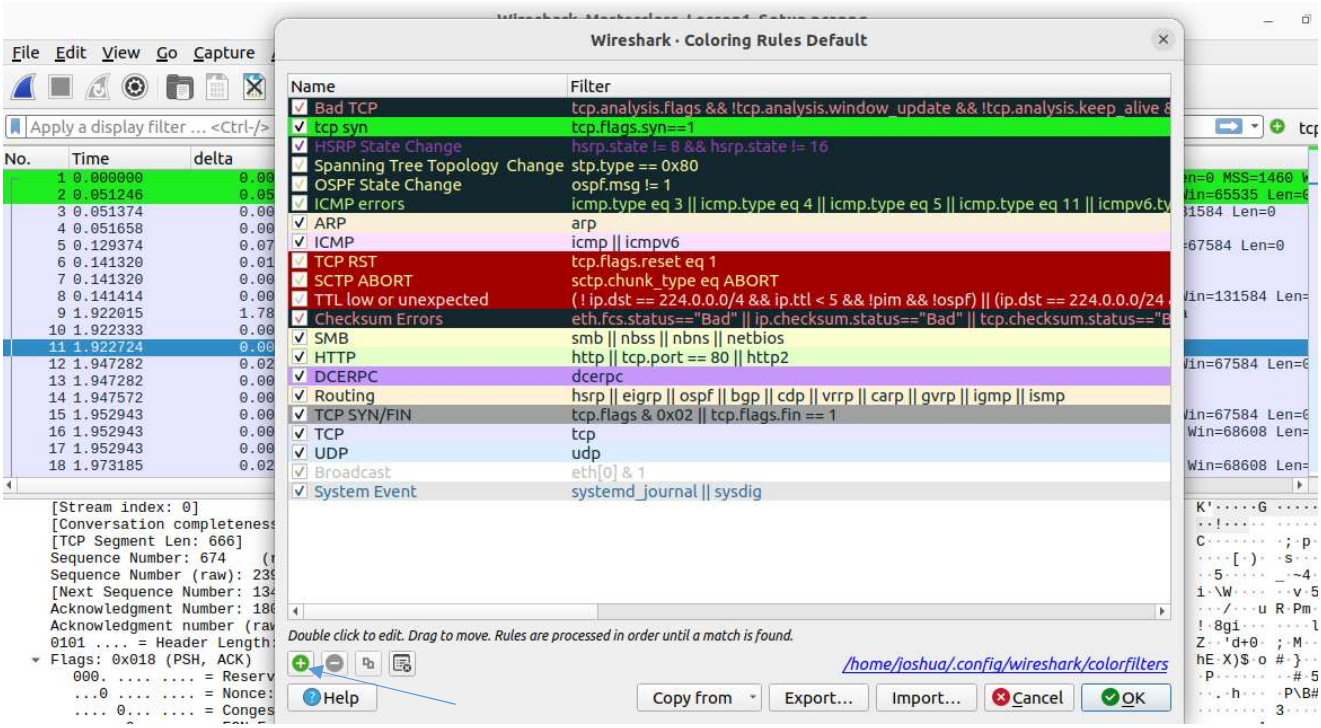
EDIT->PREFERENCES->COLUMNS



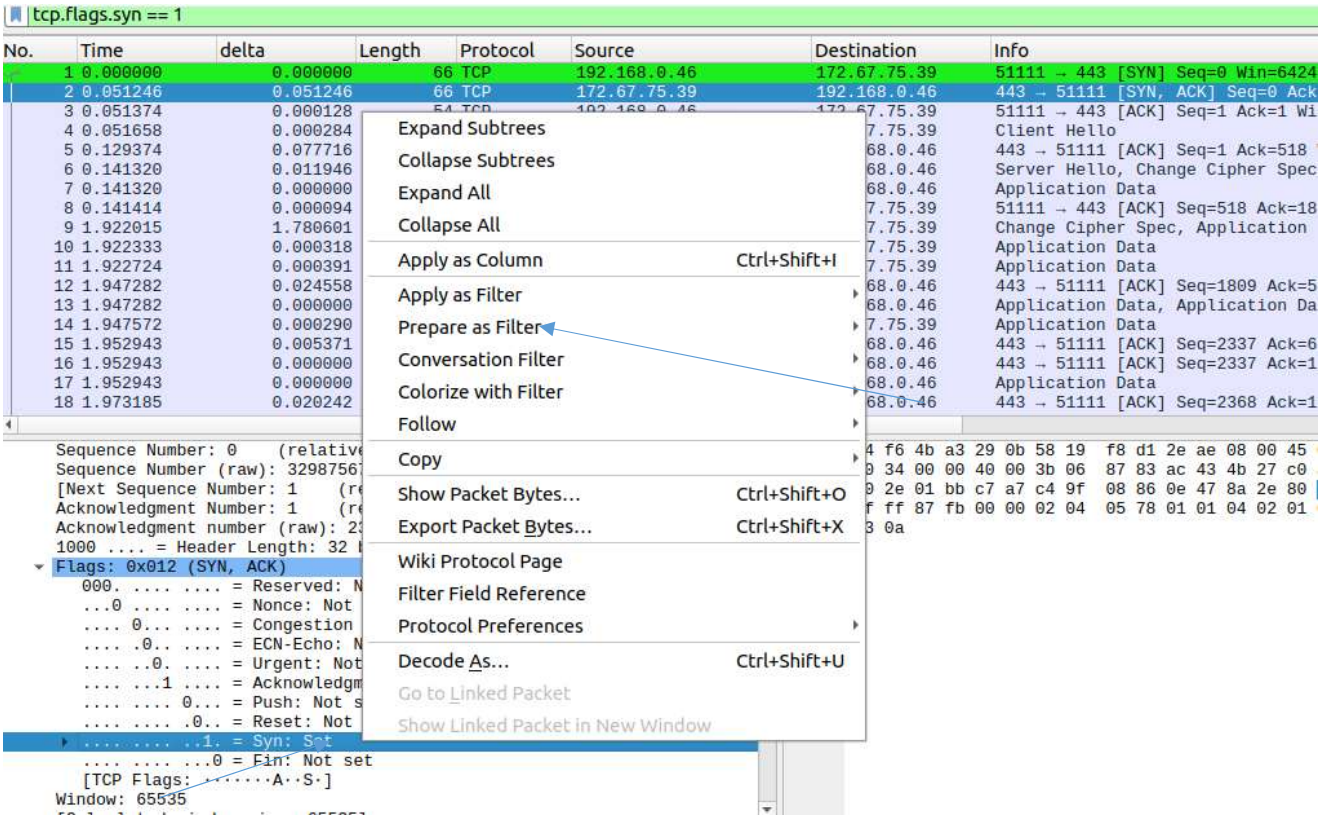


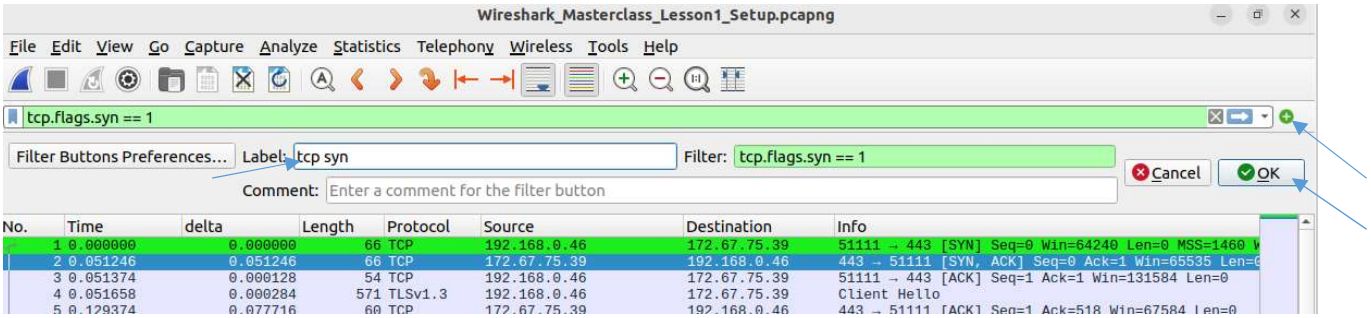
COLORING RULES:



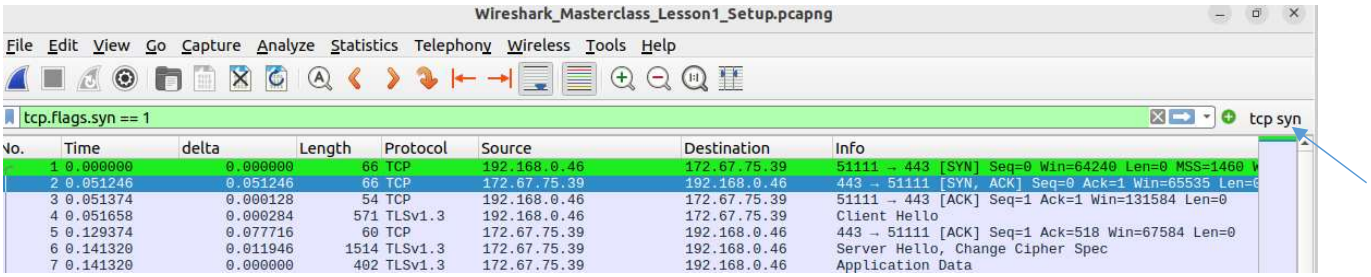


CREATING A BUTTON FOR TCP SYN:

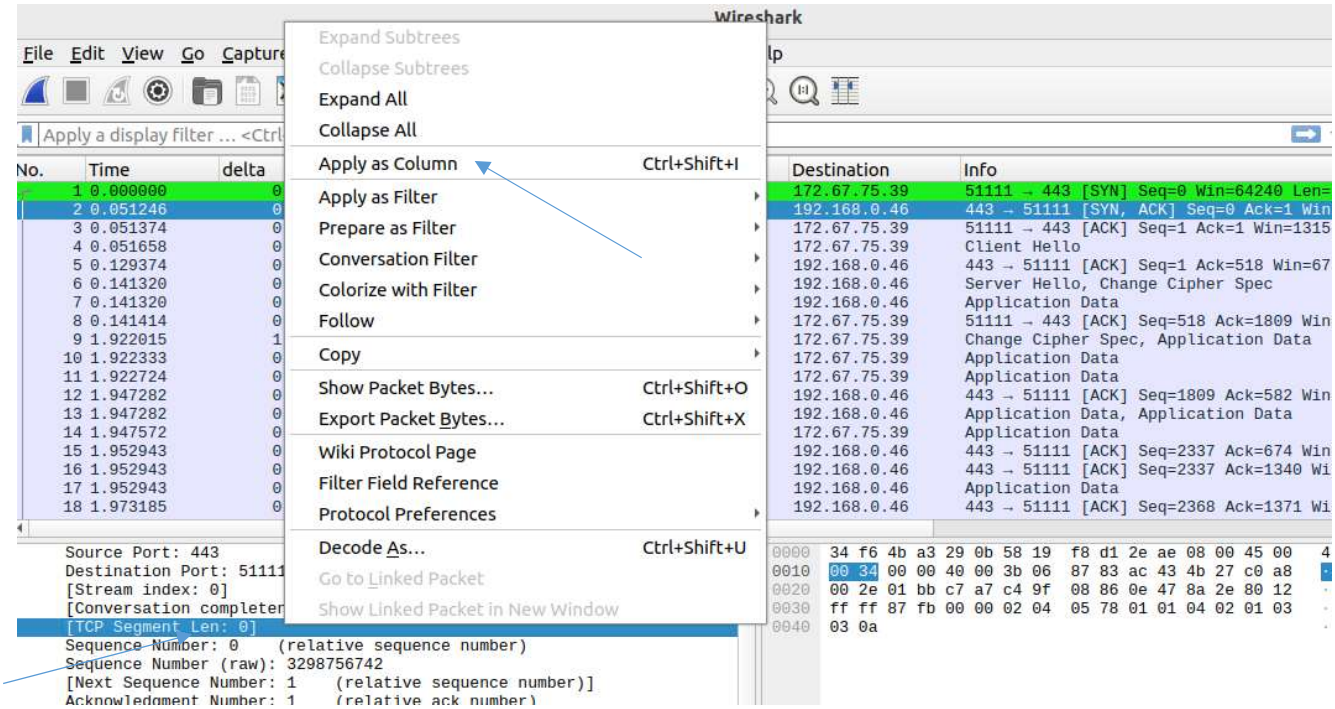




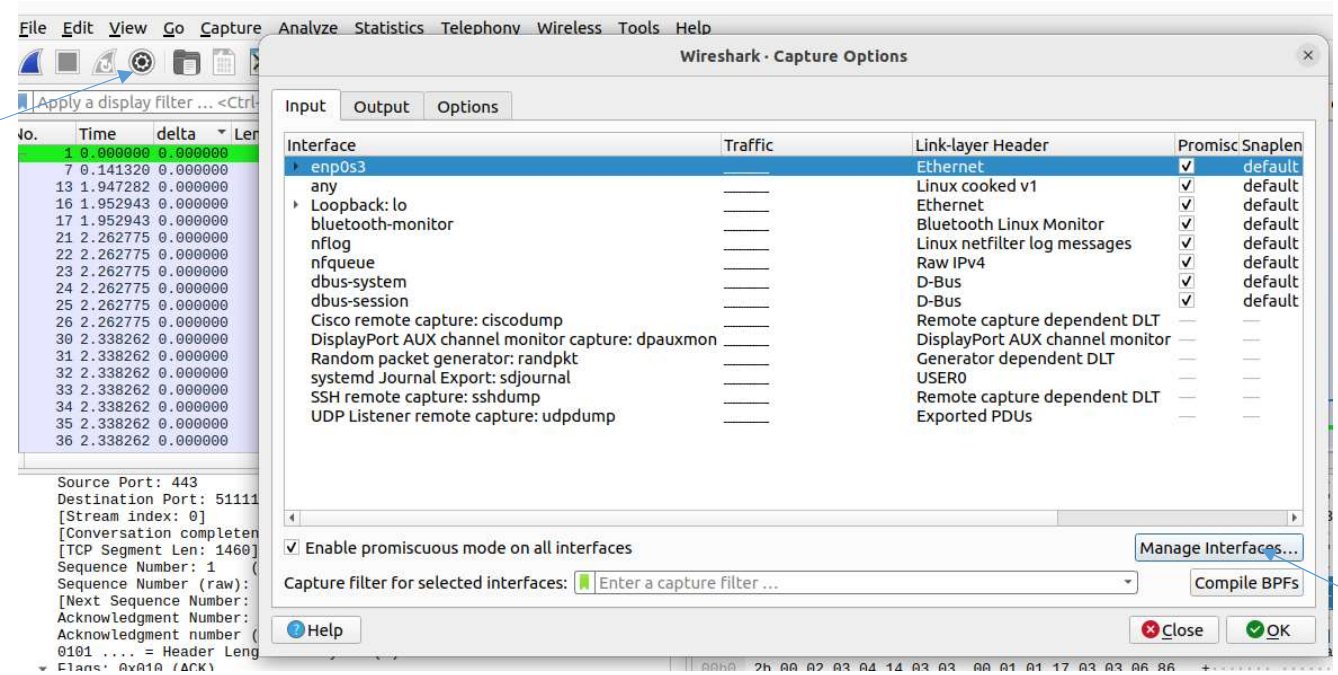
YOU CAN SEE THE BUTTON ADDED:



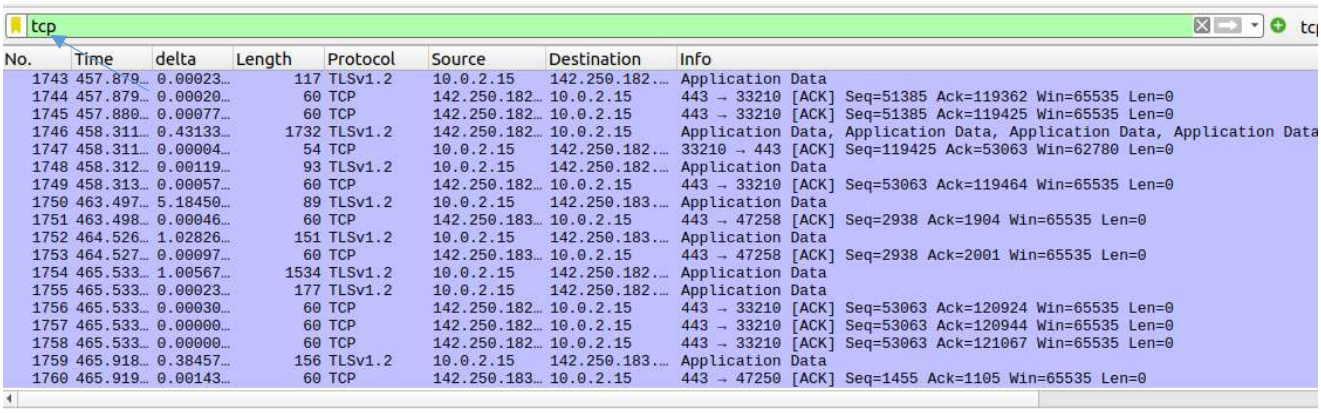
ADDING COLUMNS:

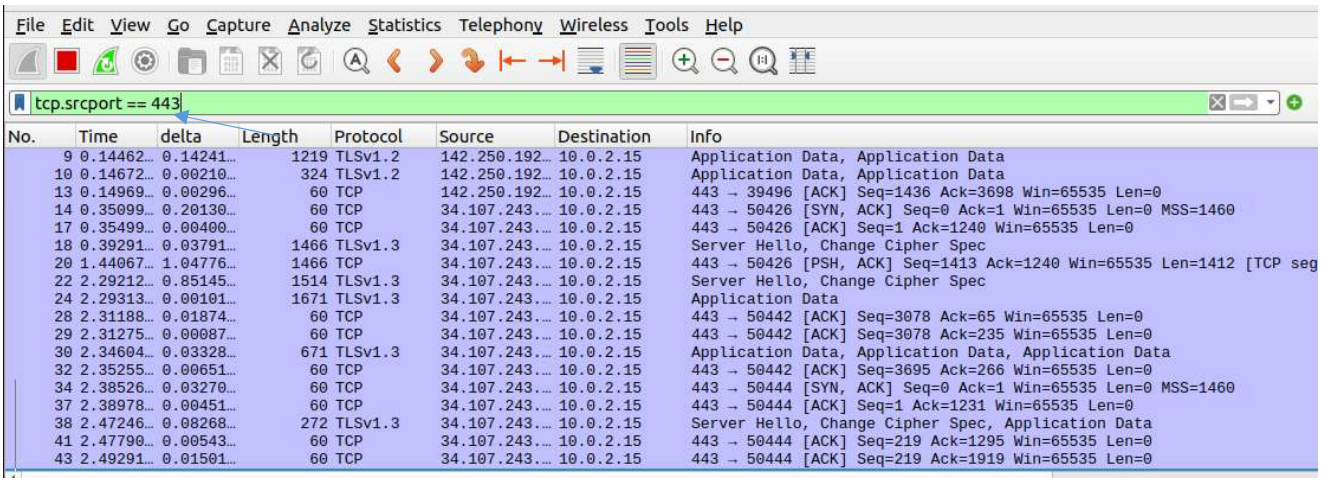


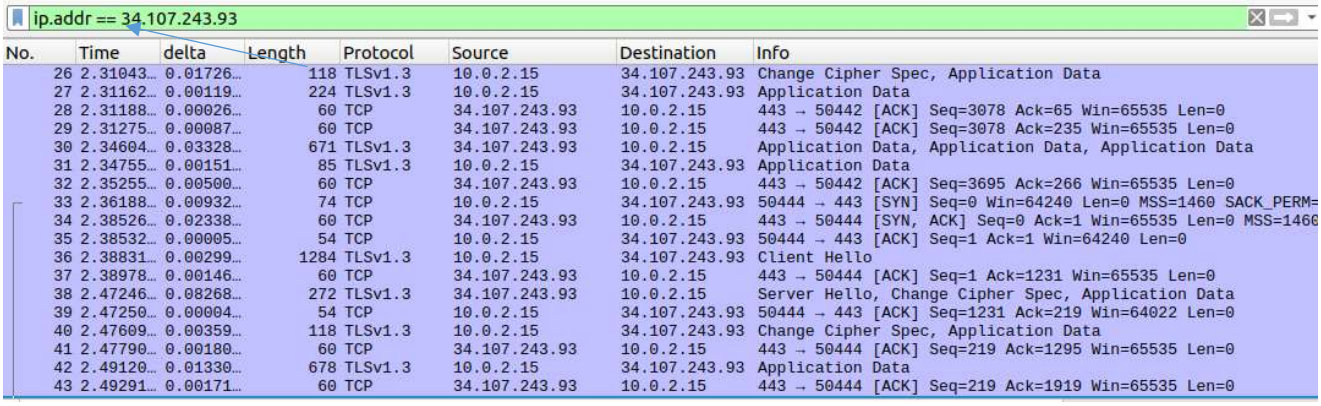
CAPTURING THE TRAFFIC:

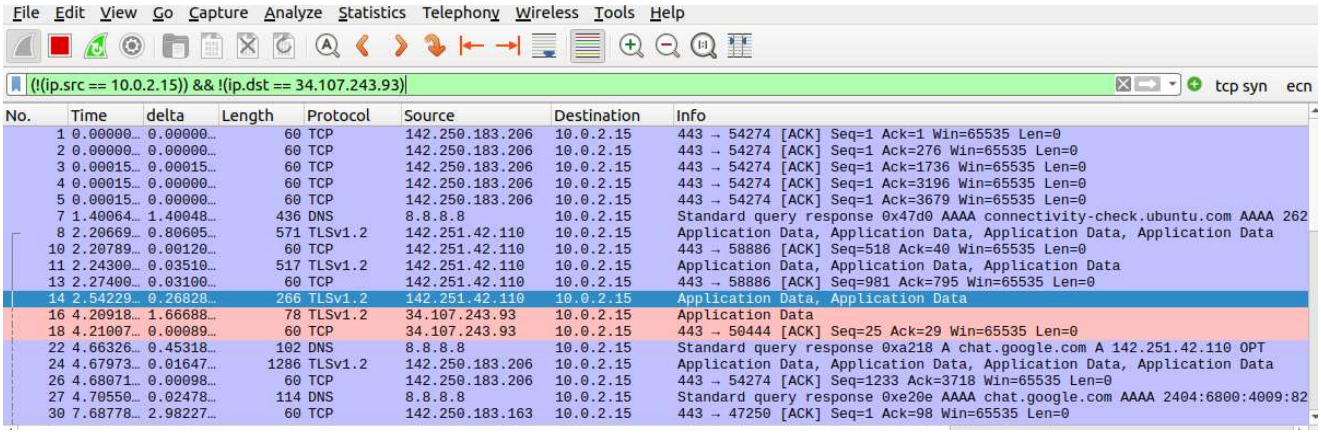


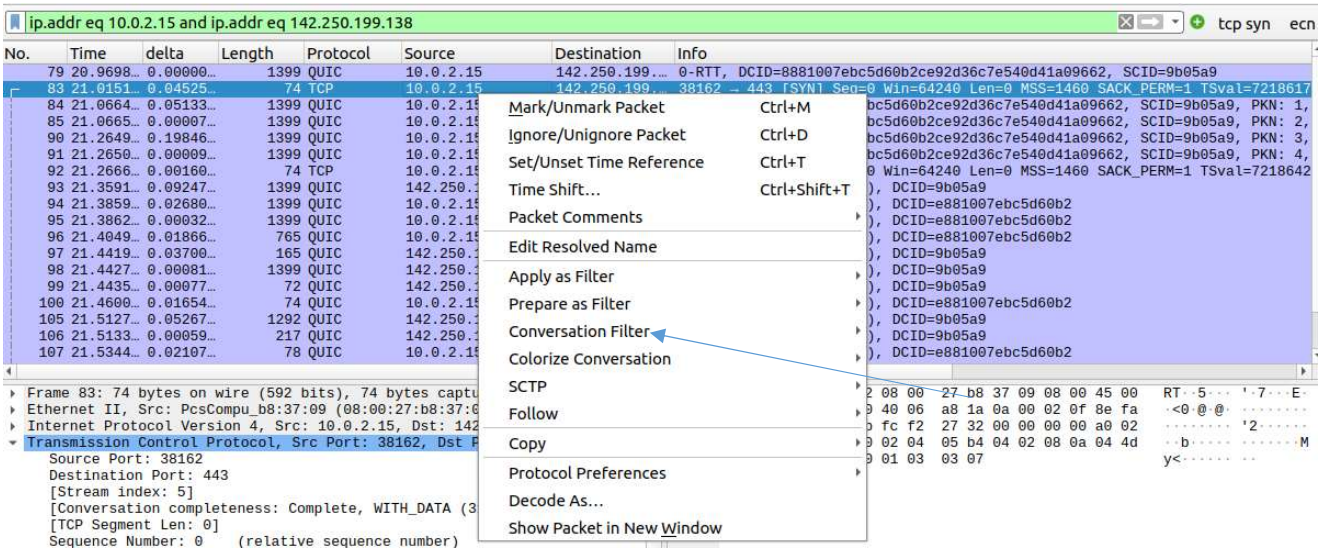
FILTERING TRAFFIC:

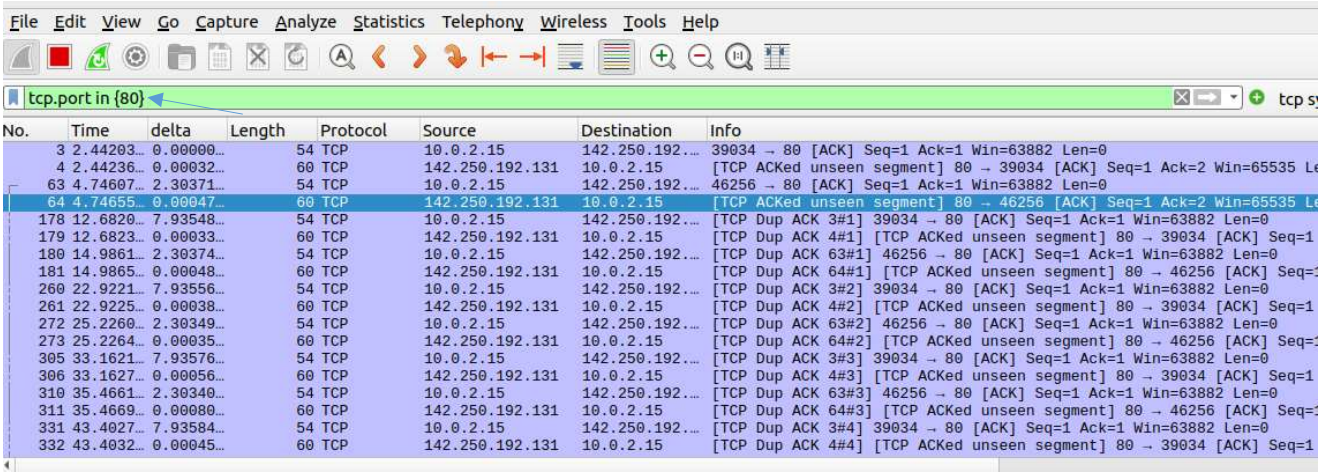




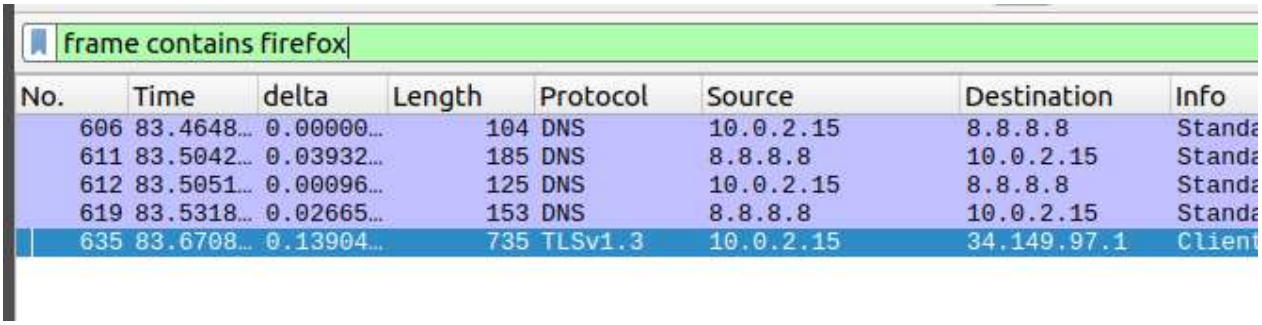


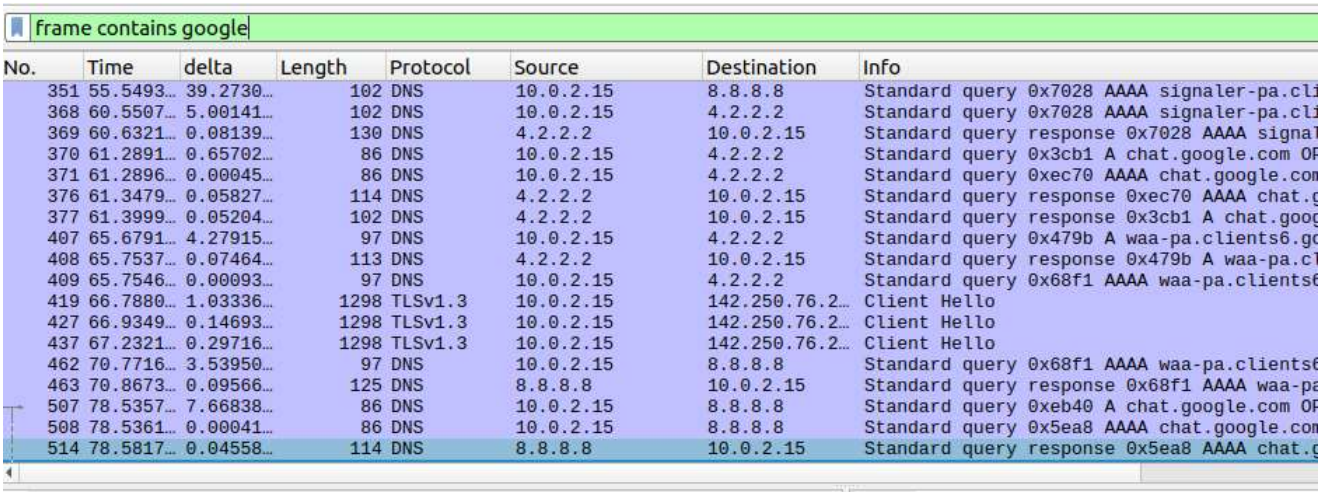

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
(!(ip.src == 10.0.2.15)) && !(ip.dst == 34.107.243.93)
tcp syn ecn
No. Time delta Length Protocol Source Destination Info
1 0.00000… 0.00000… 60 TCP 142.250.183.206 10.0.2.15 443 → 54274 [ACK] Seq=1 Ack=1 Win=65535 Len=0
2 0.00000… 0.00000… 60 TCP 142.250.183.206 10.0.2.15 443 → 54274 [ACK] Seq=1 Ack=276 Win=65535 Len=0
3 0.00015… 0.00015… 60 TCP 142.250.183.206 10.0.2.15 443 → 54274 [ACK] Seq=1 Ack=1736 Win=65535 Len=0
4 0.00015… 0.00000… 60 TCP 142.250.183.206 10.0.2.15 443 → 54274 [ACK] Seq=1 Ack=3196 Win=65535 Len=0
5 0.00015… 0.00000… 60 TCP 142.250.183.206 10.0.2.15 443 → 54274 [ACK] Seq=1 Ack=3679 Win=65535 Len=0
7 1.40064… 1.40048… 436 DNS 8.8.8.8 10.0.2.15 Standard query response 0x47d0 AAAA connectivity-check.ubuntu.com AAAA 262
8 2.20669… 0.80605… 571 TLSv1.2 142.251.42.110 10.0.2.15 Application Data, Application Data, Application Data, Application Data
10 2.20789… 0.00120… 60 TCP 142.251.42.110 10.0.2.15 443 → 58886 [ACK] Seq=518 Ack=40 Win=65535 Len=0
11 2.24300… 0.03510… 517 TLSv1.2 142.251.42.110 10.0.2.15 Application Data, Application Data, Application Data
13 2.27400… 0.03100… 60 TCP 142.251.42.110 10.0.2.15 443 → 58886 [ACK] Seq=981 Ack=795 Win=65535 Len=0
14 2.54229… 0.26828… 266 TLSv1.2 142.251.42.110 10.0.2.15 Application Data, Application Data
16 4.20918… 1.66688… 78 TLSv1.2 34.107.243.93 10.0.2.15 Application Data
18 4.21007… 0.00089… 60 TCP 34.107.243.93 10.0.2.15 443 → 50444 [ACK] Seq=25 Ack=29 Win=65535 Len=0
22 4.66326… 0.45318… 102 DNS 8.8.8.8 10.0.2.15 Standard query response 0xa218 A chat.google.com A 142.251.42.110 OPT
24 4.67973… 0.01647… 1286 TLSv1.2 142.250.183.206 10.0.2.15 Application Data, Application Data, Application Data, Application Data
26 4.68071… 0.00098… 60 TCP 142.250.183.206 10.0.2.15 443 → 54274 [ACK] Seq=1233 Ack=3718 Win=65535 Len=0
27 4.70550… 0.02478… 114 DNS 8.8.8.8 10.0.2.15 Standard query response 0xe20e AAAA chat.google.com AAAA 2404:6800:4009:82
30 7.68778… 2.98227… 60 TCP 142.250.183.163 10.0.2.15 443 → 47250 [ACK] Seq=1 Ack=98 Win=65535 Len=0


ip.addr eq 10.0.2.15 and ip.addr eq 142.250.199.138
tcp syn ecn
No. Time delta Length Protocol Source Destination Info
79 20.9698… 0.00000… 1399 QUIC 10.0.2.15 142.250.199.… 0-RTT, DCID=8881007ebc5d60b2ce92d36c7e540d41a09662, SCID=9b05a9
83 21.0151… 0.04525… 74 TCP 10.0.2.15 142.250.199.… 38162 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=7218617
84 21.0664… 0.05133… 1399 QUIC 10.0.2.15 bc5d60b2ce92d36c7e540d41a09662, SCID=9b05a9, PKN: 1,
85 21.0665… 0.00007… 1399 QUIC 10.0.2.15 bc5d60b2ce92d36c7e540d41a09662, SCID=9b05a9, PKN: 2,
90 21.2649… 0.19846… 1399 QUIC 10.0.2.15 bc5d60b2ce92d36c7e540d41a09662, SCID=9b05a9, PKN: 3,
91 21.2650… 0.00009… 1399 QUIC 10.0.2.15 bc5d60b2ce92d36c7e540d41a09662, SCID=9b05a9, PKN: 4,
92 21.2666… 0.00160… 74 TCP 10.0.2.15 0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=7218642
93 21.3591… 0.09247… 1399 QUIC 142.250.1), DCID=9b05a9
94 21.3859… 0.02680… 1399 QUIC 10.0.2.15), DCID=e881007ebc5d60b2
95 21.3862… 0.00032… 1399 QUIC 10.0.2.15), DCID=e881007ebc5d60b2
96 21.4049… 0.01866… 765 QUIC 10.0.2.15), DCID=e881007ebc5d60b2
97 21.4419… 0.03700… 165 QUIC 142.250.1), DCID=9b05a9
98 21.4427… 0.00081… 1399 QUIC 142.250.1), DCID=9b05a9
99 21.4435… 0.00077… 72 QUIC 142.250.1), DCID=9b05a9
100 21.4600… 0.01654… 74 QUIC 10.0.2.15), DCID=e881007ebc5d60b2
105 21.5127… 0.05267… 1292 QUIC 142.250.1), DCID=9b05a9
106 21.5133… 0.00059… 217 QUIC 142.250.1), DCID=9b05a9
107 21.5344… 0.02107… 78 QUIC 10.0.2.15), DCID=e881007ebc5d60b2
Frame 83: 74 bytes on wire (592 bits), 74 bytes captu
Ethernet II, Src: PcsCompu_b8:37:09 (08:00:27:b8:37:0
Internet Protocol Version 4, Src: 10.0.2.15, Dst: 142
Transmission Control Protocol, Src Port: 38162, Dst P
Source Port: 38162
Destination Port: 443
[Stream index: 5]
[Conversation completeness: Complete, WITH_DATA (3
[TCP Segment Len: 0]
Sequence Number: 0 (relative sequence number)
2 08 00 27 b8 37 09 08 00 45 00 RT 5 ' 7 E
9 40 06 a8 1a 0a 00 02 0f 8e fa <0 @ @
b fc f2 27 32 00 00 00 00 a0 02 '2
9 02 04 05 b4 04 02 08 0a 04 4d b M
9 01 03 03 07 y<
Mark/Unmark Packet Ctrl+M
Ignore/Unignore Packet Ctrl+D
Set/Unset Time Reference Ctrl+T
Time Shift… Ctrl+Shift+T
Packet Comments
Edit Resolved Name
Apply as Filter
Prepare as Filter
Conversation Filter
Colorize Conversation
SCTP
Follow
Copy
Protocol Preferences
Decode As…
Show Packet in New Window


File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp.port in {80}
tcp s
No. Time delta Length Protocol Source Destination Info
3 2.44203… 0.00000… 54 TCP 10.0.2.15 142.250.192.… 39034 → 80 [ACK] Seq=1 Ack=1 Win=63882 Len=0
4 2.44236… 0.00032… 60 TCP 142.250.192.131 10.0.2.15 [TCP ACKed unseen segment] 80 → 39034 [ACK] Seq=1 Ack=2 Win=65535 Le
63 4.74607… 2.30371… 54 TCP 10.0.2.15 142.250.192.… 46256 → 80 [ACK] Seq=1 Ack=1 Win=63882 Len=0
64 4.74655… 0.00047… 60 TCP 142.250.192.131 10.0.2.15 [TCP ACKed unseen segment] 80 → 46256 [ACK] Seq=1 Ack=2 Win=65535 Le
178 12.6820… 7.93548… 54 TCP 10.0.2.15 142.250.192.… [TCP Dup ACK 3#1] 39034 → 80 [ACK] Seq=1 Ack=1 Win=63882 Len=0
179 12.6823… 0.00033… 60 TCP 142.250.192.131 10.0.2.15 [TCP Dup ACK 4#1] [TCP ACKed unseen segment] 80 → 39034 [ACK] Seq=1
180 14.9861… 2.30374… 54 TCP 10.0.2.15 142.250.192.… [TCP Dup ACK 63#1] 46256 → 80 [ACK] Seq=1 Ack=1 Win=63882 Len=0
181 14.9865… 0.00048… 60 TCP 142.250.192.131 10.0.2.15 [TCP Dup ACK 64#1] [TCP ACKed unseen segment] 80 → 46256 [ACK] Seq=1
260 22.9221… 7.93556… 54 TCP 10.0.2.15 142.250.192.… [TCP Dup ACK 3#2] 39034 → 80 [ACK] Seq=1 Ack=1 Win=63882 Len=0
261 22.9225… 0.00038… 60 TCP 142.250.192.131 10.0.2.15 [TCP Dup ACK 4#2] [TCP ACKed unseen segment] 80 → 39034 [ACK] Seq=1
272 25.2260… 2.30349… 54 TCP 10.0.2.15 142.250.192.… [TCP Dup ACK 63#2] 46256 → 80 [ACK] Seq=1 Ack=1 Win=63882 Len=0
273 25.2264… 0.00035… 60 TCP 142.250.192.131 10.0.2.15 [TCP Dup ACK 64#2] [TCP ACKed unseen segment] 80 → 46256 [ACK] Seq=1
305 33.1621… 7.93576… 54 TCP 10.0.2.15 142.250.192.… [TCP Dup ACK 3#3] 39034 → 80 [ACK] Seq=1 Ack=1 Win=63882 Len=0
306 33.1627… 0.00056… 60 TCP 142.250.192.131 10.0.2.15 [TCP Dup ACK 4#3] [TCP ACKed unseen segment] 80 → 39034 [ACK] Seq=1
310 35.4661… 2.30340… 54 TCP 10.0.2.15 142.250.192.… [TCP Dup ACK 63#3] 46256 → 80 [ACK] Seq=1 Ack=1 Win=63882 Len=0
311 35.4669… 0.00080… 60 TCP 142.250.192.131 10.0.2.15 [TCP Dup ACK 64#3] [TCP ACKed unseen segment] 80 → 46256 [ACK] Seq=1
331 43.4027… 7.93584… 54 TCP 10.0.2.15 142.250.192.… [TCP Dup ACK 3#4] 39034 → 80 [ACK] Seq=1 Ack=1 Win=63882 Len=0
332 43.4032… 0.00045… 60 TCP 142.250.192.131 10.0.2.15 [TCP Dup ACK 4#4] [TCP ACKed unseen segment] 80 → 39034 [ACK] Seq=1

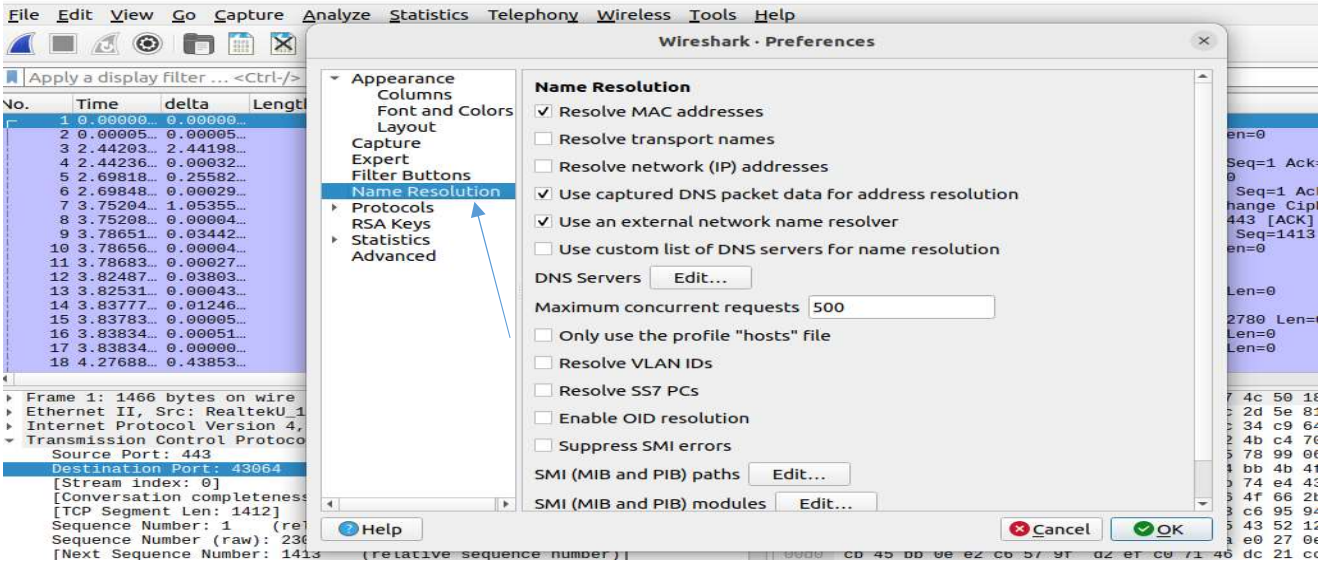

frame contains sbi
No. Time delta Length Protocol Source Destination Info
1631 290.857… 0.00000… 91 DNS 10.0.2.15 8.8.8.8 Standard query 0x7f86 A retail.onlinesbi.sbi OPT
1632 290.981… 0.12334… 91 DNS 10.0.2.15 8.8.8.8 Standard query 0x14e8 AAAA retail.onlinesbi.sbi OPT
1655 295.863… 4.88265… 91 DNS 10.0.2.15 4.2.2.2 Standard query 0x7f86 A retail.onlinesbi.sbi OPT
1656 295.986… 0.12245… 91 DNS 10.0.2.15 4.2.2.2 Standard query 0x14e8 AAAA retail.onlinesbi.sbi OPT



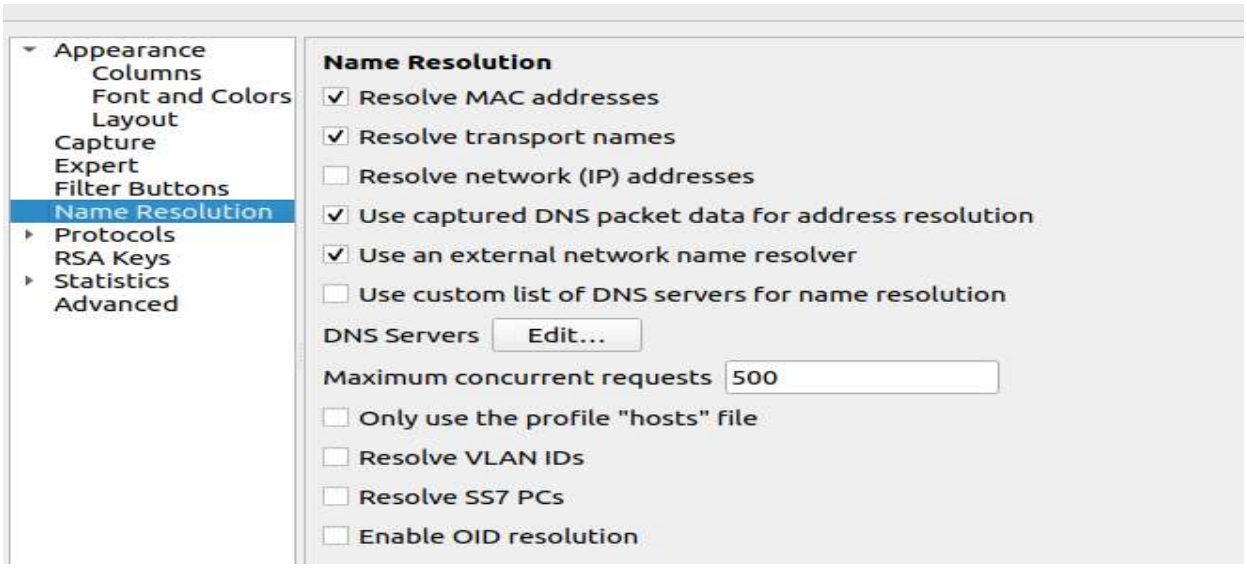


NAME RESOLUTION:

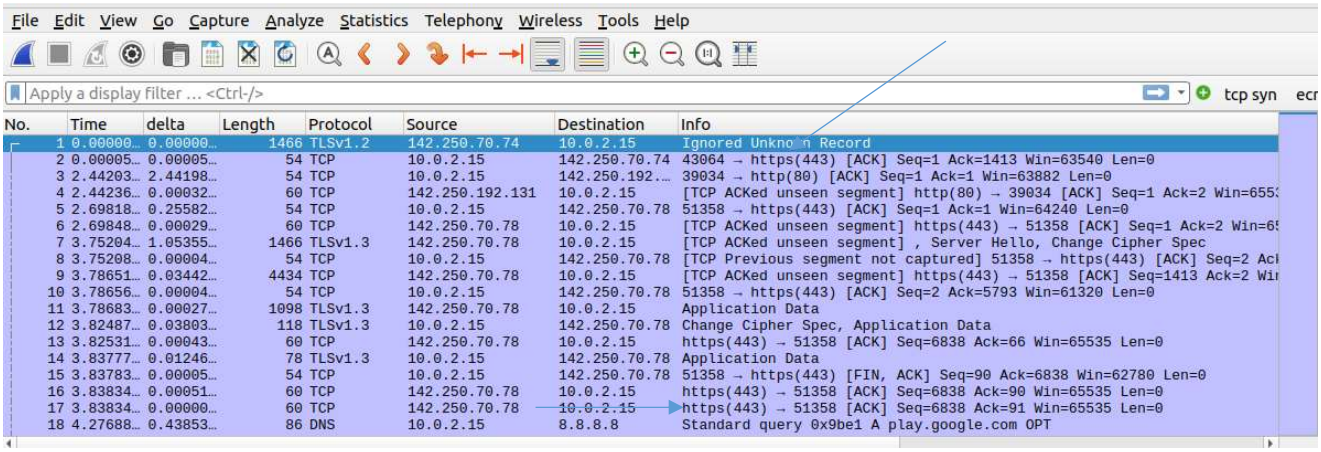
EDIT->PREFERENCES->NAME RESOLUTION

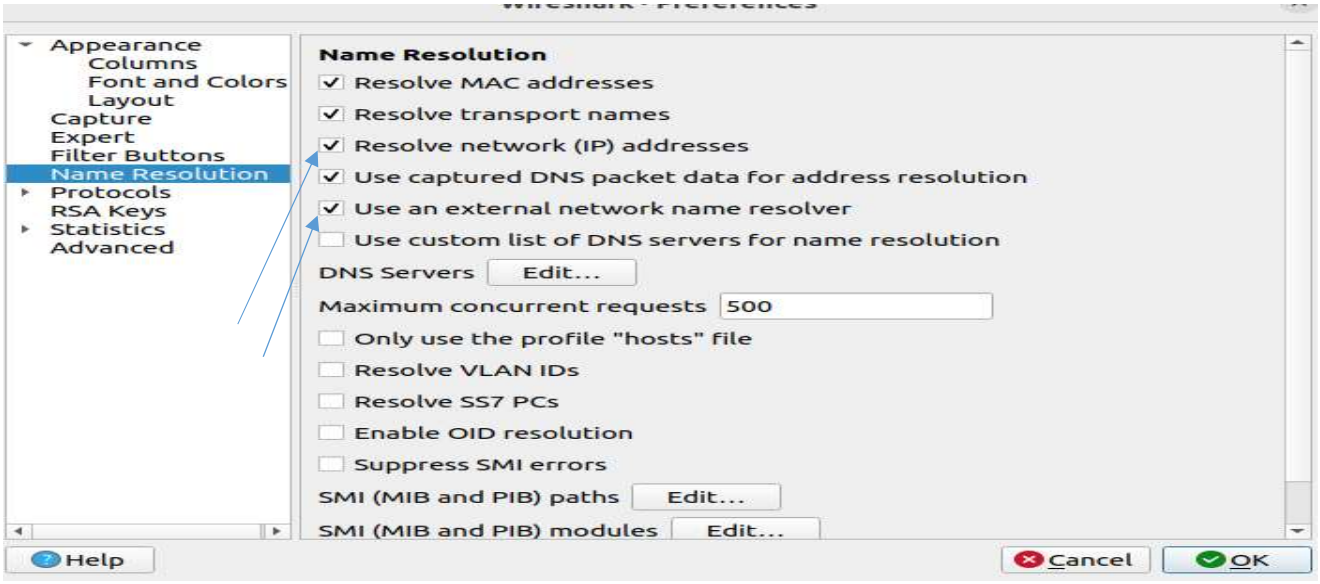


Check the “Resolve transport names” field

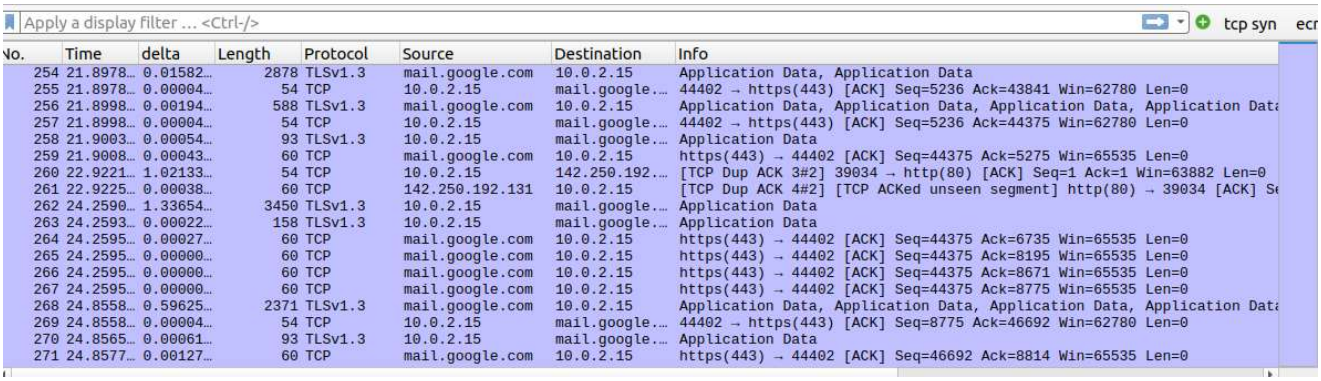


You will now see the transport names in your packet list:

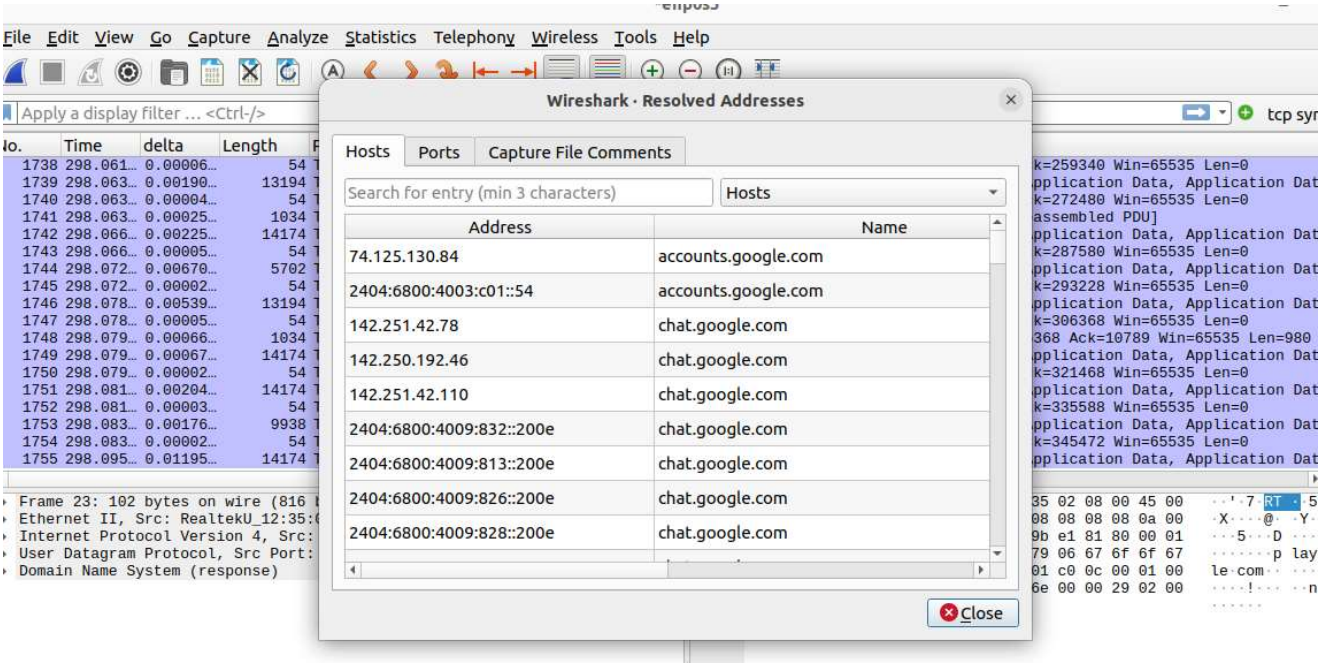




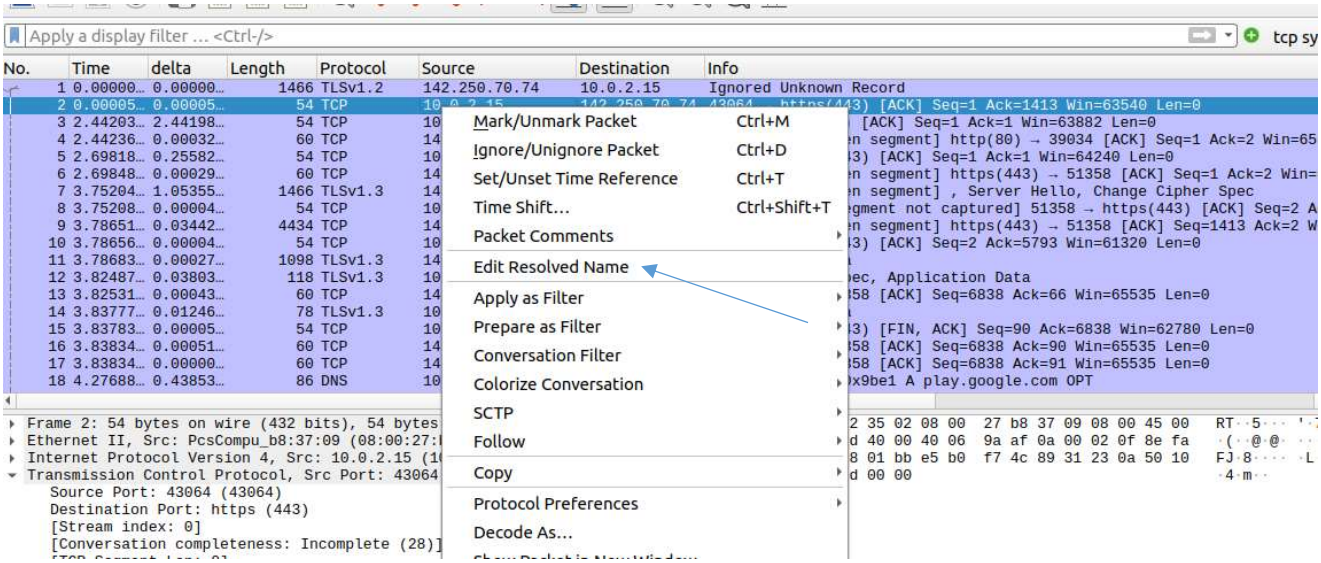
Source and destination names are resolved after checking the above options:

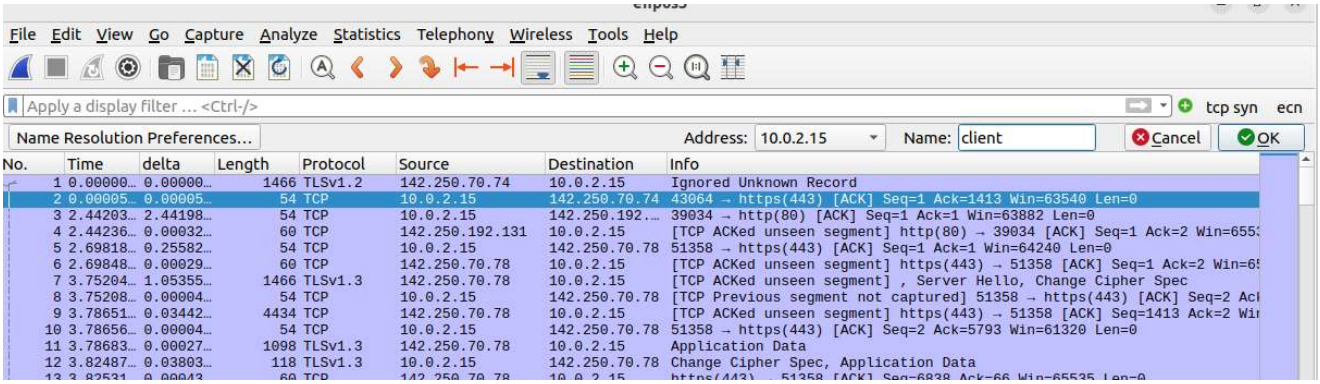


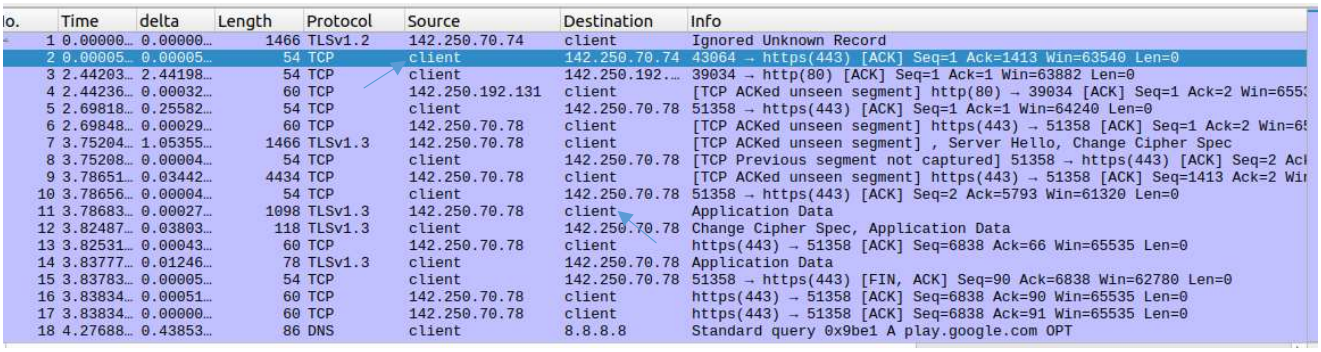
Now, go to statistics-> resolved addresses, you will find the ip addresses and resolved names.



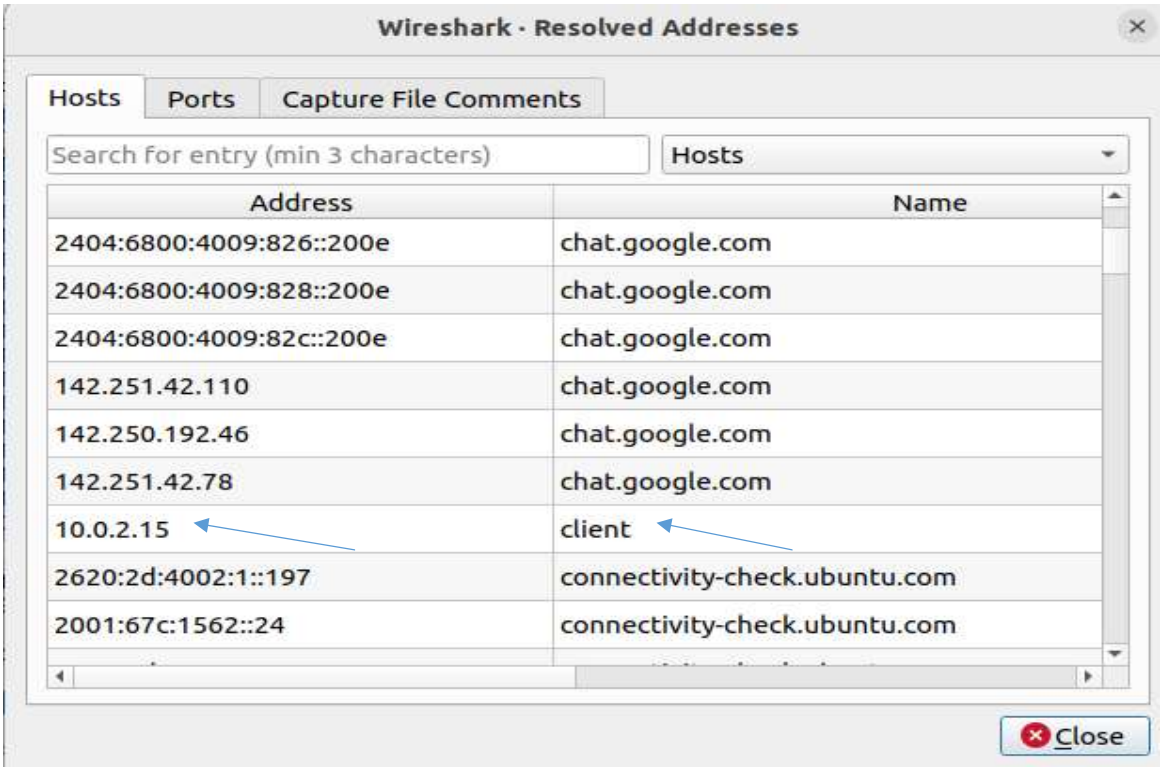
We can add names to our private ip addresses also (right click on a record):







Now, go to statistics-> resolved addresses, you will find the ip addresses and resolved name of newly added client.



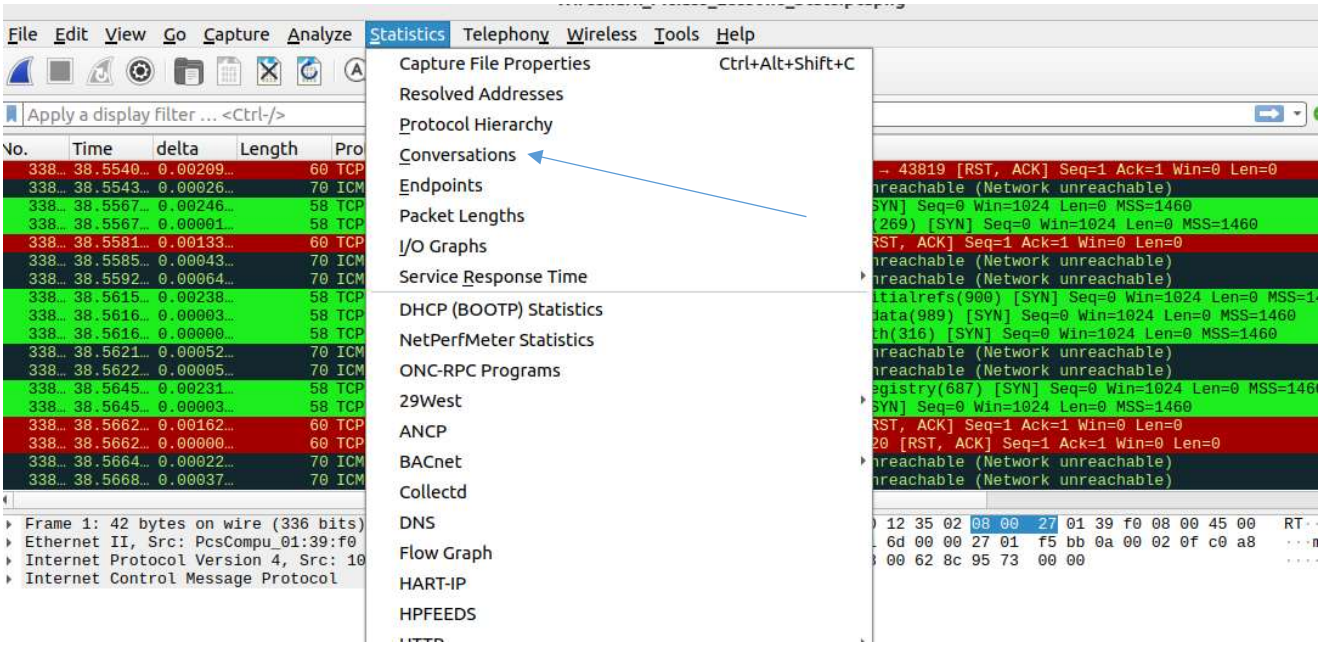
WIRESHARK STATISTICS:

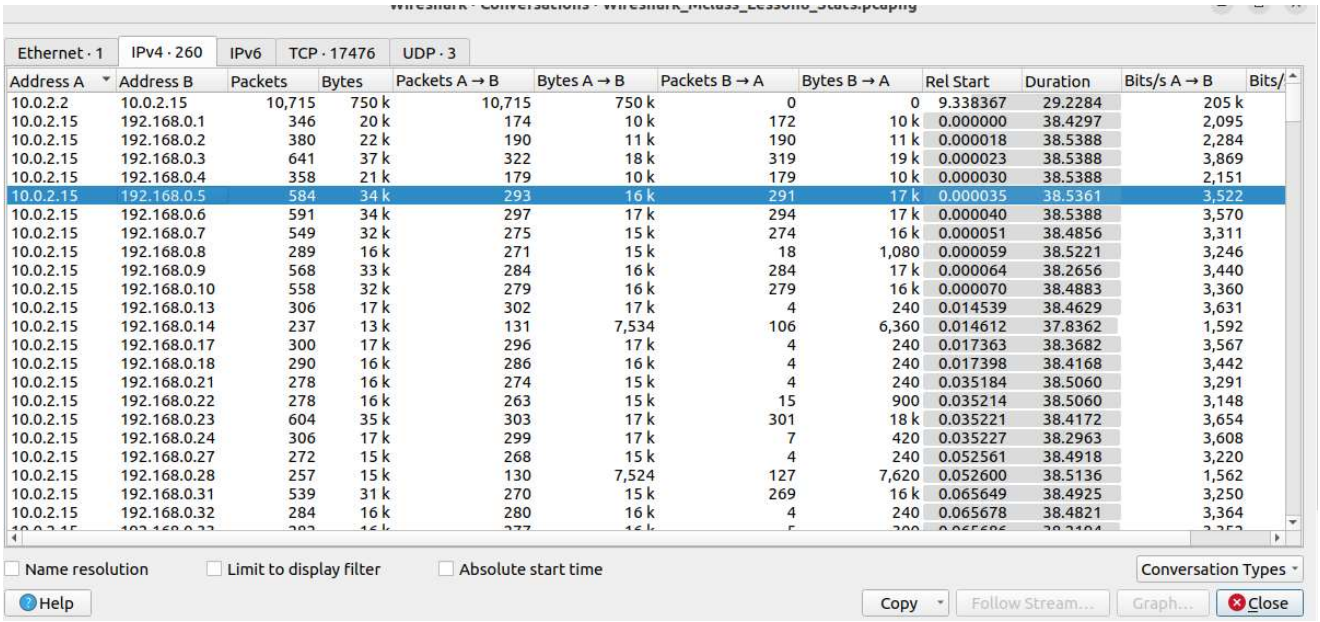
Using trace file:

uploaded in google class room with name: Wireshark_Mclass_Lesson8_Stats.pcapng

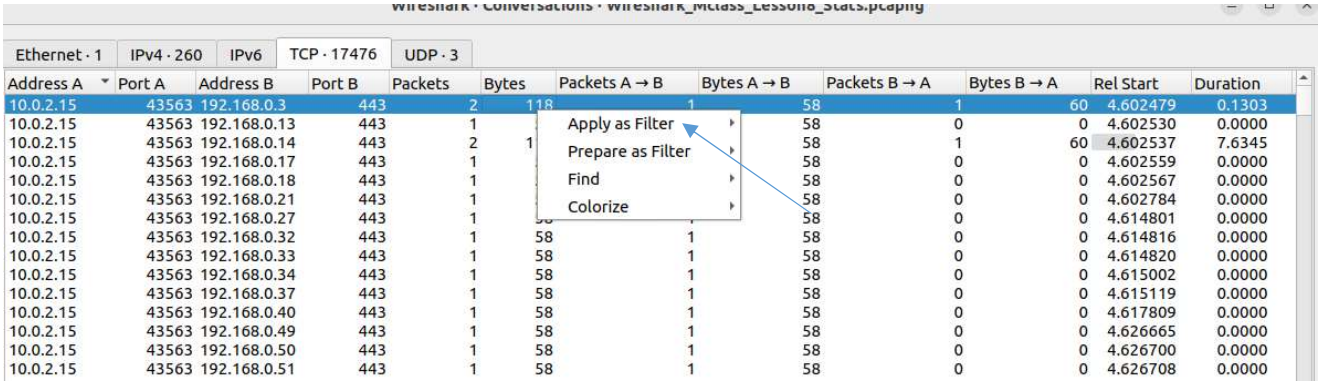
Open this file in wireshark.

Go to:

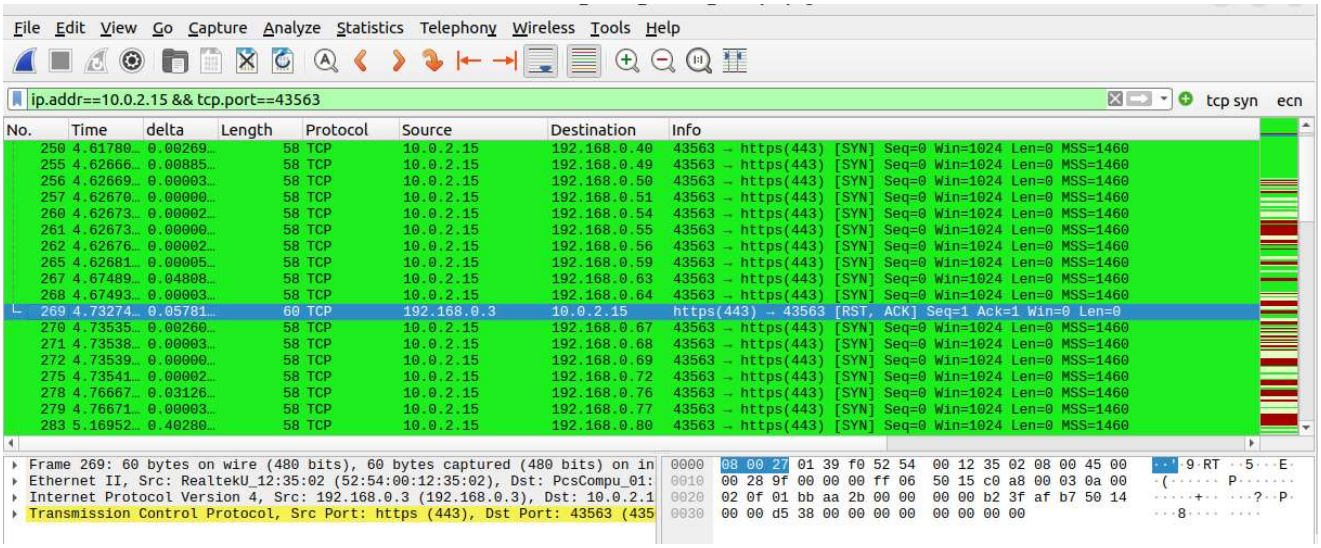




Adding filter from the conversation and viewing in the packet list:



It looks as follows:



EXPERIMENT-5

Network Simulation Using NS2 Simulator

Network Simulator 2 (NS2): Features & Basic Architecture Of NS2

1. What is NS2

NS2 stands for Network Simulator Version 2. It is an open-source event-driven simulator designed specifically for research in computer communication networks.

2. Features of NS2

Discrete Event Simulation: NS2 uses discrete event simulation to model the behavior of network protocols and applications. It simulates events such as packet transmissions, receptions, and protocol state changes at discrete points in time.

Modular Architecture: NS2 is designed with a modular architecture, allowing users to extend and customize its functionality by adding new protocols, algorithms, and network models. This makes it highly flexible and adaptable to different research needs.

Support for Various Protocols: NS2 includes support for a wide range of network protocols, including TCP, UDP, IP, routing protocols (such as OSPF, BGP, and RIP), transport layer protocols (such as TCP and UDP variants), and application layer protocols (such as FTP, HTTP, and SMTP).

Wireless Network Support: NS2 provides support for simulating wireless networks, including mobile ad hoc networks (MANETs), wireless sensor networks (WSNs), and wireless LANs (WLANs). It includes models for radio propagation, mobility, and energy consumption in wireless devices.

Visualization Tools: NS2 comes with built-in visualization tools that allow users to visualize and analyze simulation results. These tools provide graphical representations of network topologies, packet traces, and performance metrics, helping users to understand the behavior of their simulated networks.

Scripting Support: NS2 is primarily scripted using the Tool Command Language (TCL), which allows users to define network topologies, configure simulation parameters, and specify traffic patterns. Additionally, users can write C++ code to extend NS2's functionality or implement custom protocols.

Community Support: NS2 has a large and active user community, with many resources available online, including documentation, tutorials, and user forums. This community support makes it easier for users to get started with NS2, troubleshoot problems, and collaborate with other researchers.

Open Source: NS2 is open-source software, distributed under the GNU General Public License (GPL). This means that users have access to the source code and can modify it according to their needs. It also encourages collaboration and contributions from the research community.

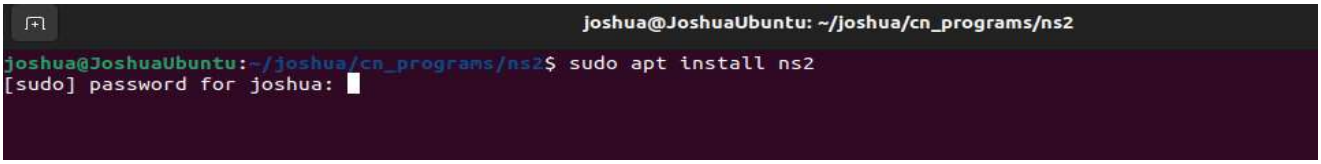
NS2 (Network Simulator 2) is primarily written in two languages: C++ and OTcl (Object-oriented Tcl).

C++: The core of NS2, including the simulation engine and many built-in models and protocols, is implemented in C++. C++ provides efficiency and low-level control, making it well-suited for tasks such as packet processing and network simulation.

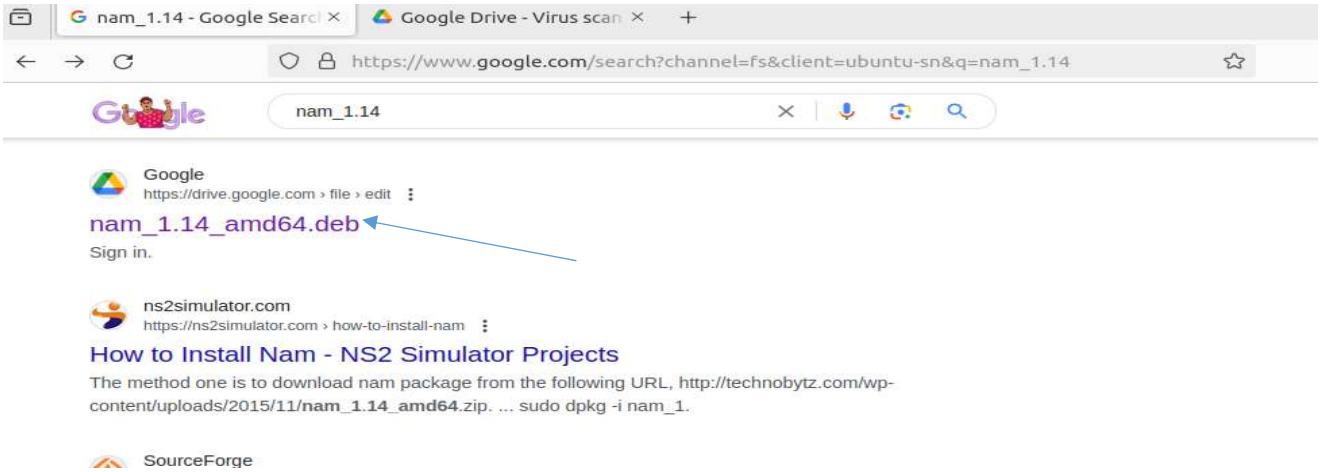
OTcl: OTcl is an extension of the Tcl (Tool Command Language) scripting language that adds object-oriented features. NS2 uses OTcl for configuration and scripting purposes. Users can define network topologies, configure simulation parameters, and specify protocols and applications using OTcl scripts.

Installing NS2:

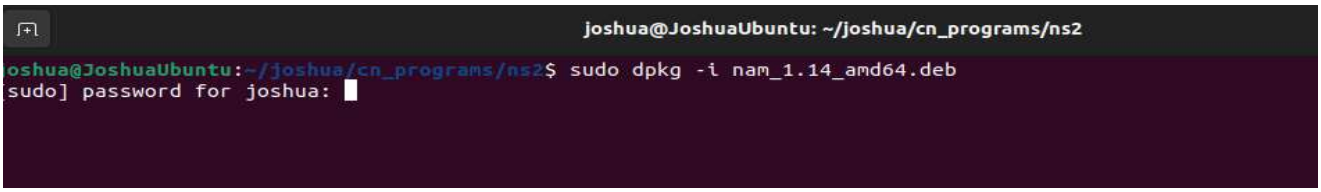
Step 1: Open the terminal and execute the following command



Step 2: Go to google and type as follows, download the file from the google drive link.

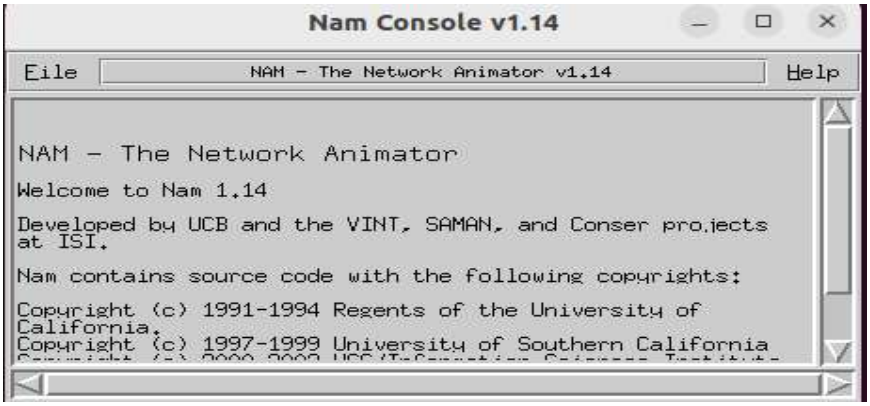


Step 3: Go to the folder where you have the downloaded file and open terminal and type the following command.



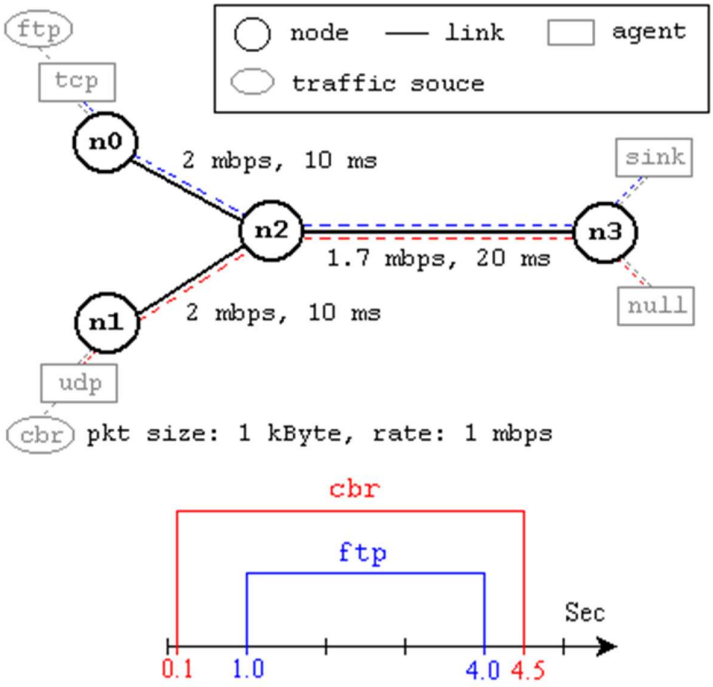
Step 4: Now, type the following command

A console opens as follows:



The installation is successful, and you can now start doing your programs.

Create a simple network configuration as follows and simulate the following network



- This network consists of 4 nodes (n0, n1, n2, n3) as shown in above figure.
- The duplex links between n0 and n2, and n1 and n2 have 2 Mbps of bandwidth and 10 ms of delay. The duplex link between n2 and n3 has 1.7 Mbps of bandwidth and 20 ms of delay.
- Each node uses a DropTail queue, of which the maximum size is 10. A "tcp" agent is attached to n0, and a connection is established to a tcp "sink" agent attached to n3.
- As default, the maximum size of a packet that a "tcp" agent can generate is 1KByte.
- A tcp "sink" agent generates and sends ACK packets to the sender (tcp agent) and frees the received packets.
- A "udp" agent that is attached to n1 is connected to a "null" agent attached to n3. A "null" agent just frees the packets received.
- A "ftp" and a "cbr" traffic generator are attached to "tcp" and "udp" agents respectively, and the "cbr" is configured to generate 1 KByte packets at the rate of 1 Mbps. The "cbr" is set to start at 0.1 sec and stop at 4.5 sec, and "ftp" is set to start at 1.0 sec and stop at 4.0 sec.

Program Code:

Save it as ns-simple.tcl as shown below:

Code:

```
#Create a simulator object
set ns [new Simulator]

#Define different colors for data flows (for NAM)
$ns color 1 Blue
$ns color 2 Red

#Open the NAM trace file
set nf [open out.nam w]
$ns namtrace-all $nf

#Define a 'finish' procedure
proc finish {} {
        global ns nf
        $ns flush-trace
        #Close the NAM trace file
        close $nf
        #Execute NAM on the trace file
        exec nam out.nam &
        exit 0
}

#Create four nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]

#Create links between the nodes
$ns duplex-link $n0 $n2 2Mb 10ms DropTail
$ns duplex-link $n1 $n2 2Mb 10ms DropTail
$ns duplex-link $n2 $n3 1.7Mb 20ms DropTail

#Set Queue Size of link (n2-n3) to 10
$ns queue-limit $n2 $n3 10

#Give node position (for NAM)
$ns duplex-link-op $n0 $n2 orient right-down
$ns duplex-link-op $n1 $n2 orient right-up
$ns duplex-link-op $n2 $n3 orient right

#Monitor the queue for link (n2-n3). (for NAM)
$ns duplex-link-op $n2 $n3 queuePos 0.5


#Setup a TCP connection
set tcp [new Agent/TCP]
$tcp set class_ 2
$ns attach-agent $n0 $tcp
set sink [new Agent/TCPSink]
$ns attach-agent $n3 $sink
$ns connect $tcp $sink
$tcp set fid_ 1

#Setup a FTP over TCP connection
set ftp [new Application/FTP]
$ftp attach-agent $tcp
$ftp set type_ FTP


#Setup a UDP connection
set udp [new Agent/UDP]
$ns attach-agent $n1 $udp
set null [new Agent/Null]
$ns attach-agent $n3 $null
```

```
$ns connect $udp $null
$udp set fid_ 2

#Setup a CBR over UDP connection
set cbr [new Application/Traffic/CBR]
$cbr attach-agent $udp
$cbr set type_ CBR
$cbr set packet_size_ 1000
$cbr set rate_ 1mb
$cbr set random_ false


#Schedule events for the CBR and FTP agents
$ns at 0.1 "$cbr start"
$ns at 1.0 "$ftp start"
$ns at 4.0 "$ftp stop"
$ns at 4.5 "$cbr stop"

#Detach tcp and sink agents (not really necessary)
$ns at 4.5 "$ns detach-agent $n0 $tcp ; $ns detach-agent $n3 $sink"

#Call the finish procedure after 5 seconds of simulation time
$ns at 5.0 "finish"

#Print CBR packet size and interval
puts "CBR packet size = [$cbr set packet_size_]"
puts "CBR interval = [$cbr set interval_]"

#Run the simulation
$ns run
```

Explanation:

set ns [new Simulator]: This line creates a new ns (Network Simulator) object named ns, which will be used to define and simulate the network topology.

$ns color 1 Blue and $ns color 2 Red: These lines define different colors for data flows in the Network Animator (NAM) visualization tool. Blue is assigned to flow 1, and red is assigned to flow 2.

set nf [open out.nam w] and $ns namtrace-all $nf: These lines open a trace file named "out.nam" and configure the ns object to trace all events in the simulation to this file using the Network Animator (NAM).

proc finish {} { ... }: This block of code defines a Tcl procedure named finish. It flushes the trace, closes the trace file, executes NAM on the trace file, and exits the simulation.

set n0 [$ns node], set n1 [$ns node], etc.: These lines create four nodes in the network simulation and store references to them in variables n0, n1, n2, and n3.

$ns duplex-link $n0 $n2 2Mb 10ms DropTail, etc.: These lines create duplex links between the nodes with specified bandwidths, delays, and queuing disciplines (DropTail).

$ns queue-limit $n2 $n3 10: This line sets the queue size of the link between nodes n2 and n3 to 10 packets.

$ns duplex-link-op $n0 $n2 orient right-down, etc.: These lines position the nodes and links in the NAM visualization.

$ns duplex-link-op $n2 $n3 queuePos 0.5: This line positions the queue monitor for the link between nodes n2 and n3.

set tcp [new Agent/TCP], set sink [new Agent/TCPSink], etc.: These lines create TCP and UDP agents for data transmission and reception.

$ns attach-agent $n0 $tcp, etc.: These lines attach the TCP and UDP agents to the corresponding nodes.

$ns connect $tcp $sink: This line establishes a connection between the TCP agent and the TCP sink agent.

$tcp set fid_ 1, $udp set fid_ 2: These lines set flow identifiers for TCP and UDP agents.

set ftp [new Application/FTP], set cbr [new Application/Traffic/CBR]: These lines create FTP and CBR (Constant Bit Rate) traffic applications.

$ftp attach-agent $tcp, $cbr attach-agent $udp: These lines attach the FTP and CBR applications to the corresponding agents.

$cbr set packet_size_ 1000, $cbr set rate_ 1mb, etc.: These lines configure the parameters for the CBR traffic, including packet size, rate, and whether the traffic is generated randomly.

$ns at 0.1 "$cbr start", etc.: These lines schedule events for starting and stopping the CBR and FTP traffic.

$ns at 5.0 "finish": This line schedules the finish procedure to be executed after 5 seconds of simulation time.

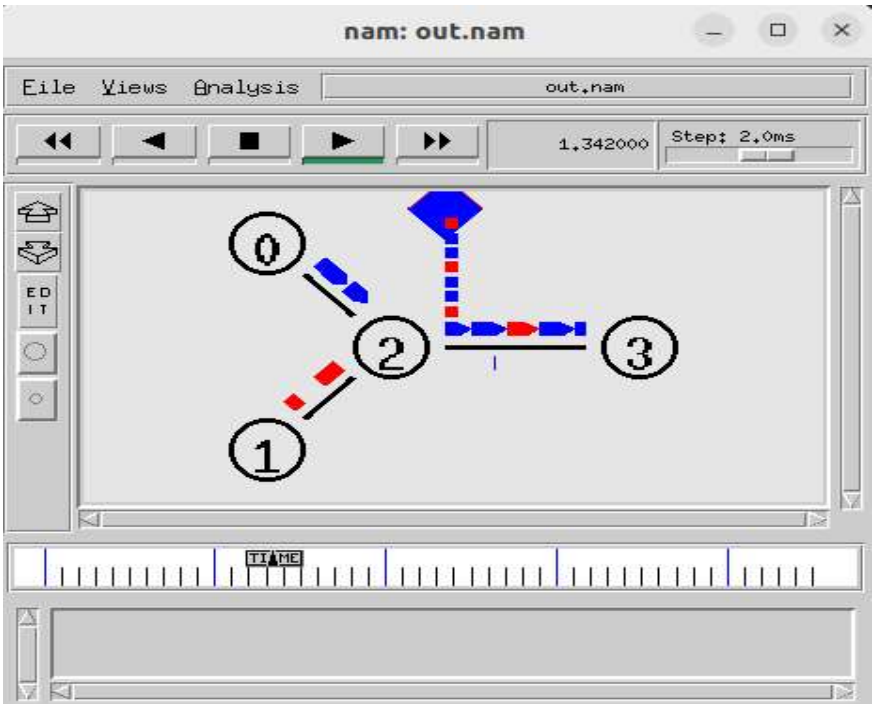
puts "CBR packet size = [$cbr set packet_size_]", etc.: These lines print out the CBR packet size and interval.

$ns run: This line starts the simulation.

Execution:

```
joshua@JoshuaUbuntu:~/joshua/cn_programs/ns2$ ns ns-simple.tcl
CBR packet size = 1000
CBR interval = 0.0080000000000000002
```

Now, the animation starts as follows:



EXPERIMENT-6

Simulating routing algorithms using ns2 simulator.

1. Simulate Distance Vector Routing Algorithm using ns2 Simulator.

Code:

```
set ns [new Simulator]
set nr [open thro.tr w]
$ns trace-all $nr
set nf [open thro.nam w]
$ns namtrace-all $nf
proc finish { } {
global ns nr nf
$ns flush-trace
close $nf
close $nr
exec nam thro.nam &
exit 0
}
for { set i 0 } { $i < 12} { incr i 1 } {
set n($i) [$ns node]}
for {set i 0} {$i < 8} {incr i} {
$ns duplex-link $n($i) $n([expr $i+1]) 1Mb 10ms DropTail }
$ns duplex-link $n(0) $n(8) 1Mb 10ms DropTail
$ns duplex-link $n(1) $n(10) 1Mb 10ms DropTail
$ns duplex-link $n(0) $n(9) 1Mb 10ms DropTail
$ns duplex-link $n(9) $n(11) 1Mb 10ms DropTail
$ns duplex-link $n(10) $n(11) 1Mb 10ms DropTail
$ns duplex-link $n(11) $n(5) 1Mb 10ms DropTail
set udp0 [new Agent/UDP]
$ns attach-agent $n(0) $udp0
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0
set null0 [new Agent/Null]
$ns attach-agent $n(5) $null0
$ns connect $udp0 $null0
set udp1 [new Agent/UDP]
$ns attach-agent $n(1) $udp1
set cbr1 [new Application/Traffic/CBR]
$cbr1 set packetSize_ 500
$cbr1 set interval_ 0.005
$cbr1 attach-agent $udp1
set null0 [new Agent/Null]
$ns attach-agent $n(5) $null0
$ns connect $udp1 $null0
$ns rtproto DV
$ns rtmodel-at 10.0 down $n(11) $n(5)
$ns rtmodel-at 15.0 down $n(7) $n(6)
$ns rtmodel-at 30.0 up $n(11) $n(5)
$ns rtmodel-at 20.0 up $n(7) $n(6)
$udp0 set fid_ 1
$udp1 set fid_ 2
$ns color 1 Red
$ns color 2 Green
$ns at 1.0 "$cbr0 start"
$ns at 2.0 "$cbr1 start"
$ns at 45 "finish"
$ns run
```

Explanation:

set ns [new Simulator]: This line creates a new simulation instance and assigns it to the variable ns. The new keyword is likely a function or command provided by the NS tool to instantiate a simulation object.

set nr [open thro.tr w]: This line opens a file named "thro.tr" in write mode and assigns the file handle to the variable nr. The open command in Tcl is used for file I/O operations.

$ns trace-all $nr: This line instructs the NS simulator ($ns) to trace all events in the simulation and write them to the file represented by the file handle $nr.

set nf [open thro.nam w]: Similar to line 2, this line opens another file named "thro.nam" in write mode and assigns the file handle to the variable nf.

$ns namtrace-all $nf: This line tells the NS simulator to trace all nam (Network Animator) events in the simulation and write them to the file represented by the file handle $nf.

proc finish { } { ... }: This block defines a Tcl procedure named finish. Procedures in Tcl are similar to functions in other programming languages. This procedure doesn't take any arguments. Inside the procedure, it does the following tasks:

$ns flush-trace: Flushes the trace information to ensure all data is written to the trace file.

close $nf and close $nr: Closes the trace files.

exec nam thro.nam &: Executes the Network Animator (nam) with the "thro.nam" trace file as input in the background.

exit 0: Exits the Tcl interpreter with a status code of 0.

The next two for loops are used to create nodes for the simulation and connect them with links. They create nodes and duplex links between them, defining characteristics like bandwidth, delay, and queue discipline (in this case, DropTail).

set udp0 [new Agent/UDP]: Creates a new UDP agent and assigns it to the variable udp0.

$ns attach-agent $n(0) $udp0: Attaches the UDP agent udp0 to the node n(0) in the simulation.

set cbr0 [new Application/Traffic/CBR]: Creates a new Constant Bit Rate (CBR) traffic application and assigns it to the variable cbr0.

$cbr0 set packetSize_ 500 and $cbr0 set interval_ 0.005: Sets the packet size and interval for the CBR traffic application cbr0.

$cbr0 attach-agent $udp0: Attaches the UDP agent udp0 to the CBR traffic application cbr0.

set null0 [new Agent/Null]: Creates a new null agent and assigns it to the variable null0.

$ns attach-agent $n(5) $null0: Attaches the null agent null0 to the node n(5) in the simulation.

$ns connect $udp0 $null0: Connects the UDP agent udp0 to the null agent null0.

Similar operations are repeated for udp1, cbr1, and null1.

$ns rtproto DV: Sets the routing protocol used in the simulation to Distance Vector (DV).

$ns rtmodel-at: Defines routing model changes at specific times in the simulation.

$udp0 set fid_ 1 and $udp1 set fid_ 2: Sets flow IDs for UDP agents.

$ns color 1 Red and $ns color 2 Green: Sets colors for visualization in the Network Animator.

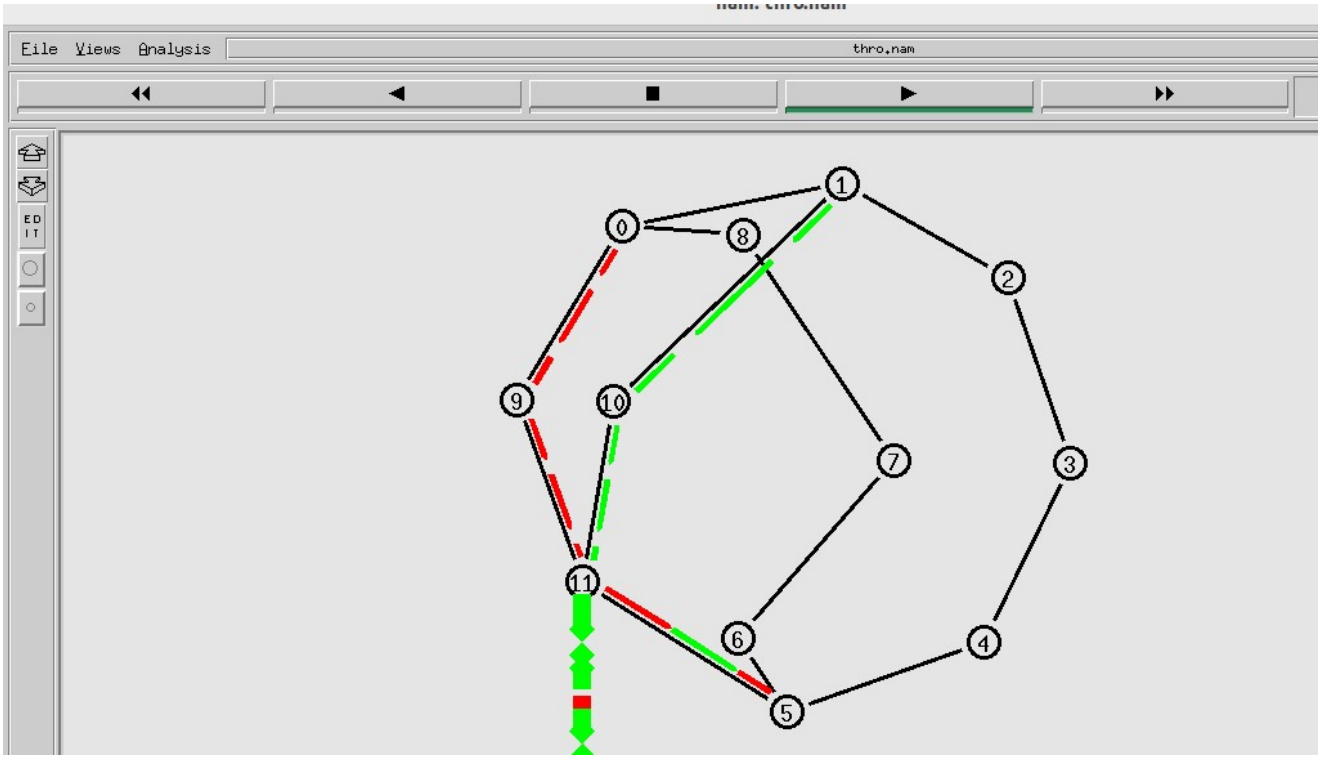
$ns at ...: Schedules events to occur at specific simulation times.

$ns run: Initiates the simulation.

Execution:

```
joshua@JoshuaUbuntu:~/joshua/cn_programs/ns2$ ns dvr.tcl
```

You need to wait for a while so that, the complete network operation can be viewed, as the packet transfer between neighboring nodes takes some time.



2. Simulate Link State Routing Algorithm using ns2 Simulator.

Code:

```
set ns [new Simulator]
set nr [open thro.tr w]
$ns trace-all $nr
set nf [open thro.nam w]
$ns namtrace-all $nf
proc finish { } {
global ns nr nf
$ns flush-trace
close $nf
close $nr
exec nam thro.nam &
exit 0
}
for { set i 0 } { $i < 12} { incr i 1 } {
set n($i) [$ns node]}
for {set i 0} {$i < 8} {incr i} {
$ns duplex-link $n($i) $n([expr $i+1]) 1Mb 10ms DropTail }
```

```
$ns duplex-link $n(0) $n(8) 1Mb 10ms DropTail
$ns duplex-link $n(1) $n(10) 1Mb 10ms DropTail
$ns duplex-link $n(0) $n(9) 1Mb 10ms DropTail
$ns duplex-link $n(9) $n(11) 1Mb 10ms DropTail
$ns duplex-link $n(10) $n(11) 1Mb 10ms DropTail
$ns duplex-link $n(11) $n(5) 1Mb 10ms DropTail
set udp0 [new Agent/UDP]
$ns attach-agent $n(0) $udp0
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0
set null0 [new Agent/Null]
$ns attach-agent $n(5) $null0
$ns connect $udp0 $null0
set udp1 [new Agent/UDP]
$ns attach-agent $n(1) $udp1
set cbr1 [new Application/Traffic/CBR]
$cbr1 set packetSize_ 500
$cbr1 set interval_ 0.005
$cbr1 attach-agent $udp1
set null0 [new Agent/Null]
$ns attach-agent $n(5) $null0
$ns connect $udp1 $null0
$ns rtproto LS
$ns rtmodel-at 10.0 down $n(11) $n(5)
$ns rtmodel-at 15.0 down $n(7) $n(6)
$ns rtmodel-at 30.0 up $n(11) $n(5)
$ns rtmodel-at 20.0 up $n(7) $n(6)
$udp0 set fid_ 1
$udp1 set fid_ 2
$ns color 1 Red
$ns color 2 Green
$ns at 1.0 "$cbr0 start"
$ns at 2.0 "$cbr1 start"
$ns at 45 "finish"
$ns run
```

Explanation:

$ns rtmodel-at 10.0 down $n(11) $n(5): This line specifies a routing model change at simulation time 10.0 where the link between node 11 ($n(11)) and node 5 ($n(5)) is going down. In network simulations, this indicates that the link is becoming unavailable or is being taken offline, which will trigger the routing protocol to update its routing tables accordingly.

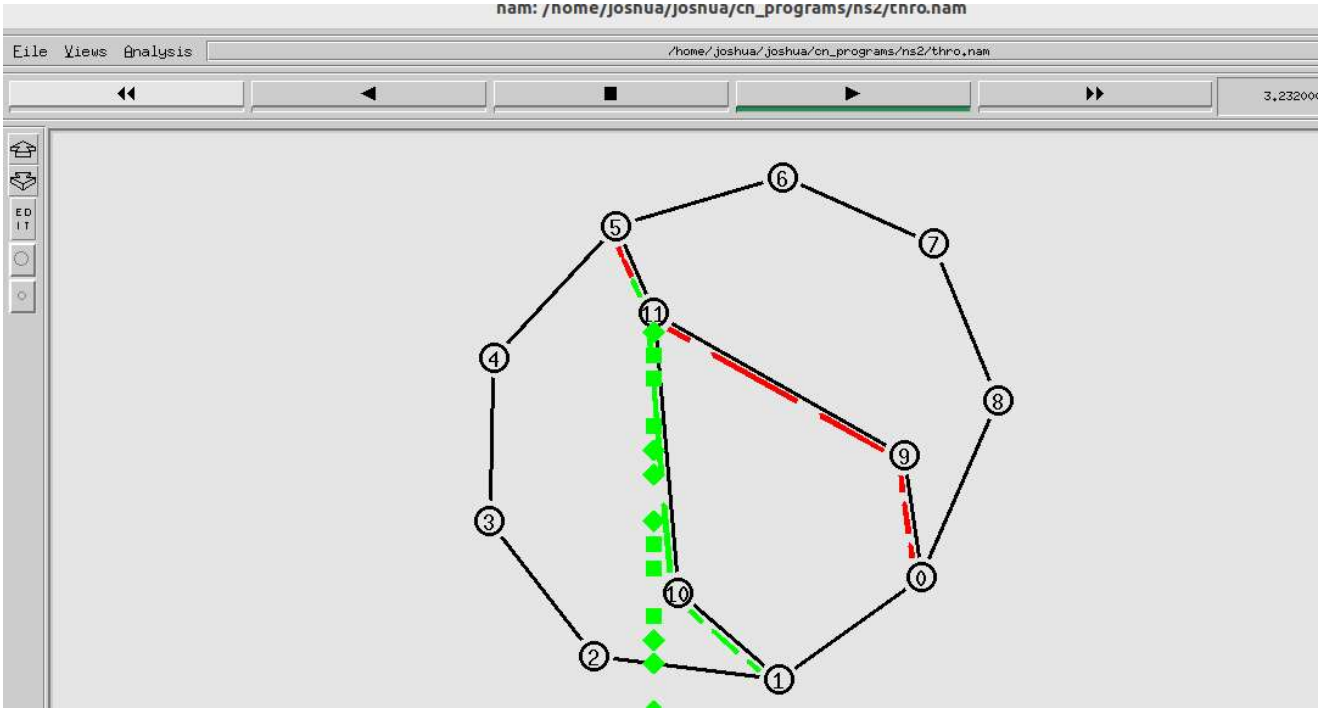
$ns rtmodel-at 15.0 down $n(7) $n(6): Similarly, this line specifies a routing model change at time 15.0 where the link between node 7 ($n(7)) and node 6 ($n(6)) is going down. Again, this indicates a change in the network topology, requiring the routing protocol to adjust its routing tables.

$ns rtmodel-at 30.0 up $n(11) $n(5): This line indicates a routing model change at time 30.0 where the link between node 11 ($n(11)) and node 5 ($n(5)) is coming back up. After being down, the link is now restored or becomes available again. This change will prompt the routing protocol to update its routing tables to reflect the restored link.

$ns rtmodel-at 20.0 up $n(7) $n(6): Similarly, this line specifies a routing model change at time 20.0 where the link between node 7 ($n(7)) and node 6 ($n(6)) is coming back up. After being down, the link is now restored or becomes available again, triggering the routing protocol to update its routing tables.

Execution:

```
joshua@JoshuaUbuntu:~/joshua/cn_programs/ns2$ ns lsr.tcl
```


nam: /nome/josnua/josnua/cn_programs/nsz/cnro.nam
File Views Analysis
/home/joshua/joshua/cn_programs/ns2/thro.nam
EDIT
6
5
7
11
4
8
9
3
0
10
2
1

EXPERIMENT-7

i) Write a socket program to implement raw sockets.

Raw Sockets:

A raw socket is a type of network socket that provides applications with direct access to the underlying transport protocol. Unlike standard sockets, which abstract away much of the networking protocol's complexity, raw sockets allow programs to directly interact with the network layer. This grants the ability to send and receive network packets without any protocol-specific transport layer formatting or processing.

Key Characteristics of Raw Sockets

1. **Direct Packet Access**: Raw sockets allow an application to read and write packets directly to and from the network interface.
2. **Bypassing the Transport Layer**: They bypass the usual TCP/UDP transport layer processing, giving control over the packet's headers, such as IP and ICMP.
3. **Customization and Control**: Applications can craft packets with custom headers and payloads, making raw sockets useful for network diagnostics, security tools, and custom network protocol implementations.
4. **Higher Privileges**: Operating systems often require elevated privileges to create raw sockets due to their powerful and potentially disruptive capabilities.

Uses of Raw Sockets

- **Network Diagnostic Tools**: Tools like ping and traceroute use raw sockets to send ICMP packets.
- **Security Applications**: Packet sniffers and network intrusion detection systems (like Wireshark and Snort) use raw sockets to monitor and analyze network traffic.
- **Custom Protocol Implementation**: Developers implementing protocols not natively supported by the operating system may use raw sockets.
- **Testing and Debugging**: They can be used to generate and test packets in a controlled environment for protocol compliance and robustness testing.

Program:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<netinet/ip.h>
#include<sys/socket.h>
#include<arpa/inet.h>

int main() {
      struct sockaddr_in source_socket_address, dest_socket_address;
//Structures to hold source and destination socket addresses.
   int packet_size;
//Variable to store the size of received packets.
   unsigned char *buffer = (unsigned char *)malloc(65536);
//Allocates 65536 bytes of memory for buffer to store packet data.
   int sock = socket (PF_INET, SOCK_RAW, IPPROTO_TCP);
//Creates a raw socket using IPv4 protocol family (PF_INET), raw socket type (SOCK_RAW), and TCP protocol
//(IPPROTO_TCP).
   if(sock == -1)
   {
      perror("Failed to create socket");
      exit(1);
```

```
    }
    while(1) {
      packet_size = recvfrom(sock , buffer , 65536 , 0 , NULL, NULL);
//Receives packets using recvfrom and stores them in buffer. The size of the received packet is stored in
//packet_size.
     if (packet_size == -1) {
       printf("Failed to get packets\n");
       return 1;
      }

      struct iphdr *ip_packet = (struct iphdr *)buffer;
      //Casts the buffer to an iphdr structure pointer to interpret the buffer as an IP packet.
      memset(&source_socket_address, 0, sizeof(source_socket_address));
     //Clears the memory for source_socket_address and dest_socket_address structures.
      source_socket_address.sin_addr.s_addr = ip_packet->saddr;
     //Assigns the source address from the IP header to source_socket_address.
      memset(&dest_socket_address, 0, sizeof(dest_socket_address));
      dest_socket_address.sin_addr.s_addr = ip_packet->daddr;

      printf("Incoming Packet: \n");
      printf("Packet Size (bytes): %d\n",ntohs(ip_packet->tot_len));
      printf("Source Address: %s\n", (char *)inet_ntoa(source_socket_address.sin_addr));
      printf("Destination Address: %s\n", (char *)inet_ntoa(dest_socket_address.sin_addr));
      printf("Identification: %d\n\n", ntohs(ip_packet->id));
    }

    return 0;
}
```

Output:

```
joshua@JoshuaUbuntu:~/joshua/cn_programs$ cc raw_sockets.c
joshua@JoshuaUbuntu:~/joshua/cn_programs$ sudo ./a.out
Incoming Packet:
Packet Size (bytes): 40
Source Address: 142.250.192.99
Destination Address: 10.0.2.15
Identification: 8305

Incoming Packet:
Packet Size (bytes): 79
Source Address: 142.250.192.99
Destination Address: 10.0.2.15
Identification: 8306
```

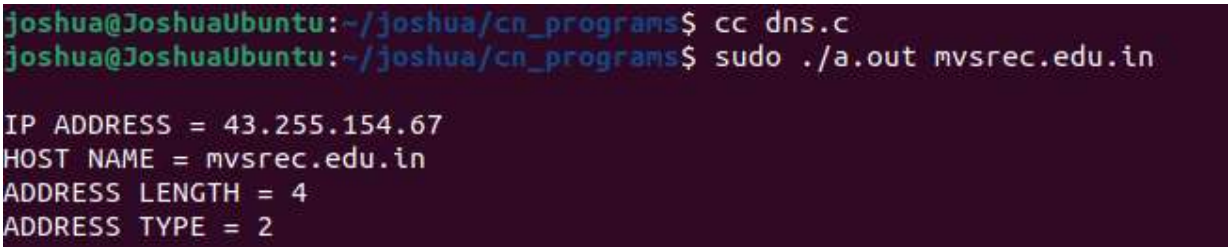
ii) Program to implement DNS

Program:

```
#include<stdio.h>

#include<netdb.h>

#include<arpa/inet.h>

#include<netinet/in.h>

#include<stdlib.h>
```

```
int main(int argc, char** argv)
{
   struct hostent *host;
   struct in_addr h_addr;

   if (argc != 2)
   {
      fprintf(stderr, "USAGE: nslookup <hostname>\n");
      exit(1); // Exit the program if the usage is incorrect
   }

   if ((host = gethostbyname(argv[1])) == NULL)
   {
      fprintf(stderr, "(mini)nslookup failed on %s\n", argv[1]);
      exit(1); // Exit the program if the lookup fails
   }

   h_addr.s_addr = *((unsigned long*)host->h_addr_list[0]);
   printf("\nIP ADDRESS = %s\n", inet_ntoa(h_addr));
   printf("HOST NAME = %s\n", host->h_name);
   printf("ADDRESS LENGTH = %d\n", host->h_length);
   printf("ADDRESS TYPE = %d\n", host->h_addrtype);
   return 0;
}
```

Output:



EXPERIMENT-8

Simulate the following advanced socket system calls

i) **getsockopt** and **setsockopt**

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
int main() {
    int sockfd;
    int optval;
    socklen_t optlen = sizeof(optval);
    struct sockaddr_in servaddr;
    // Create socket
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        perror("socket");
        exit(EXIT_FAILURE);
    }
    // Set socket option SO_REUSEADDR
    optval = 1;
    if (setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof(optval)) < 0) {
        perror("setsockopt");
        close(sockfd);
        exit(EXIT_FAILURE);
    }
    // Get and print socket option SO_REUSEADDR
    if (getsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &optval, &optlen) < 0) {
        perror("getsockopt");
        close(sockfd);
```

```
        exit(EXIT_FAILURE);
    }
    printf("SO_REUSEADDR is %s\n", (optval ? "ON" : "OFF"));

    // Bind socket to an address and port
    memset(&servaddr, 0, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port = htons(8080);
    if (bind(sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr)) < 0) {
        perror("bind");
        close(sockfd);
        exit(EXIT_FAILURE);
    }
    // Listen for incoming connections
    if (listen(sockfd, 5) < 0) {
        perror("listen");
        close(sockfd);
        exit(EXIT_FAILURE);
    }
    printf("Server is listening on port 8080\n");
    // Close the socket
    close(sockfd);
    return 0;
}
```

Output:

ii) ioctl

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ioctl.h>
#include <net/if.h>
#include <arpa/inet.h>
#include <unistd.h>

int main() {
    int sockfd;
    struct ifreq ifr;
    struct sockaddr_in *addr;

    // Create a socket
    sockfd = socket(AF_INET, SOCK_DGRAM, 0); // not intended for communication but for interfacing with the network device.
    if (sockfd < 0) {
        perror("socket");
        exit(EXIT_FAILURE);
    }

    // Specify the interface name
    strncpy(ifr.ifr_name, "enp0s3", IFNAMSIZ-1);

    // Get the current IP address
    if (ioctl(sockfd, SIOCGIFADDR, &ifr) < 0) {
        perror("ioctl(SIOCGIFADDR)");
        close(sockfd);
        exit(EXIT_FAILURE);
    }
```

```
    addr = (struct sockaddr_in *)&ifr.ifr_addr;
    printf("Current IP address: %s\n", inet_ntoa(addr->sin_addr));
```

// convert the binary representation of an IP address (which is how addresses are stored in struct in_addr) into a string that is easily readable by humans.

```
    // Set a new IP address
    addr->sin_family = AF_INET;
    inet_pton(AF_INET, "192.168.1.100", &addr->sin_addr); //text to binary format

    if (ioctl(sockfd, SIOCSIFADDR, &ifr) < 0) {
        perror("ioctl(SIOCSIFADDR)");
        close(sockfd);
        exit(EXIT_FAILURE);
    }

    printf("New IP address set to 192.168.1.100\n");

    // Close the socket
    close(sockfd);

    return 0;
}
```

Output:

```
joshua@JoshuaUbuntu:~/joshua/cn_programs/advanced sockets$ cc ioctl.c
joshua@JoshuaUbuntu:~/joshua/cn_programs/advanced sockets$ ./a.out
Current IP address: 10.0.2.15
ioctl(SIOCSIFADDR): Operation not permitted
```

iii) select

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/select.h>
#include <sys/time.h>
#include <fcntl.h>

int main() {
```

```
    int fd1, fd2, nfds; // file descriptors for the files to be monitored
    fd_set readfds; // set of file descriptors to be monitored for readability.
    struct timeval timeout; // maximum time to wait for any file descriptor to become ready.


    // Open two files (or devices)
// fd1 and fd2 are opened for reading.
    fd1 = open("file1.txt", O_RDONLY);
    if (fd1 < 0) {
        perror("open file1");
        exit(EXIT_FAILURE);
    }

    fd2 = open("file2.txt", O_RDONLY);
    if (fd2 < 0) {
        perror("open file2");
        close(fd1);
        exit(EXIT_FAILURE);
    }
     // Initialize the file descriptor set
// FD_ZERO initializes the set of file descriptors.
    FD_ZERO(&readfds);
//FD_SET adds fd1 and fd2 to the set.

    FD_SET(fd1, &readfds);
    FD_SET(fd2, &readfds);

    // Determine the highest file descriptor
//nfds is set to the highest file descriptor value plus one, which is required for the select call.
    nfds = (fd1 > fd2 ? fd1 : fd2) + 1;

    // Set the timeout value (e.g., 5 seconds)
    timeout.tv_sec = 5;
    timeout.tv_usec = 0;

    // Call select
// select is called to monitor fd1 and fd2 for readability with a 5-second timeout.
    int ret = select(nfds, &readfds, NULL, NULL, &timeout);
    if (ret == -1) {
        perror("select");
        close(fd1);
        close(fd2);
        exit(EXIT_FAILURE);
    } else if (ret == 0) {
        printf("Timeout occurred! No file descriptor is ready.\n");
        } else {
        if (FD_ISSET(fd1, &readfds)) {
            printf("File descriptor %d is ready to read.\n", fd1);
        }
        if (FD_ISSET(fd2, &readfds)) {
            printf("File descriptor %d is ready to read.\n", fd2);
        }
    }

    // Close the file descriptors
    close(fd1);
    close(fd2);
```

```
    return 0;
}
```

Note: Need to create two text files with some text before executing the program.

Output: